

A REPORT
ON
APPLYING ML ON RT-IOT '22 DATASET

BY

Sandeep Venkata Kollipara
9067-37236

FOR

Final Term Project
CS 5805 – Machine Learning
Dr. Reza Jafari

AT



VIRGINIA POLYTECHNIC INSTITUTE AND STATE UNIVERSITY

10th May 2025

A REPORT
ON
APPLYING MACHINE LEARNING ON REAL TIME - INTERNET OF
THINGS 2022 DATASET

BY

Sandeep V Kollipara

9067-37236

M.Eng. CSA

Prepared in partial fulfillment of the
Graduate Course
CS 5805 - Machine Learning
taught by Dr. Reza Jafari

AT



VIRGINIA POLYTECHNIC INSTITUTE AND STATE UNIVERSITY

10th May 2025

Abstract

This report details the application of machine learning on RT-IoT 2022 dataset that involves network traffic features extracted from an experiment across multiple IoT devices that are subject to attacks. The objective is to use the data obtained from the experiment to train a machine learning model capable of predicting the nature of attack based on the features obtained from the IoT devices. Best practices are followed to ensure that collinear and low variance features are eliminated before being fitted to the model. Stratified cross validation is conducted to ensure that the model is generalizing and not overfitting. Also, multiple statistical methods were employed across data preprocessing, feature selection, regression and classification analysis to scrutinize the data well. A descriptive step-by-step process of the flow alongside comparison of various methods are provided to select the best model for the use case, which is the Post-pruned Decision Tree.

Table of Contents

1. Introduction.....	7
2. Data Description.....	8
3. Phase I: EDA and Preprocessing.....	9
3.1 Exploratory Data Analysis:	10
3.2 Data Preprocessing:.....	11
3.3 Feature Selection:.....	17
4. Phase II: Regression Analysis	19
4.1 Setup.....	19
4.2 Backward Stepwise Linear Regression	19
5. Phase III: Classification Analysis.....	25
5.1 Decision Tree	25
5.1.1 Decision Tree (Original)	25
5.1.2 Decision Tree (Pre-pruned).....	27
5.1.3 Decision Tree (Post-pruned)	29
5.2 Logistic Regression	32
5.2.1 Logistic Regression (One vs. One).....	32
5.2.2 Logistic Regression (One vs. All)	33
5.3 Support Vector Classifier	35
5.3.1 SVC (One vs. One).....	35
5.3.2 SVC (One vs. Rest)	40
5.4 Naïve Bayes	44
5.5 K-Nearest Neighbors.....	46
5.5.1 Elbow method	46
5.6 Neural network.....	49
5.7 ROC-AUC Comparison of all classifiers	50
6. Conclusion: Recommendations.....	52
7. References	52
8. Appendix	53

Table of figures:

Figure 1: Experimental setup for generating RT-IoT '22 dataset	8
Figure 2: Network traffic data retrieved from IoT devices	8
Figure 3: High RE features extracted from the datasets of IoT devices	8
Figure 4: Description of normal/anomalous traffic across the IoT devices.....	9
Figure 5: Count plot of the categorical variable 'proto'	10
Figure 6: Count plot of the categorical variable 'service'	10
Figure 7: Count plot of target variable 'Attack_type'	10
Figure 8: Frequency distribution of target after oversampling with SMOTE	11
Figure 9: Frequency distribution of target after undersampling randomly to 5%	12
Figure 10: Boxplots of 5 features that were redundant after outlier treatment.....	12
Figure 11: Covariance Matrix Heatmap of the feature space	13
Figure 12: Pearson Correlation Coefficient Matrix Heatmap for the feature space	14
Figure 13: Histogram of Cumulative Explained Variance of features after PCA	15
Figure 14: Correlation matrix of transformed features after LDA	16
Figure 15: Separation of classes in LDA across 2 principal components	16
Figure 16: Random Forest Analysis of the feature space before reduction.....	17
Figure 17: Random Forest Analysis of feature space after recursive feature elimination through VIF	18
Figure 18: Model summary of the best fitted linear regression using the 15 selected features	23
Figure 19: Scatter plot of the test and predicted values from Linear Regression model	24
Figure 20: Original Decision Tree without pruning for only 10 features	26
Figure 21: Confusion Matrix of the predictions for Decision Tree (Original)	26
Figure 22: ROC curve for the Decision Tree (Original)	27
Figure 23: Pre-pruned Decision Tree with a simpler schematic.....	28
Figure 24: Confusion Matrix of the predictions for Decision Tree (Pre-pruned).....	28
Figure 25: ROC curve for the Decision Tree (Pre-pruned)	29
Figure 26: Line plot showing the relation between alpha and impurities of leaves	30
Figure 27: Line plot comparison of the test and train accuracies versus CCP alpha	30
Figure 28: Post-pruned Decision Tree still having a complex schematic.....	31
Figure 29: Confusion Matrix of the predictions for Decision Tree (Post-pruned)	31
Figure 30: ROC curve for the Decision Tree (Post-pruned)	32
Figure 31: Confusion Matrix of the predictions for Logistic Regression (OvO)	33
Figure 32: ROC curve for the Logistic Regression (OvR)	33
Figure 33: Confusion Matrix of the predictions for Logistic Regression (OvR).....	34
Figure 34: ROC curve for the Logistic Regression (OvR)	35
Figure 35: Confusion Matrix of the predictions for Linear Kernel SVC (OvO)	36
Figure 36: ROC curve for the Linear Kernel SVC (OvO)	37
Figure 37: Confusion Matrix of the predictions for RBF Kernel SVC (OvO).....	38
Figure 38: ROC curve for the RBF Kernel SVC (OvO)	38
Figure 39: Confusion Matrix of the predictions for Polynomial Kernel SVC (OvO)	39
Figure 40: ROC curve for the Polynomial Kernel SVC (OvO).....	40
Figure 41: Confusion Matrix of the predictions for Linear Kernel SVC (OvR)	41
Figure 42: ROC curve for the Linear Kernel SVC (OvR).....	41
Figure 43: Confusion Matrix of the predictions for RBF Kernel SVC (OvR)	42
Figure 44: ROC curve for the RBF Kernel SVC (OvR).....	43
Figure 45: Confusion Matrix of the predictions for Polynomial Kernel SVC (OvR)	44
Figure 46: ROC curve for the Polynomial Kernel SVC (OvR).....	44
Figure 47: Confusion Matrix of the predictions for Naïve Bayes	45
Figure 48: ROC curve for the Naïve Bayes.....	46
Figure 49: Line plot of error for various k-NN models with increasing k values	47
Figure 50: Confusion Matrix of the predictions for k-Nearest Neighbors	48
Figure 51: ROC curve for the k-Nearest Neighbors.....	48

Figure 52: The schema of the 4-layer neural network.....	49
Figure 53: Confusion Matrix of the predictions for Neural net (4-layer).....	50
Figure 54: ROC curve for the Neural net (4-layer).....	50
Figure 55: ROC overlay for all classifiers.....	51

Table of tables:

Table 1: Iterative t-test, F-test and R-squared analysis in Backward Stepwise Linear Regression	20
Table 2: Model statistics after fitting with the best features.....	23
Table 3: Decision Tree (Original) accuracies for 5-fold stratified cross-validation.....	25
Table 4: Test metrics for the fitted Decision Tree (Original).....	25
Table 5: Decision Tree (Pre-pruned) accuracies for 5-fold stratified cross-validation.....	27
Table 6: Test metrics for the fitted Decision Tree (Pre-pruned)	27
Table 7: Decision Tree (Post-pruned) accuracies for 5-fold stratified cross-validation.....	29
Table 8: Test metrics for the fitted Decision Tree (Post-pruned).....	29
Table 9: Logistic Regression (OvO) accuracies for 5-fold stratified cross-validation	32
Table 10: Test metrics for the fitted Logistic Regression (OvO)	32
Table 11: Logistic Regression (OvR) accuracies for 5-fold stratified cross-validation	33
Table 12: Test metrics for the fitted Logistic Regression (OvR)	34
Table 13: Linear SVC (OvO) accuracies for 5-fold stratified cross-validation.....	35
Table 14: Test metrics for the fitted Linear SVC (OvO).....	36
Table 15: RBF SVC (OvO) accuracies for 5-fold stratified cross-validation.....	37
Table 16: Test metrics for the fitted RBF SVC (OvO).....	37
Table 17: Polynomial SVC (OvO) accuracies for 5-fold stratified cross-validation.....	39
Table 18: Test metrics for the fitted Polynomial SVC (OvO).....	39
Table 19: Linear SVC (OvR) accuracies for 5-fold stratified cross-validation	40
Table 20: Test metrics for the fitted Linear SVC (OvR).....	40
Table 21: RBF SVC (OvR) accuracies for 5-fold stratified cross-validation.....	42
Table 22: Test metrics for the fitted RBF SVC (OvR).....	42
Table 23: Polynomial SVC (OvR) accuracies for 5-fold stratified cross-validation	43
Table 24: Test metrics for the fitted Polynomial SVC (OvR)	43
Table 25: Naïve Bayes accuracies for 5-fold stratified cross-validation	45
Table 26: Test metrics for the fitted Naïve Bayes	45
Table 27: k-Nearest Neighbors accuracies for 5-fold stratified cross-validation	46
Table 28: Test metrics for the fitted k-Nearest Neighbors	46
Table 29: Neural network (4-layer) accuracies for 5-fold stratified cross-validation	49
Table 30: Test metrics for the trained Neural network (4-layer).....	49
Table 31: Comparison of test metrics for all the classifiers	51

1. Introduction

The subject of my final term project for the course CS5805 - Machine Learning is the Real Time - Internet of Things 2022 dataset, also shortened as RT-IoT '22. It is a proprietary dataset derived from real-time IoT infrastructure and was introduced by a Cybersecurity paper in 2023 (Sharmila, 2023). Its features are extracted from network traffic monitoring tools, like Wire shark and Zeek analyzer plus Flowmeter plugin, to monitor IoT devices like Amazon Alexa, Think-Speak-LED, MQTT-Temp, and Wipro Bulbs. An interesting facet concerning its features is that they are acquired not only from the victim device but also the router and the attacking devices as well. The labels of the dataset identify the type of attack experienced by the IoT device if under attack, otherwise the label would mention the device name for normal traffic.

The dataset is primarily meant for training classification and clustering algorithms of machine learning, but there are continuous parameters usable for testing linear regression using an alternate hypothesis as well as constructing an ordinal scale using the given categorical label and fitting the linear model with this transformed label and evaluating its prediction.

In the initial sections, we get better acquainted with the data by understanding the sources and providing a detailed description for each of its attributes. This is followed by exploratory data analysis with the focus on the three categorical features (including the label) and some of the notable numerical features. This allows us to gauge the balance of observations across the features and determine if resampling is necessary.

Next is the preprocessing phase where we clean the data through null value treatment and check for multicollinearity via correlation matrix analysis. We also calculate similarity measures at this juncture to discern the identical features to eliminate them in the next section. Also, we apply scaling and encoding to the numerical and categorical features respectively after necessary evaluation of the data.

Since the number of variables initially are quite high (84), they are reduced through feature selection and dimensionality reduction techniques while eliminating the multicollinearity detected previously. This is done parallelly by removing the previously detected identical features contributing to the multicollinearity of the data. This should bring the data closer to its latent space, ready to be fitted to machine learning algorithms.

Starting off with classification methods, the data is fitted with Logistic Regression, Decision Tree, Support Vector Machine and Naïve Bayes. Next up is clustering with K-means clustering and finally ending with the Linear Regression. Each model is evaluated individually as well as together within its group for comparative analysis at the end of each section.

2. Data Description

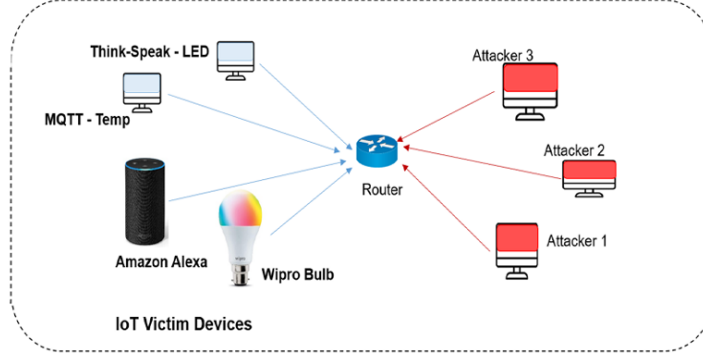


Figure 1: Experimental setup for generating RT-IoT '22 dataset

A little about the experiment that led to the generation of the dataset that is depicted in the above figure: the infrastructure consists of two parts, namely IoT victim devices and IoT attacker devices, both connected through a router. The network traffic through a router using Wireshark, which is an open-source monitoring tool for network traffic.

Dataset	PCAP	CICFlowmeter	Protocol	Service	SubCategory	Category
ThingSpeak-LED	10,526	8108	TCP, UDP	DNS, HTTP	ThingSpeak	0
MQTT-Temp	8162	4146	TCP	MQTT	MQTT	0
Amazon-Alexa	6056	5023	TCP, UDP, ICMP	DNS, HTTP	Alexa	0
Wipro-Bulb	1265	253	TCP, UDP	SSL, DNS, IRC	Wipro_Bulb	0
SSH brute-force	1564	526	TCP, UDP	DNS, SSH	SSH_Brute_Force	1
DDoS	1786	534	TCP	HTTP	DDoS	1

Figure 2: Network traffic data retrieved from IoT devices

PCAP files were collected from each device for both normal and attack profiles. In the figure above, the source datasets and their characteristics such as ‘protocol’ and ‘service’ are stated. These are also the only two categorical features aside from the target variable present in the dataset.

Dataset	Features
ThingSpeak-LED	bwd_pkts_payload.avg, flow_pkts_payload.avg, bwd_iat.avg, flow_iat.max, bwd_init_window_size
MQTT-Temp	id.orig_p, idle.avg, idle.std, fwd_last_window_size
Amazon-Alexa	id.orig_p, id.resp_p, down_up_ratio, fwd_header_size_max, flow_ECE_flag_count, fwd_pkts_payload.std, bwd_pkts_payload.std
Wipro-Bulb	flow_RST_flag_count, fwd_pkts_payload.avg, bwd_pkts_payload.max, bwd_pkts_payload.avg, bwd_pkts_payload.std, flow_pkts_payload.max, flow_pkts_payload.avg, flow_pkts_payload.std, flow_iat.std, payload_bytes_per_second, fwd_subflow_bytes, bwd_subflow_bytes
Final RT-IOT2022	All above features

Figure 3: High RE features extracted from the datasets of IoT devices

Through feature engineering, the above data was converted from PCAP to CSV using CICFlowmeter tool and then collated to generate the RT-IoT '22 dataset. The preprocessed data was also reconstructed using an autoencoder. Using the reconstruction error from the autoencoder, the high RE features depicted in the above figure were selected as they were integral to differentiating anomalous traffic from normal traffic.

Category	Subcategory	Tools	Service	Wireshark instances	CIC Flowmeter instances	Class
Normal	ThinkSpeak-LED	—	DNS, HTTP	10526	8108	0
	MQTT-Temp	—	MQTT	8162	4146	0
	Amazon-Alexa	—	DNS, HTTP	6056	5023	0
	Wipro-Bulb	—	SSL, DNS, IRC	1265	253	0
Brute force	SSH Brute Force	Metasploit	SSH, DNS, HTTP	857	37	1
DDoS	SlowlorisDDoS	Slowloris	HTTP, DNS, DHCP	5920	534	2
	SYN Flood DDoS	Hping3	—	712850	94659	3
Nmap	FIN SCAN	Network	DNS, HTTP	69	28	4
	OS Fingerprinting	Mapper	—	8008	2000	5
	UDP scan	—	HTTP, DNS, NTP, DHCP, RADIUS	5606	2590	6
ARP poisoning	XMAS Tree scan	—	DNS, HTTP	8045	2010	7
	MiTM	Ettercap	DNS, SSL, HTTP, DHCP, NTP	306	7750	8

Figure 4: Description of normal/anomalous traffic across the IoT devices

The above figure from a follow-up paper on the same dataset gives a detailed representation of the target variable 'Attack_type' mentioned as the 'Subcategory' with 12 classes.

Basically, this dataset has features that are already extracted based on their effectiveness for detecting anomalous traffic. In this project, basic ML algorithms will be fitted with the features and tested.

3. Phase I: EDA and Preprocessing

The dataset contains 84 features: 81 numerical features, 3 categorical features (including the target variable). The data has 123,117 observations and it is prudent to examine the distributions in the variables of interest before proceeding towards preprocessing and model building. Exploratory data analysis is conducted on the categorical features before diving into data preprocessing stage.

3.1 Exploratory Data Analysis:

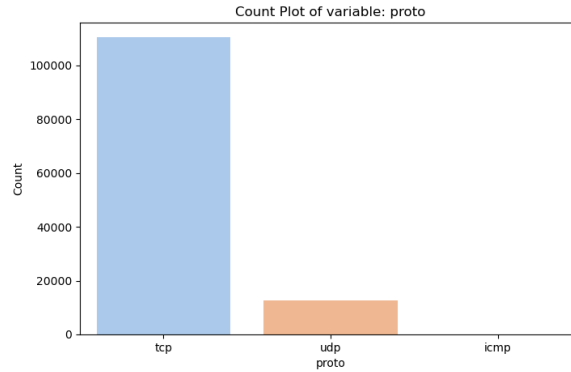


Figure 5: Count plot of the categorical variable 'proto'

The 'proto' (protocol) is highly imbalanced with very few observations for the category 'icmp'.

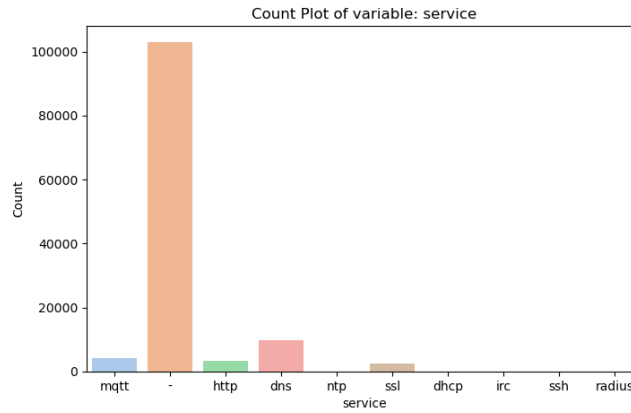


Figure 6: Count plot of the categorical variable 'service'

Same goes for several classes within the 'service' feature as well.

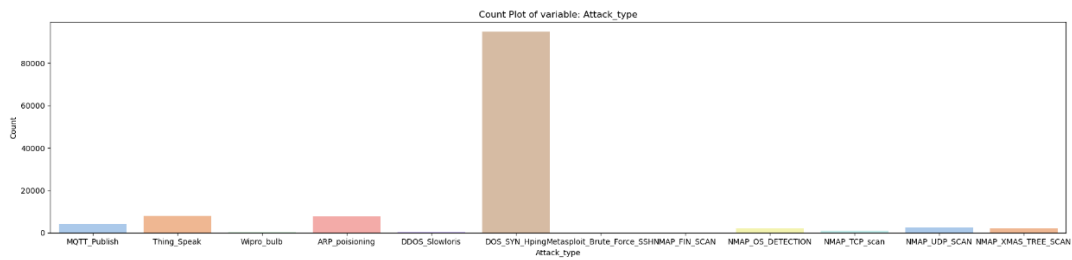


Figure 7: Count plot of target variable 'Attack_type'

There is a massive imbalance in the target variable 'Attack_type' with one class 'DOS_SYN_Hping' accounting for more than 75% of the observation of the dataset. On the opposite spectrum, other classes like 'NMAP_FIN_SCAN' and 'Metasploit_Brute_Force_SSH' only possess 28 and 37 observations respectively.

3.2 Data Preprocessing:

- 1) **Test Train Split Stratify:** The main data is divided into test and train sets of 20:80 proportions with stratified split across the target variable 'Attack_type'. This is done so at the beginning to isolate the test data from the resampling methods applied on the train.
- 2) **Data Cleaning:** The data is tested for identical features and constant features. One feature 'bwd_URG_flag_count' was found to only contain 0s. Hence, it was dropped from both test and train which now contain 83 features.
- 3) **Synthetic Minority Oversampling Technique (SMOTE):** This method oversamples all the observation based on the selected variable's classes to match the majority class. In our case, the variable is 'Attack_type' and all other classes are oversampled to be on par with majority class 'DOS_Syn_HPing'. It is applied on only train data with the target variable 'Attack_type' resulting in all labels being oversampled to the count of 94,659 each resulting in a dataset of 1,135,908 observations.

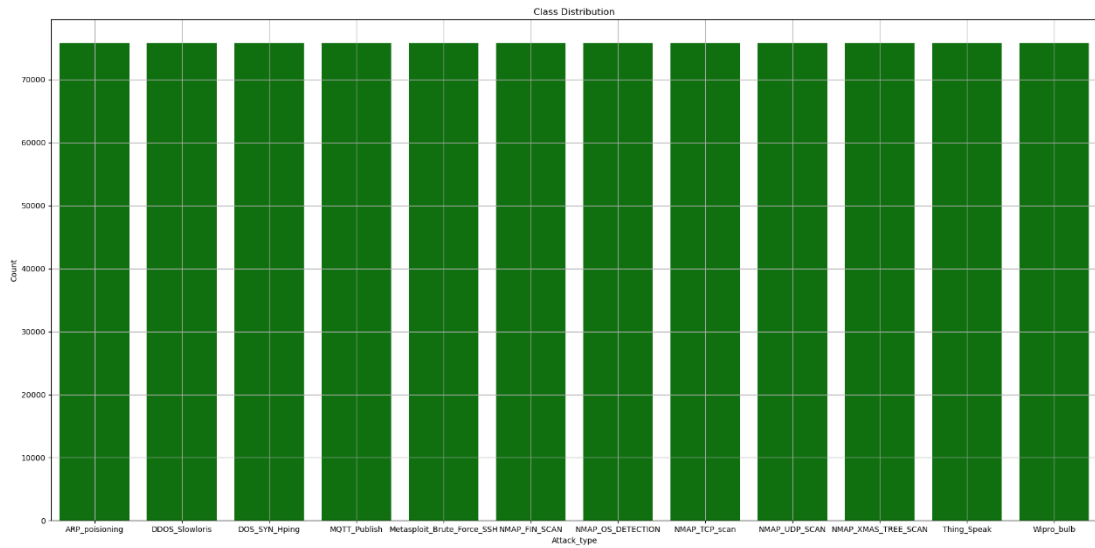


Figure 8: Frequency distribution of target after oversampling with SMOTE

- 4) **Random Undersampling:** This method is used to reduce the sample size by randomly selecting observations. In our case, it is applied in a stratified manner across all the equal frequency classes of target variable to reduce its count down to 5%. Each of the 12 classes is reduced to 4,732 observations and the total number goes down to 56,784 for our train dataset.

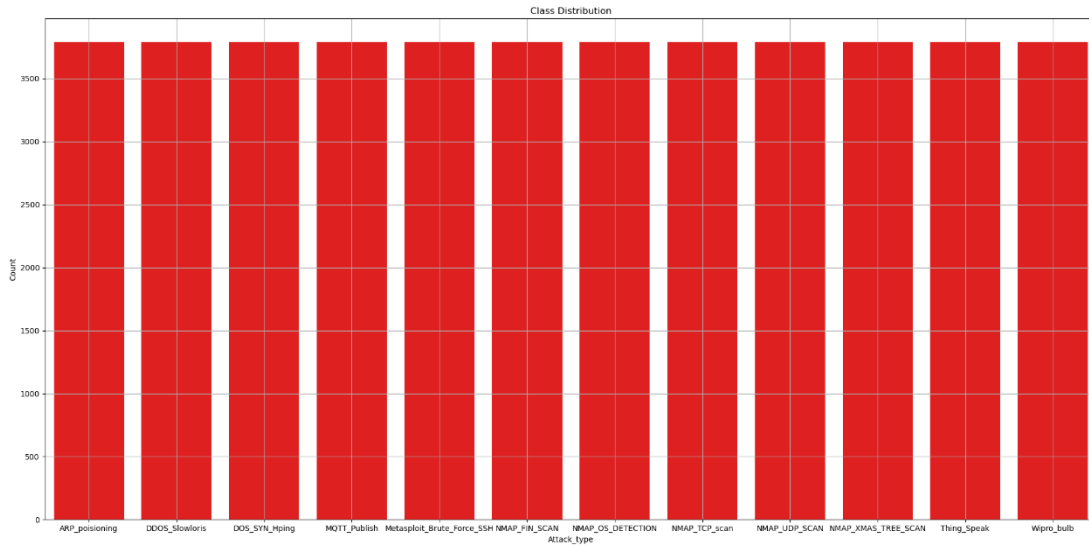


Figure 9: Frequency distribution of target after undersampling randomly to 5%

- 5) **Scaling:** Both test and train are scaled using standardization for all numerical features
- 6) **One-hot encoding:** Both test and train are one-hot encoded for the 2 categorical variables 'proto' and 'service' increasing the features to 91.
- 7) **Outlier Treatment:** The extreme 1 percentile of interquartile range is adjusted for all the numerical features and then another round of data cleaning conducted. The following 5 columns were dropped due to becoming constant columns from the above treatment: 'fwd_bulk_packets', 'flow_CWR_flag_count', 'flow_ECE_flag_count', 'fwd_bulk_rate' and 'fwd_bulk_bytes' reducing the feature count to 86. Shown below are the box plots for the same depicting the outliers.

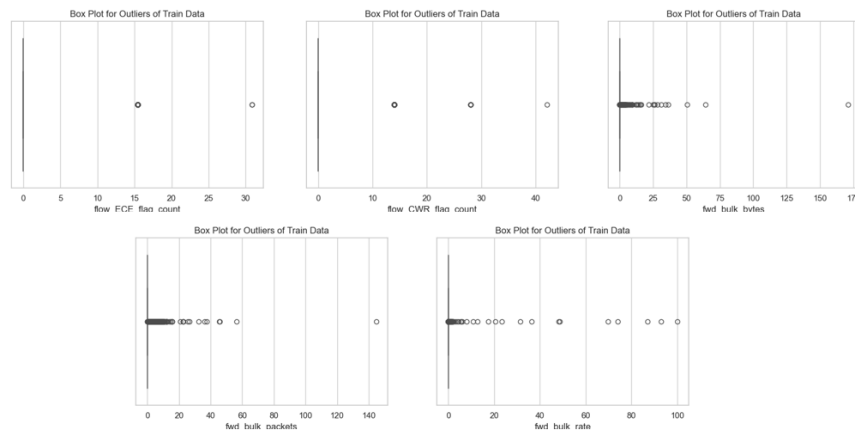


Figure 10: Boxplots of 5 features that were redundant after outlier treatment

8) Covariance Matrix Heatmap: As seen below, there are very few features responsible for contributing to the variance in the data as seen by the minimal red markers on the matrix. This indicates that a lot of features can be reduced on this basis through feature selection techniques.

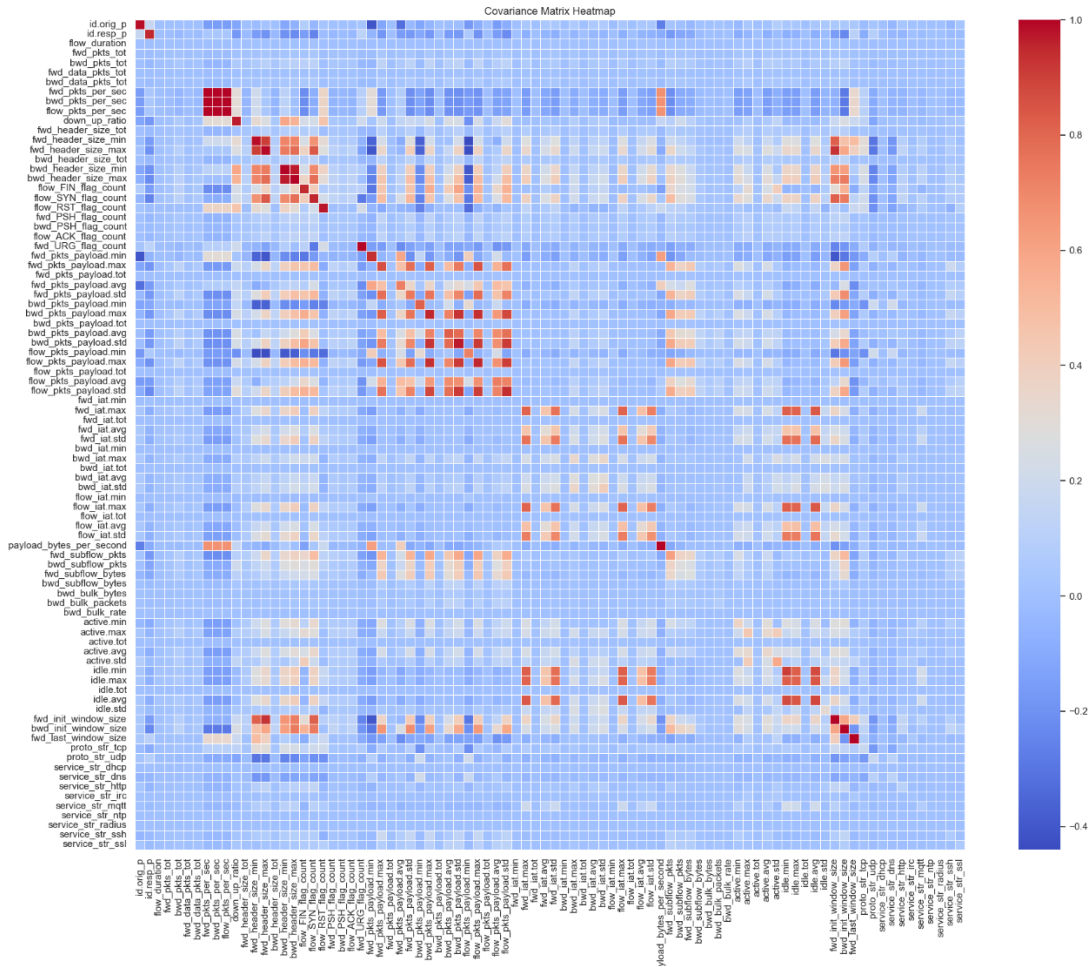


Figure 11: Covariance Matrix Heatmap of the feature space

- 9) **Pearson Correlation Coefficient Matrix Heatmap:** As seen below, very high correlation exists between the features as seen by the high red markers on the matrix. A fair number of these features are correlated to one another and need to be removed before fitting to models.

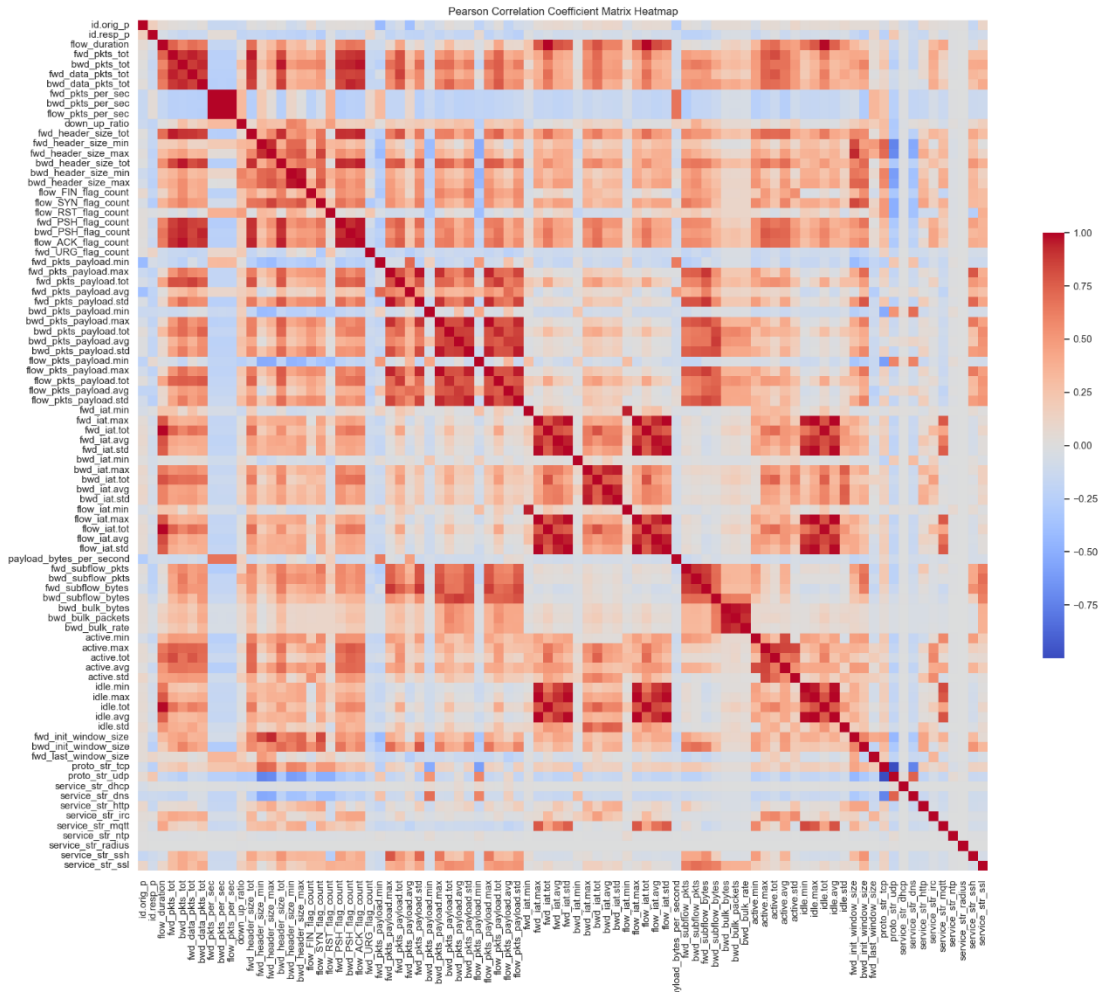


Figure 12: Pearson Correlation Coefficient Matrix Heatmap for the feature space

10) Principal Component Analysis: Though not applied for the purpose of feature extraction, the PCA tells us the proportion of features contributing to the total variance in the data. About 20 percent of the features, i.e., close to ~15 contribute almost ~90 percent of the explained variance. This means a lot of the features can be eliminated before proceeding with model building.

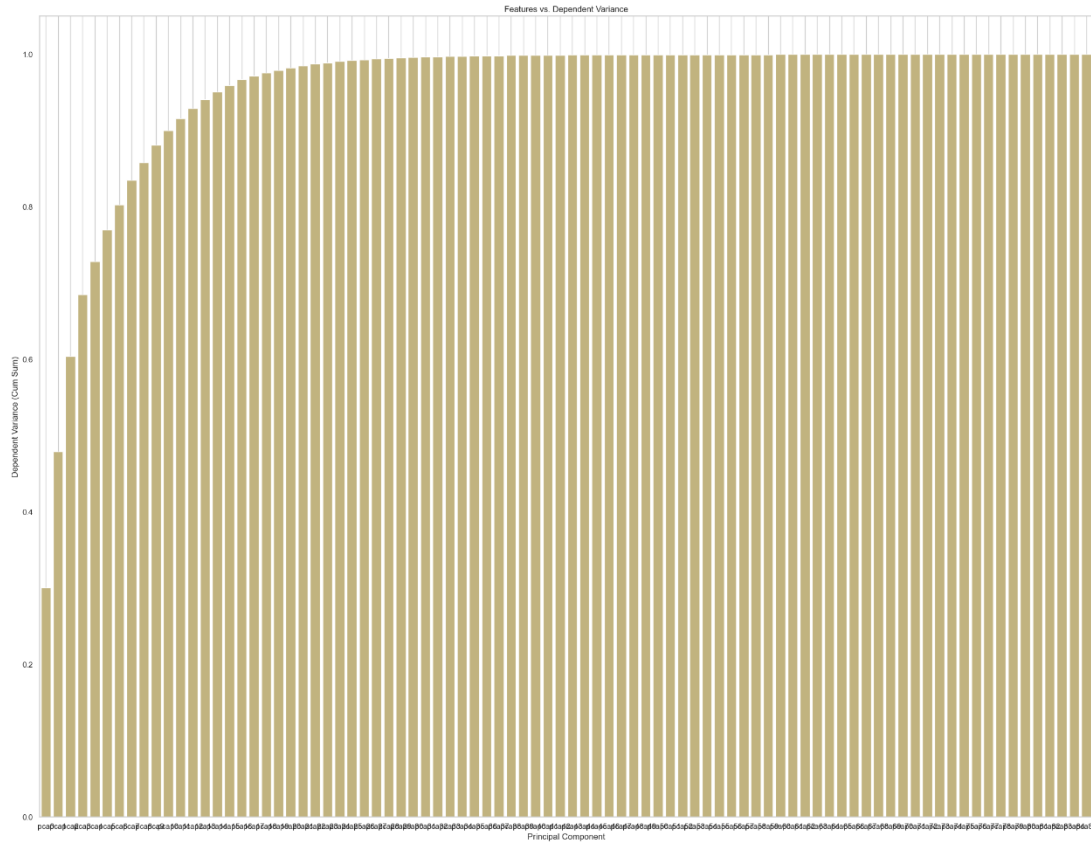


Figure 13: Histogram of Cumulative Explained Variance of features after PCA

11) Linear Discriminant Analysis: With the target variable being categorical, LDA is appropriate for maximizing the distance between the classes through Fischer Criterion. Also, the transformed feature space gives an idea of the possible final features we could have after feature selection.

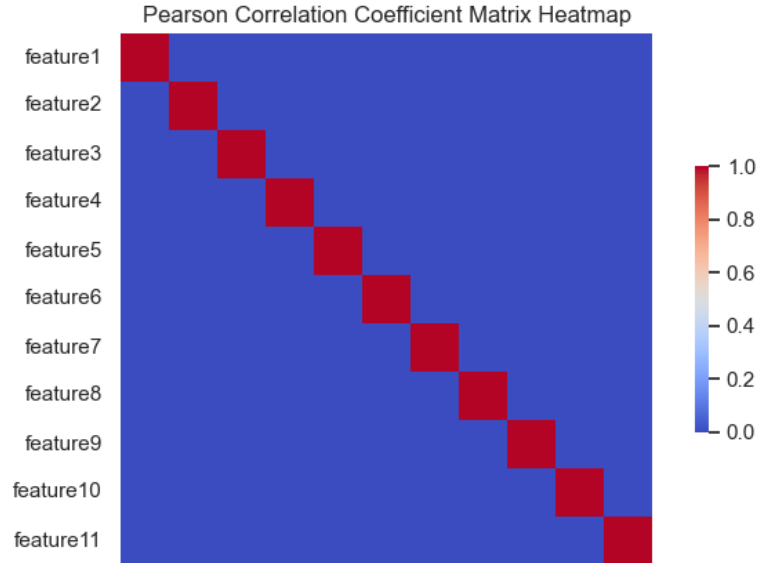


Figure 14: Correlation matrix of transformed features after LDA

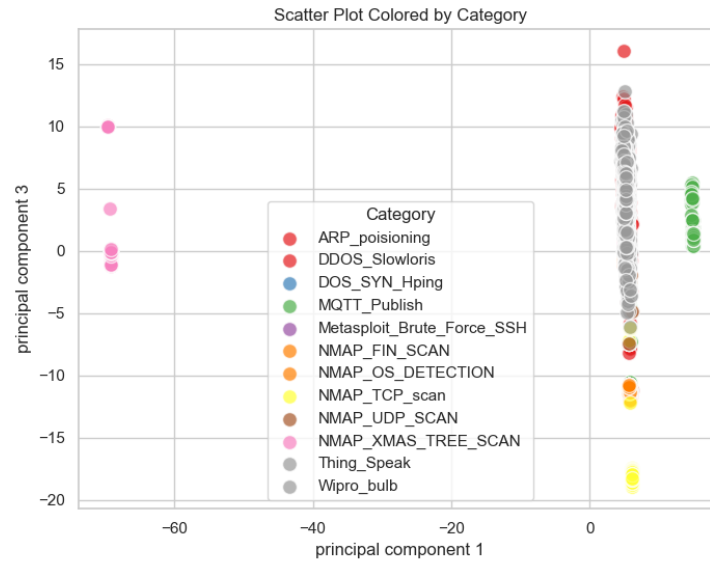


Figure 15: Separation of classes in LDA across 2 principal components

3.3 Feature Selection:

We start off with 86 features not including the target variable and apply Random Forest Analysis to gauge the number of important features in the feature space. Note: Since Random Forest requires that numerical variable be provided as a target variable, we binarize the 'Attack_type' from 12 classes to 2: 0 for Normal and 1 for Attack.

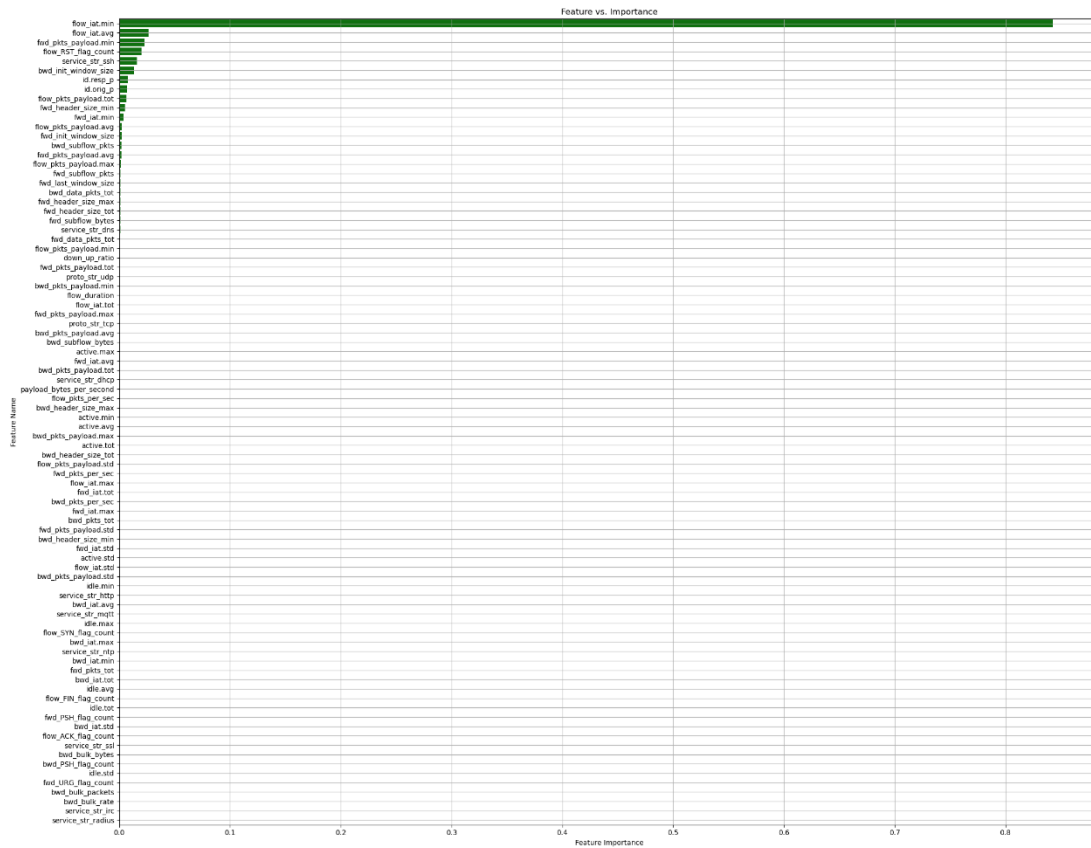


Figure 16: Random Forest Analysis of the feature space before reduction

This is a test case where we assess the most important features before feature selection despite collinearity in the data. As we see, majority of the features are not having any variable importance (entropy).

Singular Value Decomposition (SVD): Applied at the beginning and the end along to check for collinearity in the feature space. About 9 out of 86 features have singular values less than 1.

Condition Number (K): The condition number can be referenced as a degree of collinearity for the entire feature space. For the initial set of 86 features, the condition

number (K) is 10008412894282.53. This is indicative of features contributing to high degree of collinearity.

Variance Inflation Factor and Recursive Feature Elimination: Calculating the VIF recursively and removing the feature having the highest VIF is the simplest way to remove collinearity in the feature space. This was conducted on the set of 86 features until the highest VIF was under 5 and reduced the feature space to only 30 features. These features are again tested for collinearity using the SVD and K: Only 1 out of 30 features has singular value less than 1 and the condition number has reduced to 341.68 which is indicative of moderate degree of collinearity.

Random Forest Analysis: The feature space of 30 is now tested for variable importance using the binary 'Attack_type' as target variable. We see that only 10 out of 30 variables have non-zero feature importance. These were selected as the final features to be used for classification analysis.

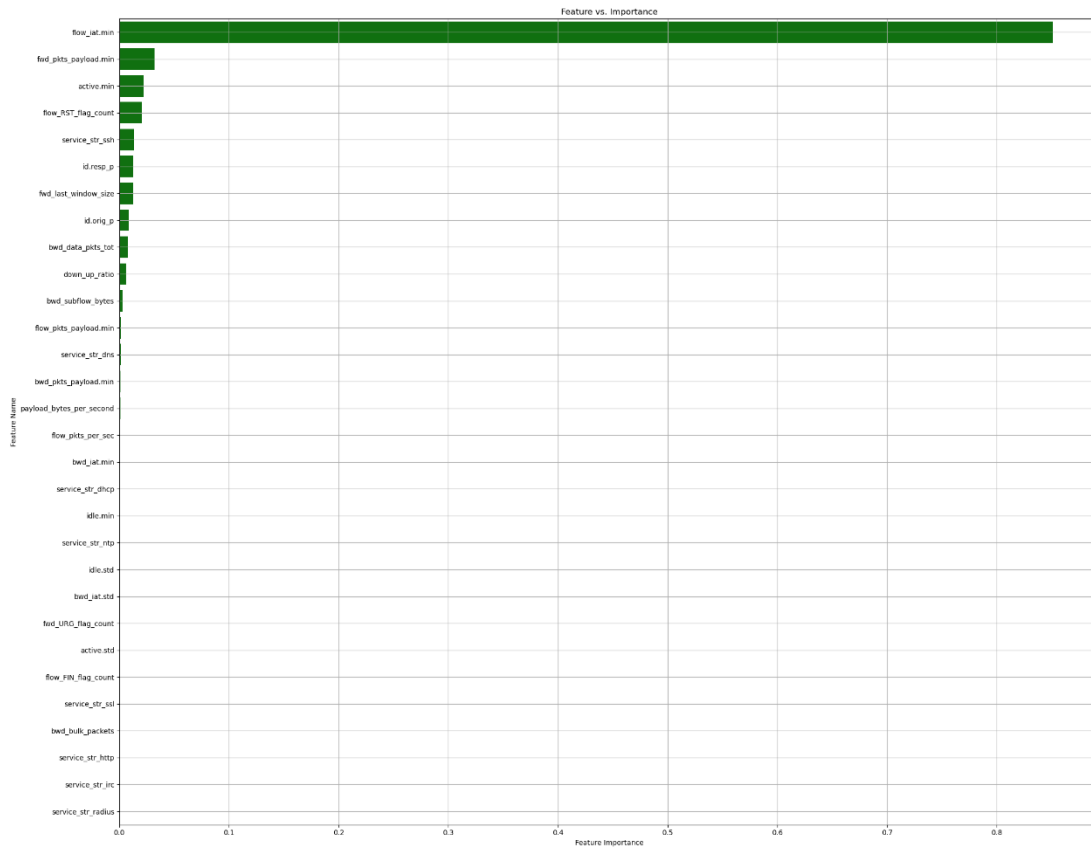


Figure 17: Random Forest Analysis of feature space after recursive feature elimination through VIF

Another round of checks for collinearity are conducted on these final 10 features for classification. SVD indicated 0 out of 10 features having singular values less than 1. The condition number K is now only 12.46 indicative of low degree of collinearity.

Hence, these 10 features have no collinearity between themselves and should be ready for model fitting.

For regression analysis, we do not use the 'Attack_type' as target and instead use the continuous variable 'down_up_ratio'. For this reason, we do not consider the results of Random Forest for linear regression and directly use the 30 features after Recursive Feature Elimination through VIF. We still perform stepwise regression with t-test and R-squared analysis to remove least fitted features.

4. Phase II: Regression Analysis

The target variable for regression is 'down_up_ratio' which is defined as the ratio of download to upload packet rates for a device. This is originally a non-zero ratio type and a flow-based network parameter. For linear regression, we work with the hypothesis that the features including the 'Attack_type' can be fitted to a linear model to predict the 'down_up_ratio' of a networking instance.

4.1 Setup

As explained in the previous section, the input data for regression is the features that are treated for collinearity using VIF without undergoing feature selection by Random Forest Analysis. This means we have 30 features now including the 'Attack_type' which is categorical.

One-hot encoding: The 'Attack_type' is one-hot encoded to 12 new numerical features increasing the feature space to 41.

SVD and Condition Number: Reconducting the test gives near identical values from the previous section.

4.2 Backward Stepwise Linear Regression

The linear model is first fitted with all the 41 features and then the model parameters AIC, BIC, R-squared, Adjusted R-squared alongside p-values for the F-test and highest p-value of the t-test for a feature is displayed in a table like the one below. The feature with the highest p-value of the t-test is then dropped and then the model refitted with remaining features and the parameters recalculated, as so on. This is recursively carried out until every feature is eventually dropped off.

T-test Analysis: We find that the initially dropped off features have considerably high p-values from t-tests of that iteration. This is seen for the first 5 iterations until the p-value drops below 0.05 (95% confidence range). These 5 features were not contributing to the linear model and have been dropped with justification of t-test.

F-test Analysis: The F-test calculates the statistic indicating the significance of independent variables in a linear model. We have p-value of almost zero from the very start, indicating that the feature space is indeed significant. Also the F-statistic is higher than 5 which means low p-value as seen (means null hypothesis of F-test is rejected) as also seen from the model summary figure after the table.

Table 1: Iterative t-test, F-test and R-squared analysis in Backward Stepwise Linear Regression

Iter	AIC	BIC	R ²	R ² -Adj	Dropped feature	p-value from t-test	p-value from F-test
1	41431	41788.7	0.85	0.85	service_str_radius	0.819	0
2	41429	41778	0.85	0.85	Attack_type_str_NMAP_XMAS_TREE_SCAN	0.427	0
3	41427.7	41767.9	0.85	0.85	flow_pkts_per_sec	0.408	0
4	41426.3	41757.9	0.85	0.85	payload_bytes_per_second	0.188	0
5	41426.1	41748.9	0.85	0.85	Attack_type_str_Wipro_bulb	0.092	0
6	41426.9	41741	0.85	0.85	idle.min	0.01	0
7	41431.6	41736.9	0.85	0.85	service_str_irc	0.003	0
8	41438.4	41735	0.85	0.85	service_str_dns	0.002	0
9	41445.9	41733.8	0.85	0.85	service_str_dhcp	0	0
10	41461.2	41740.4	0.85	0.85	idle.std	0	0
11	41499	41769.4	0.85	0.85	service_str_ssh	0	0
12	41558.3	41820	0.85	0.85	const	0	0
13	41621	41873.9	0.85	0.849	service_str_ntp	0	0
14	41728	41972.3	0.849	0.849	id.resp_p	0	0

15	41847.2	42082.7	0.849	0.849	bwd_subflow_bytes	0	0
16	42028.5	42255.4	0.848	0.848	active.std	0	0
17	42189.5	42407.6	0.848	0.848	flow_RST_flag_count	0	0
18	42416.5	42625.9	0.847	0.847	service_str_http	0	0
19	42684.5	42885.1	0.846	0.846	fwd_last_window_size	0	0
20	43047	43238.9	0.845	0.845	bwd_bulk_packets	0	0
21	43535.8	43719	0.843	0.843	Attack_type_str_DOS_SY N_Hping	0	0
22	44069.6	44244.1	0.841	0.841	active.min	0	0
23	44562.5	44728.2	0.839	0.839	id.orig_p	0	0
24	45312.1	45469.1	0.837	0.837	flow_iat.min	0	0
25	46399	46547.3	0.833	0.833	bwd_iat.std	0	0
26	47431.4	47571	0.829	0.829	service_str_ssl	0	0
27	52135.7	52266.6	0.81	0.81	fwd_pkts_payload.min	0	0
28	69813.6	69935.7	0.72	0.72	Attack_type_str_DDOS_Sl owloris	0.674	0
29	69811.8	69925.2	0.72	0.72	Attack_type_str_Metasploit _Brute_Force SSH	0.353	0
30	69810.6	69915.3	0.72	0.72	bwd_data_pkts_tot	0	0
31	70042.1	70138.1	0.719	0.719	Attack_type_str_Thing_Sp eak	0	0
32	70477	70564.2	0.716	0.716	Attack_type_str_MQTT_P ublish	0	0
33	71217	71295.5	0.711	0.711	bwd_iat.min	0	0

34	72292	72361.8	0.704	0.704	fwd_URG_flag_count	0	0
35	75213	75274.1	0.685	0.685	flow_pkts_payload.min	0	0
36	81803.2	81855.6	0.635	0.635	flow_FIN_flag_count	0	0
37	93294.5	93338.1	0.53	0.53	bwd_pkts_payload.min	0	0
38	95157.3	95192.2	0.511	0.511	Attack_type_str_NMAP_UDP_SCAN	0	0
39	113167.8	113193.9	0.273	0.273	Attack_type_str_NMAP_TCP_scan	0	0
40	115455.5	115472.9	0.235	0.235	Attack_type_str_NMAP_OSS_DETECTION	0	0
41	117685.2	117693.9	0.197	0.197	Attack_type_str_NMAP_FIN_SCAN	0	0

Adjusted R-squared Analysis: We choose the best set of features based on the Adjusted R-squared for the iterations where the metric maintains a high enough value before plummeting because of a significant feature being dropped off. That point occurs at iteration 27 where 'fwd_pkts_payload.min' is dropped off and Adjusted R-squared sharply falls from 81% to 72%.

```

Summary of LR model is
:
=====
OLS Regression Results
=====
Dep. Variable:      down_up_ratio      R-squared:      0.811
Model:              OLS                Adj. R-squared:  0.811
Method:             Least Squares       F-statistic:     1.302e+04
Date:               Sat, 10 May 2025     Prob (F-statistic): 0.00
Time:               23:14:19            Log-Likelihood:  -25933.
No. Observations:   45432              AIC:             5.190e+04
Df Residuals:       45416              BIC:             5.204e+04
Df Model:           15
Covariance Type:    nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	0.1436	0.009	15.492	0.000	0.125	0.162
Attack_type_str_NMAP_FIN_SCAN	-0.9978	0.010	-97.229	0.000	-1.018	-0.978
Attack_type_str_NMAP_OS_DETECTION	1.5278	0.011	140.576	0.000	1.507	1.549
Attack_type_str_NMAP_TCP_scan	1.5198	0.011	139.837	0.000	1.498	1.541
Attack_type_str_NMAP_UDP_SCAN	-0.7473	0.010	-73.265	0.000	-0.767	-0.727
bwd_pkts_payload.min	0.4331	0.003	156.684	0.000	0.428	0.439
flow_FIN_flag_count	0.3431	0.003	112.623	0.000	0.337	0.349
flow_pkts_payload.min	-0.4881	0.003	-156.061	0.000	-0.494	-0.482
fwd_URG_flag_count	0.3297	0.003	113.787	0.000	0.324	0.335
bwd_iat.min	6.7710	0.166	40.825	0.000	6.446	7.096
Attack_type_str_MQTT_Publish	0.3384	0.010	33.583	0.000	0.319	0.358
Attack_type_str_Thing_Speak	0.4592	0.009	52.266	0.000	0.442	0.476
bwd_data_pkts_tot	1.6064	0.025	65.039	0.000	1.558	1.655
Attack_type_str_Metasploit_Brute_Force_SSH	0.2831	0.009	31.728	0.000	0.266	0.301
Attack_type_str_DDOS_Slowloris	0.5593	0.010	55.468	0.000	0.540	0.579
fwd_pkts_payload.min	0.4852	0.003	139.974	0.000	0.478	0.492

```

=====
Omnibus:      6304.584    Durbin-Watson:      2.019
Prob(Omnibus): 0.000    Jarque-Bera (JB):    56756.338
Skew:         -0.377    Prob(JB):            0.00

```

Figure 18: Model summary of the best fitted linear regression using the 15 selected features

We selected the final 15 features after this iteration as the best fitting linear model. The F-statistic is also in high 10,000s indicating that all the features are significant. For the predictions on the test, we have the following metrics:

Table 2: Model statistics after fitting with the best features

MSE	Confidence Interval (Lower bound)	Confidence Interval (Upper bound)
0.04387	-0.41092	0.41019

We also plot the test values and predicted values below against the index to see how they align with one another. We see that the predicted values appearing in close vicinity of the test.

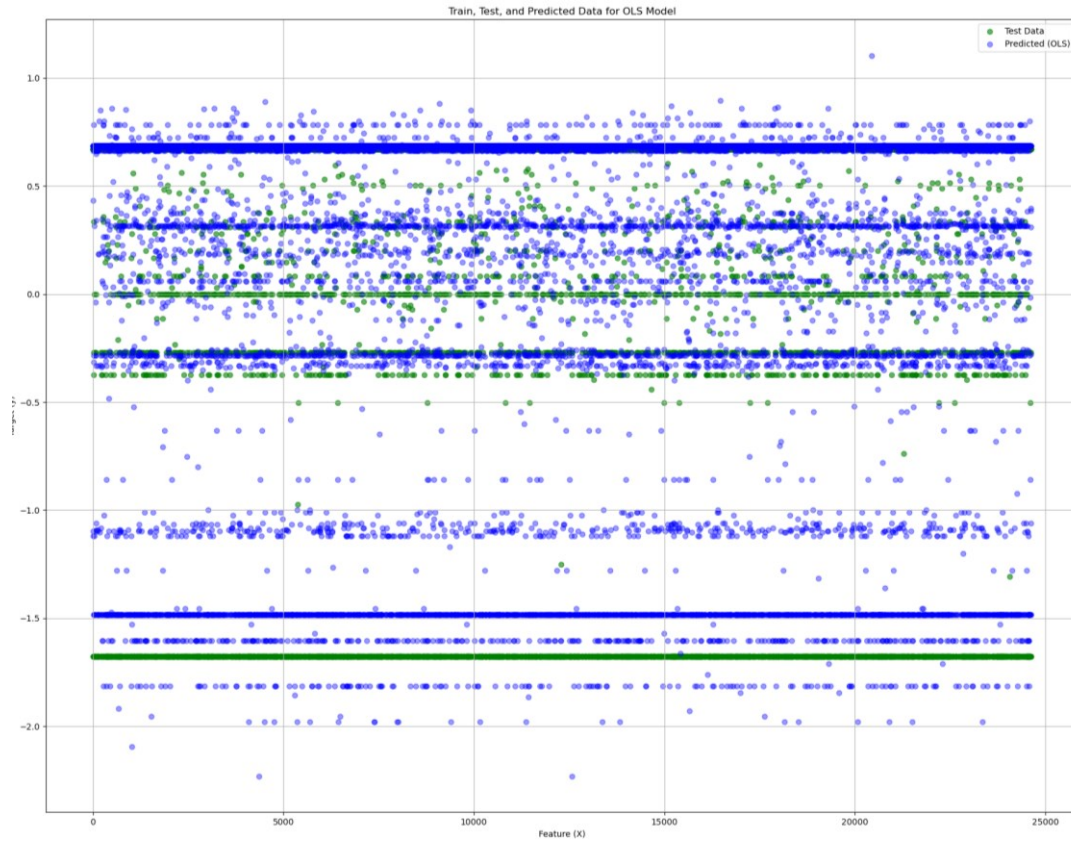


Figure 19: Scatter plot of the test and predicted values from Linear Regression model

5. Phase III: Classification Analysis

For classification analysis, we apply one vs one and one vs rest (one vs all) classifications for our multiclass target variable ‘Attack_type’ for models that are primarily meant for binary classifications.

5.1 Decision Tree

We have three variants of Decision Tree, the original (which is not given parameters for limiting its growth), the pre-pruned and the post-pruned.

5.1.1 Decision Tree (Original)

Table 3: Decision Tree (Original) accuracies for 5-fold stratified cross-validation

Fold #1 Accuracy	Fold #2 Accuracy	Fold #3 Accuracy	Fold #4 Accuracy	Fold #5 Accuracy	Average Accuracy
0.99296	0.99362	0.99417	0.99252	0.99483	0.99362

Table 4: Test metrics for the fitted Decision Tree (Original)

Micro-averaged One-vs-Rest ROC AUC score	Micro-averaged Specificity score	Micro-averaged Precision/Recall/F1 score
0.99579	0.99927	0.99200

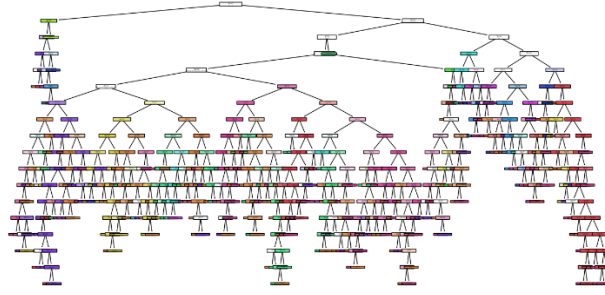


Figure 20: Original Decision Tree without pruning for only 10 features

This tree has the following parameters: {'ccp_alpha': 0.0, 'class_weight': None, 'criterion': 'gini', 'max_depth': None, 'max_features': None, 'max_leaf_nodes': None, 'min_impurity_decrease': 0.0, 'min_samples_leaf': 1, 'min_samples_split': 2, 'min_weight_fraction_leaf': 0.0, 'random_state': 5805, 'splitter': 'best'}

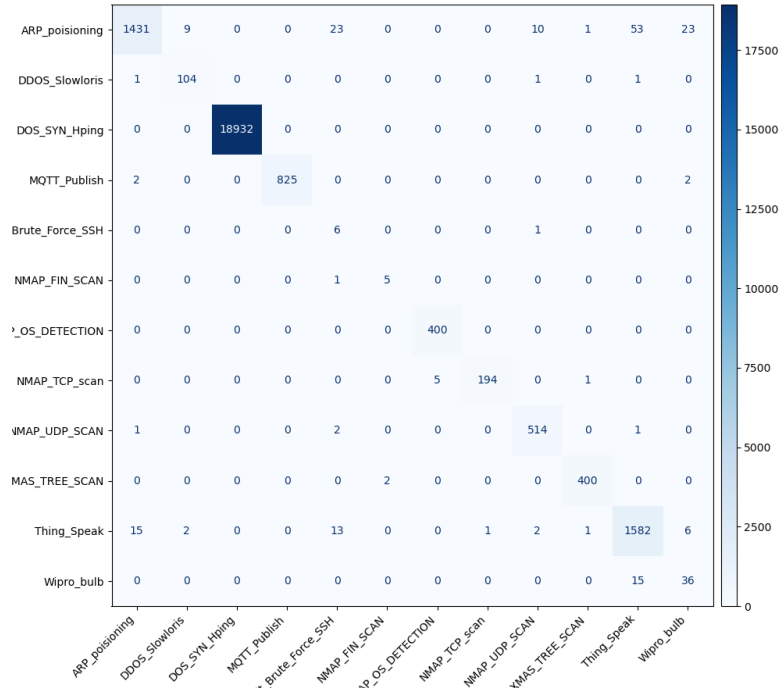


Figure 21: Confusion Matrix of the predictions for Decision Tree (Original)

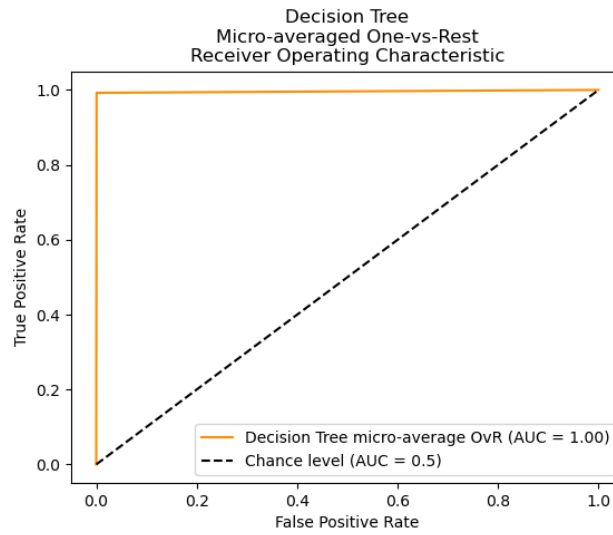


Figure 22: ROC curve for the Decision Tree (Original)

5.1.2 Decision Tree (Pre-pruned)

The selected parameters after pre-pruning were: {'criterion': 'entropy', 'max_depth': 5, 'max_features': 'sqrt', 'min_samples_leaf': 10, 'min_samples_split': 40, 'splitter': 'best'}

Table 5: Decision Tree (Pre-pruned) accuracies for 5-fold stratified cross-validation

Fold #1 Accuracy	Fold #2 Accuracy	Fold #3 Accuracy	Fold #4 Accuracy	Fold #5 Accuracy	Average Accuracy
0.79773	0.81061	0.86595	0.84482	0.80013	0.82385

Table 6: Test metrics for the fitted Decision Tree (Pre-pruned)

Micro-averaged One-vs-Rest ROC AUC score	Micro-averaged Specificity score	Micro-averaged Precision/Recall/F1 score
0.99365	0.99183	0.91013

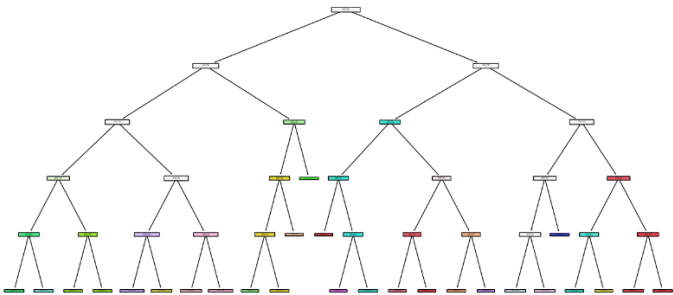


Figure 23: Pre-pruned Decision Tree with a simpler schematic

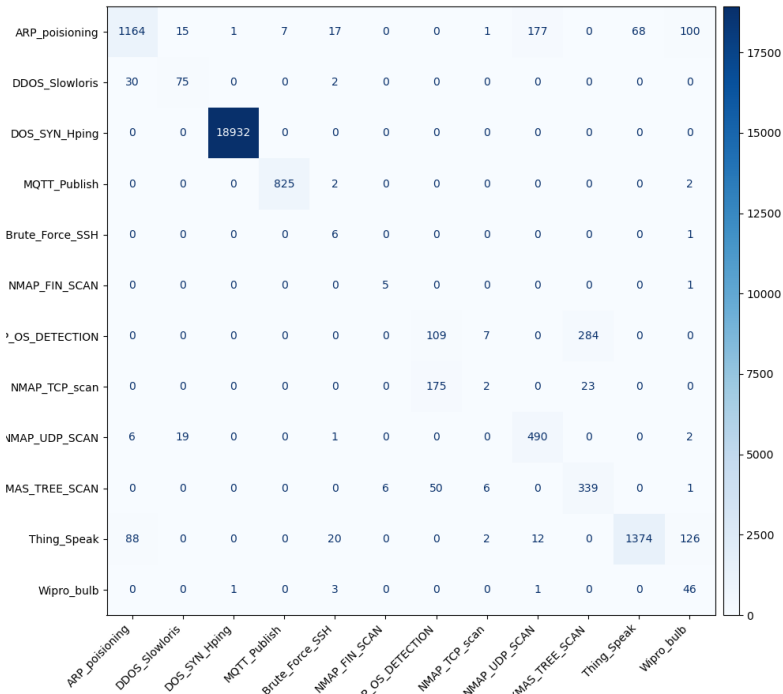


Figure 24: Confusion Matrix of the predictions for Decision Tree (Pre-pruned)

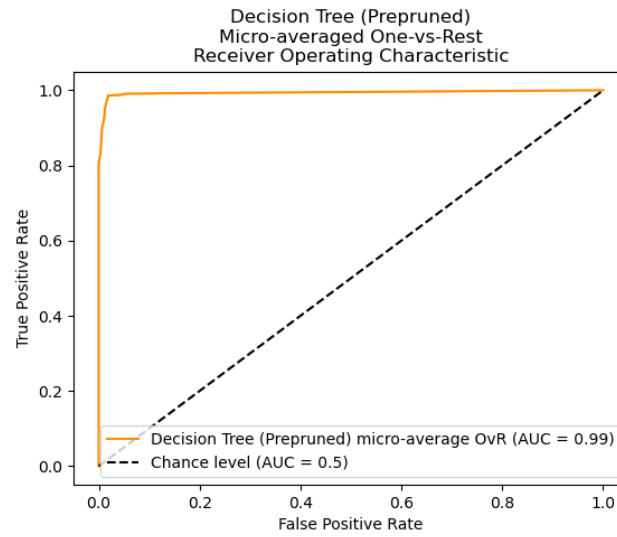


Figure 25: ROC curve for the Decision Tree (Pre-pruned)

5.1.3 Decision Tree (Post-pruned)

Optimum CCP Alpha was 0.000039.

Table 7: Decision Tree (Post-pruned) accuracies for 5-fold stratified cross-validation

Fold #1 Accuracy	Fold #2 Accuracy	Fold #3 Accuracy	Fold #4 Accuracy	Fold #5 Accuracy	Average Accuracy
0.99285	0.99362	0.99296	0.99263	0.99483	0.99337

Table 8: Test metrics for the fitted Decision Tree (Post-pruned)

Micro-averaged One-vs-Rest ROC AUC score	Micro-averaged Specificity score	Micro-averaged Precision/Recall/F1 score
0.99674	0.99927	0.99192

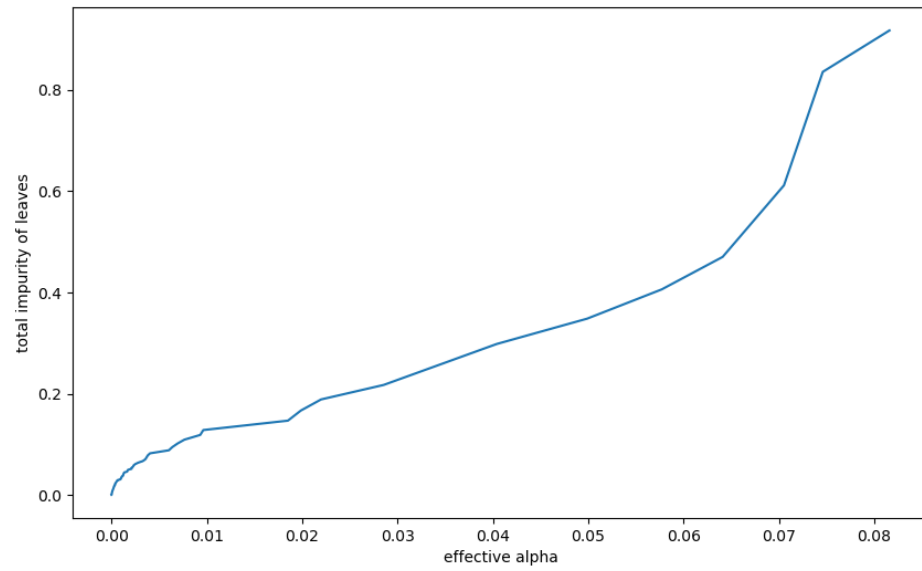


Figure 26: Line plot showing the relation between alpha and impurities of leaves

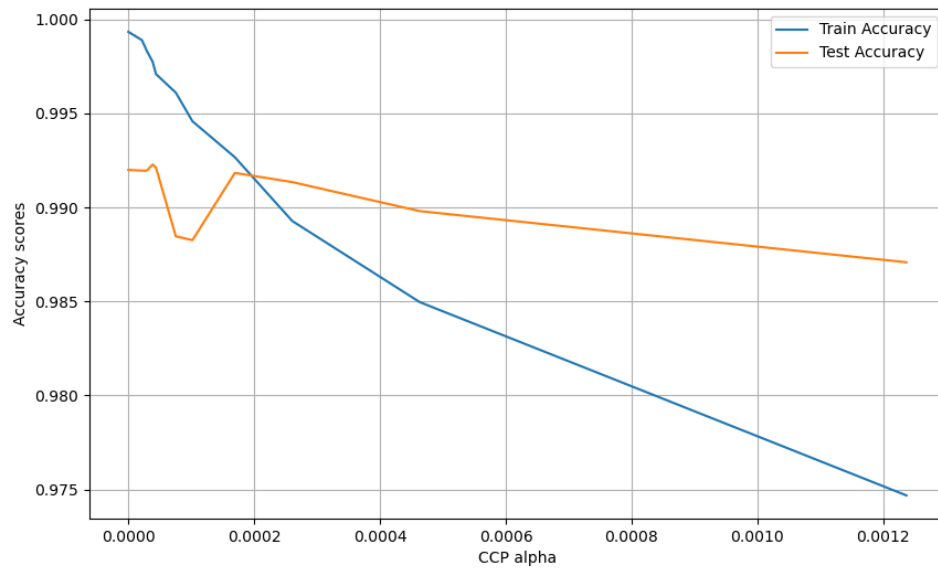


Figure 27: Line plot comparison of the test and train accuracies versus CCP alpha

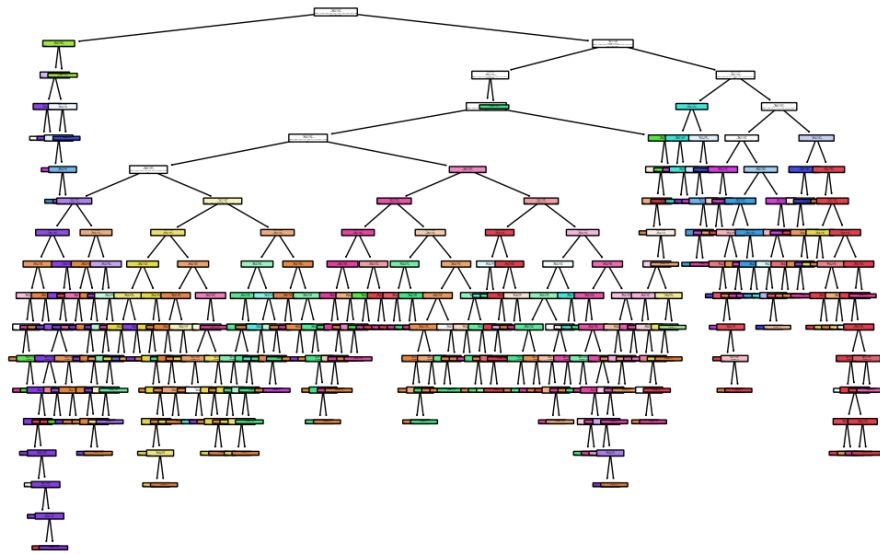


Figure 28: Post-pruned Decision Tree still having a complex schematic

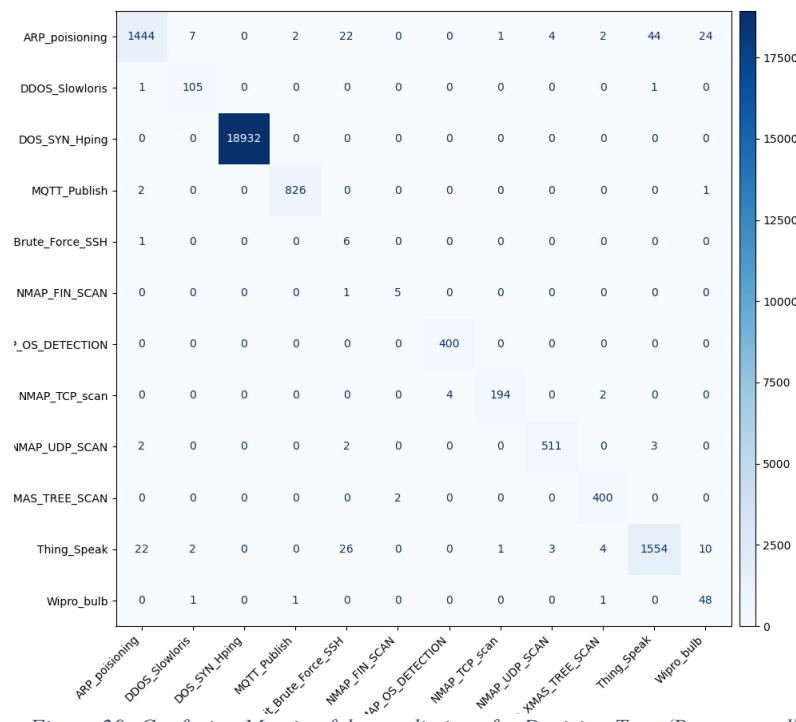


Figure 29: Confusion Matrix of the predictions for Decision Tree (Post-pruned)

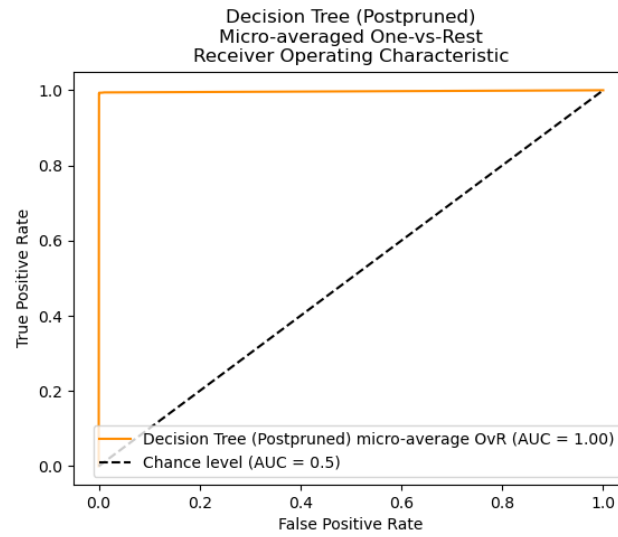


Figure 30: ROC curve for the Decision Tree (Post-pruned)

5.2 Logistic Regression

5.2.1 Logistic Regression (One vs. One)

Table 9: Logistic Regression (OvO) accuracies for 5-fold stratified cross-validation

Fold #1 Accuracy	Fold #2 Accuracy	Fold #3 Accuracy	Fold #4 Accuracy	Fold #5 Accuracy	Average Accuracy
0.83603	0.85991	0.85604	0.84030	0.86022	0.85050

Table 10: Test metrics for the fitted Logistic Regression (OvO)

Micro-averaged One-vs-Rest ROC AUC score	Micro-averaged Specificity score	Micro-averaged Precision/Recall/F1 score
0.99035	0.99558	0.95135

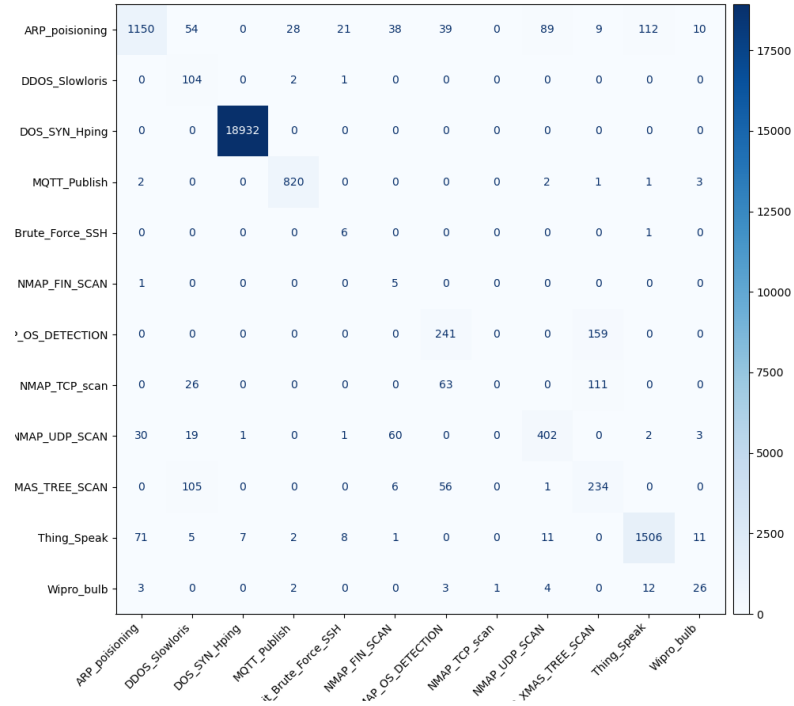


Figure 31: Confusion Matrix of the predictions for Logistic Regression (OvO)

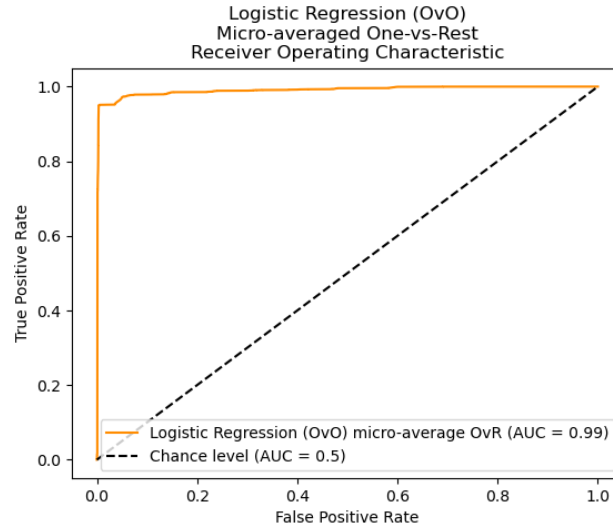


Figure 32: ROC curve for the Logistic Regression (OvR)

5.2.2 Logistic Regression (One vs. All)

Table 11: Logistic Regression (OvR) accuracies for 5-fold stratified cross-validation

Fold #1 Accuracy	Fold #2 Accuracy	Fold #3 Accuracy	Fold #4 Accuracy	Fold #5 Accuracy	Average Accuracy

0.75702	0.77495	0.76590	0.75930	0.77383	0.76620
---------	---------	---------	---------	---------	---------

Table 12: Test metrics for the fitted Logistic Regression (OvR)

Micro-averaged One-vs-Rest ROC AUC score	Micro-averaged Specificity score	Micro-averaged Precision/Recall/F1 score
0.99342	0.99234	0.91573

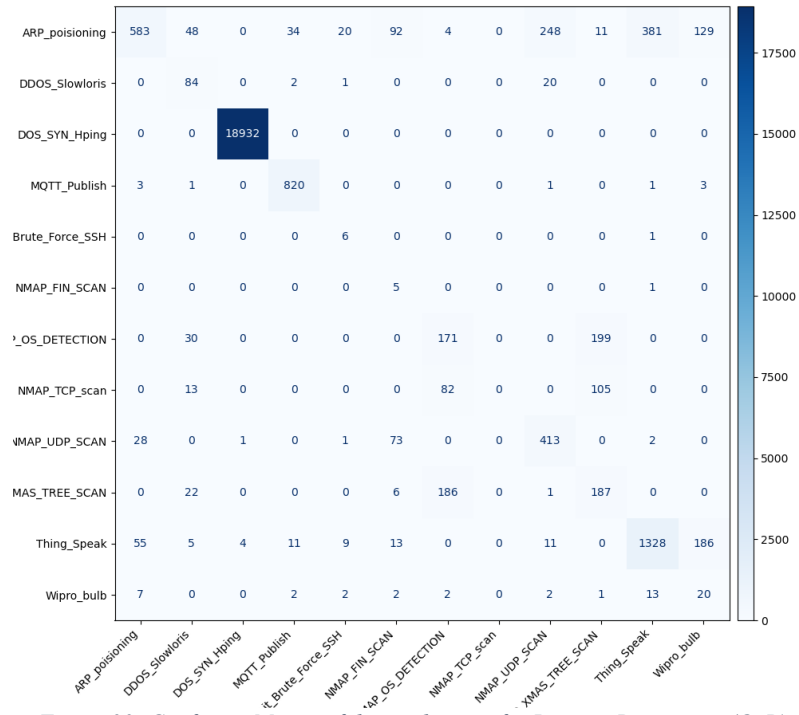


Figure 33: Confusion Matrix of the predictions for Logistic Regression (OvR)

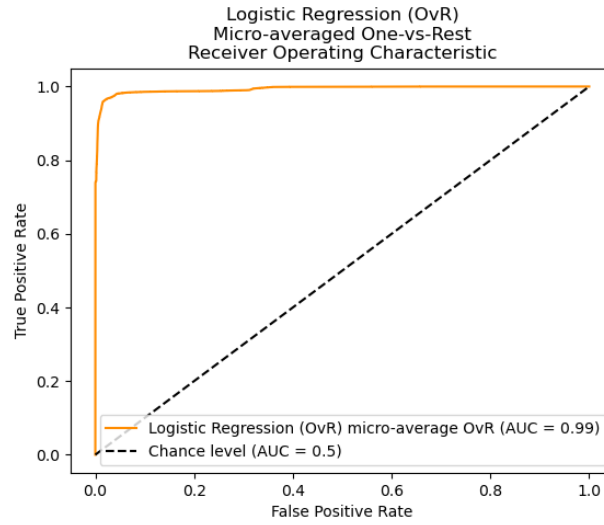


Figure 34: ROC curve for the Logistic Regression (OvR)

5.3 Support Vector Classifier

Both One vs One and One vs Rest (One vs All) methods were employed alongside testing across 3 different kernels: Linear, Radial Basis Function (RBF) and Polynomial.

5.3.1 SVC (One vs. One)

5.3.1.1 Linear Kernel (SVC OvO)

Table 13: Linear SVC (OvO) accuracies for 5-fold stratified cross-validation

Fold #1 Accuracy	Fold #2 Accuracy	Fold #3 Accuracy	Fold #4 Accuracy	Fold #5 Accuracy	Average Accuracy
0.88335	0.88489	0.88092	0.87838	0.88697	0.88290

Table 14: Test metrics for the fitted Linear SVC (OvO)

Micro-averaged One-vs-Rest ROC AUC score	Micro-averaged Specificity score	Micro-averaged Precision/Recall/F1 score
0.97707	0.99618	0.95797

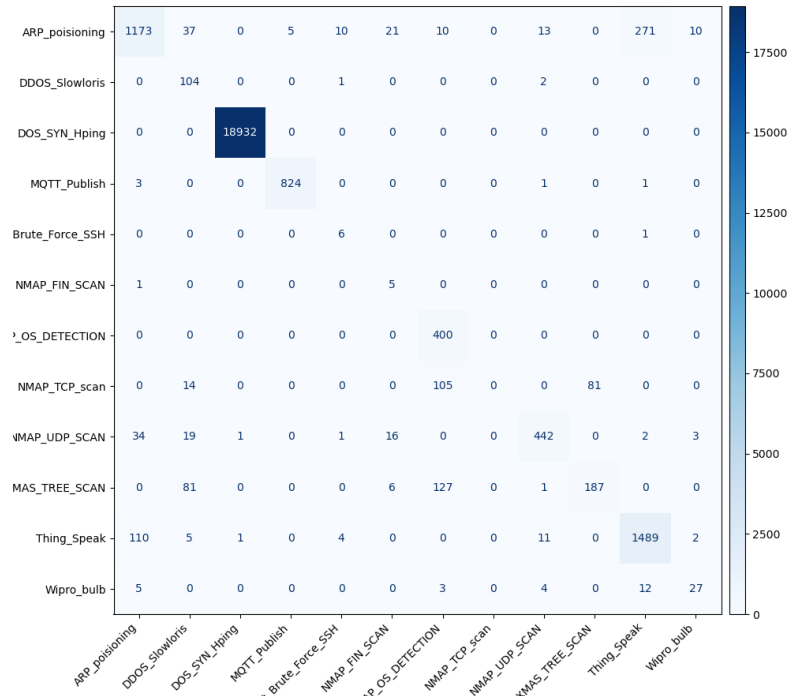


Figure 35: Confusion Matrix of the predictions for Linear Kernel SVC (OvO)

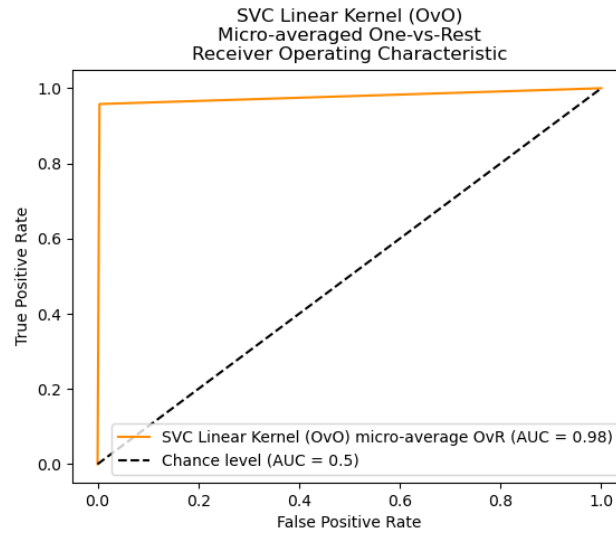


Figure 36: ROC curve for the Linear Kernel SVC (OvO)

5.3.1.2 Radial Basis Function Kernel (SVC OvO)

Table 15: RBF SVC (OvO) accuracies for 5-fold stratified cross-validation

Fold #1 Accuracy	Fold #2 Accuracy	Fold #3 Accuracy	Fold #4 Accuracy	Fold #5 Accuracy	Average Accuracy
0.94234	0.94696	0.94959	0.94695	0.95135	0.94744

Table 16: Test metrics for the fitted RBF SVC (OvO)

Micro-averaged One-vs-Rest ROC AUC score	Micro-averaged Specificity score	Micro-averaged Precision/Recall/F1 score
0.98602	0.99767	0.97437

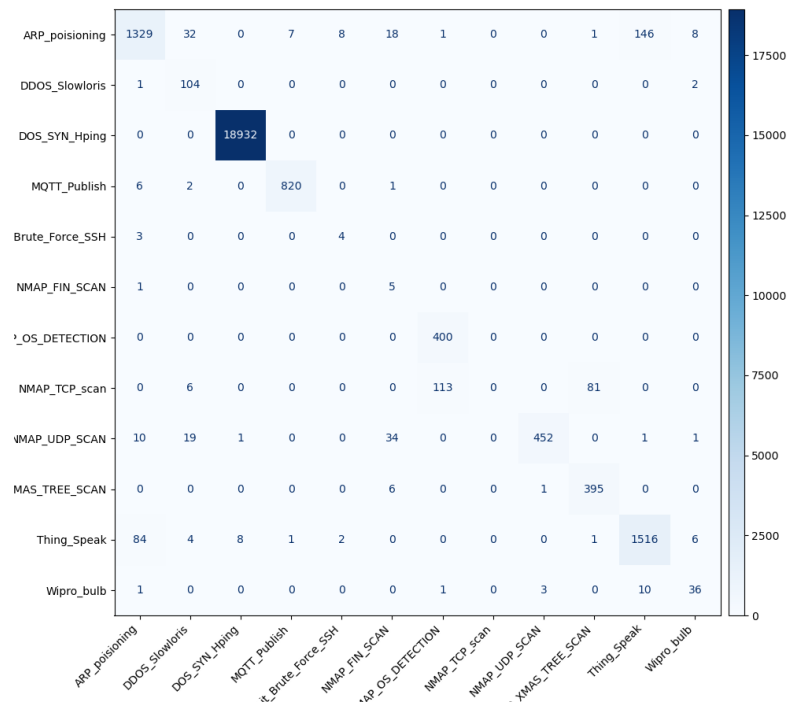


Figure 37: Confusion Matrix of the predictions for RBF Kernel SVC (OvO)

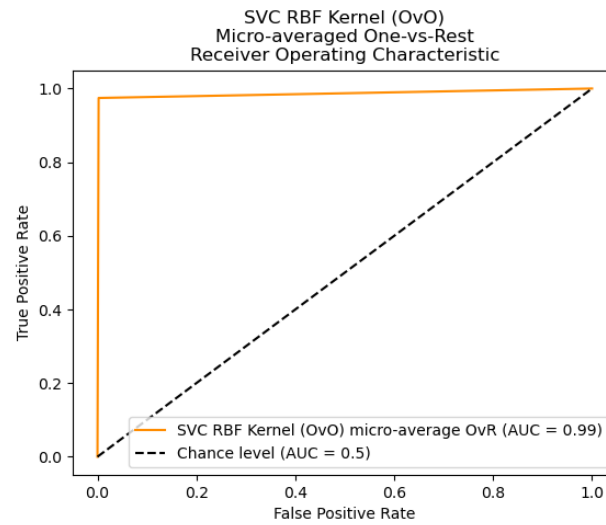


Figure 38: ROC curve for the RBF Kernel SVC (OvO)

5.3.1.3 Polynomial Kernel (SVC OvO)

Table 17: Polynomial SVC (OvO) accuracies for 5-fold stratified cross-validation

Fold #1 Accuracy	Fold #2 Accuracy	Fold #3 Accuracy	Fold #4 Accuracy	Fold #5 Accuracy	Average Accuracy
0.92374	0.93078	0.93033	0.92439	0.93308	0.92846

Table 18: Test metrics for the fitted Polynomial SVC (OvO)

Micro-averaged One-vs-Rest ROC AUC score	Micro-averaged Specificity score	Micro-averaged Precision/Recall/F1 score
0.98179	0.99697	0.96662

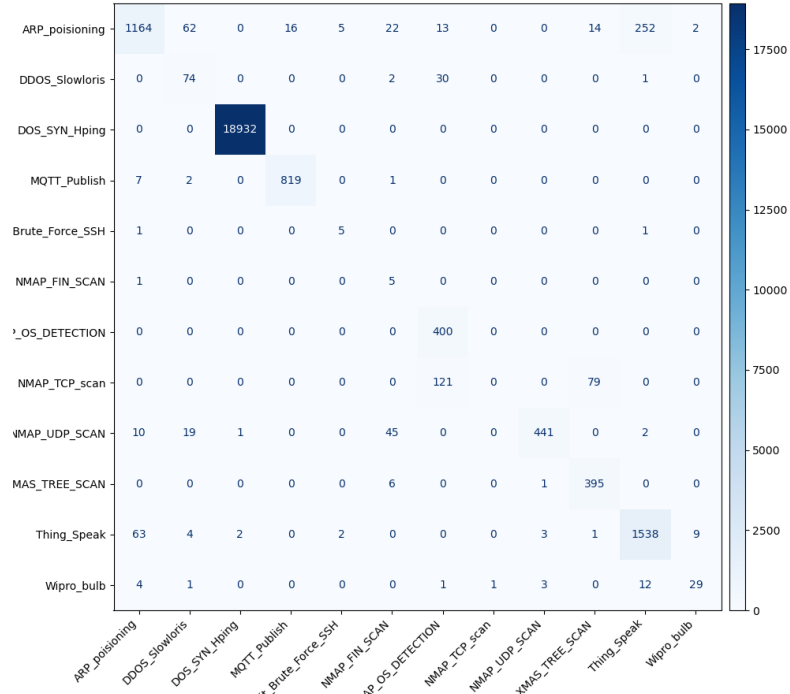


Figure 39: Confusion Matrix of the predictions for Polynomial Kernel SVC (OvO)

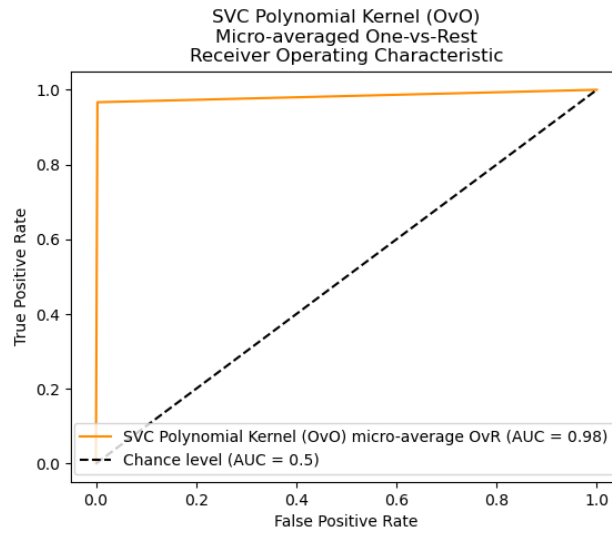


Figure 40: ROC curve for the Polynomial Kernel SVC (OvO)

5.3.2 SVC (One vs. Rest)

5.3.2.1 Linear Kernel (SVC OvR)

Table 19: Linear SVC (OvR) accuracies for 5-fold stratified cross-validation

Fold #1 Accuracy	Fold #2 Accuracy	Fold #3 Accuracy	Fold #4 Accuracy	Fold #5 Accuracy	Average Accuracy
0.88335	0.88489	0.88092	0.87838	0.88697	0.88290

Table 20: Test metrics for the fitted Linear SVC (OvR)

Micro-averaged One-vs-Rest ROC AUC score	Micro-averaged Specificity score	Micro-averaged Precision/Recall/F1 score
0.97707	0.99618	0.95797

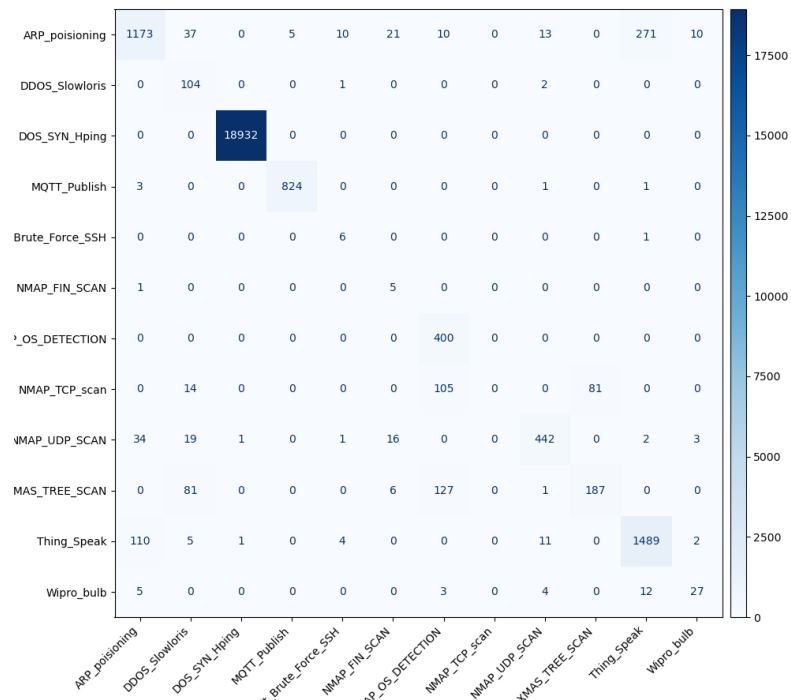


Figure 41: Confusion Matrix of the predictions for Linear Kernel SVC (OvR)

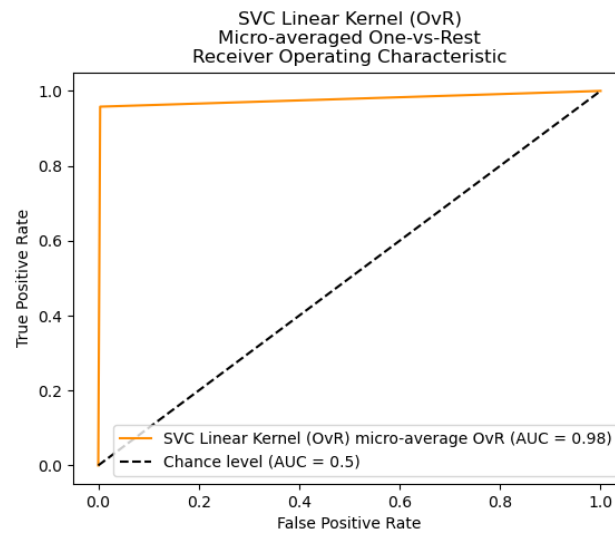


Figure 42: ROC curve for the Linear Kernel SVC (OvR)

5.3.2.2 Radial Basis Function Kernel (SVC OvR)

Table 21: RBF SVC (OvR) accuracies for 5-fold stratified cross-validation

Fold #1 Accuracy	Fold #2 Accuracy	Fold #3 Accuracy	Fold #4 Accuracy	Fold #5 Accuracy	Average Accuracy
0.94234	0.94696	0.94959	0.94695	0.95135	0.94744

Table 22: Test metrics for the fitted RBF SVC (OvR)

Micro-averaged One-vs-Rest ROC AUC score	Micro-averaged Specificity score	Micro-averaged Precision/Recall/F1 score
0.98602	0.99767	0.97437

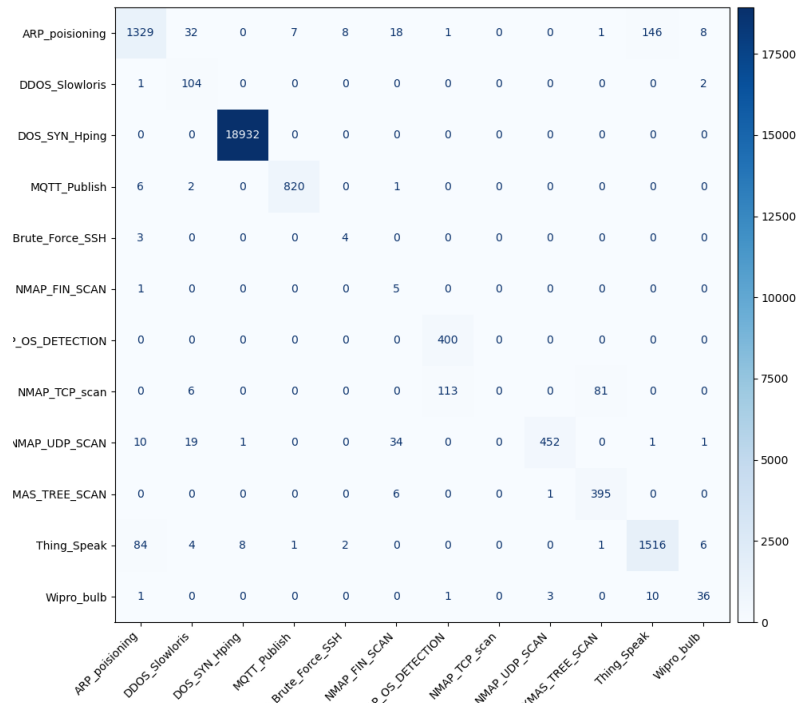


Figure 43: Confusion Matrix of the predictions for RBF Kernel SVC (OvR)

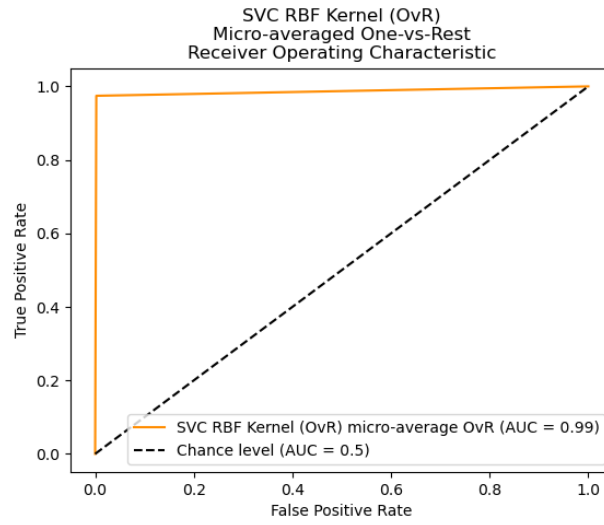


Figure 44: ROC curve for the RBF Kernel SVC (OvR)

5.3.2.3 Polynomial Kernel (SVC OvR)

Table 23: Polynomial SVC (OvR) accuracies for 5-fold stratified cross-validation

Fold #1 Accuracy	Fold #2 Accuracy	Fold #3 Accuracy	Fold #4 Accuracy	Fold #5 Accuracy	Average Accuracy
0.92374	0.93078	0.93033	0.92439	0.93308	0.92846

Table 24: Test metrics for the fitted Polynomial SVC (OvR)

Micro-averaged One-vs-Rest ROC AUC score	Micro-averaged Specificity score	Micro-averaged Precision/Recall/F1 score
0.98179	0.99697	0.96662

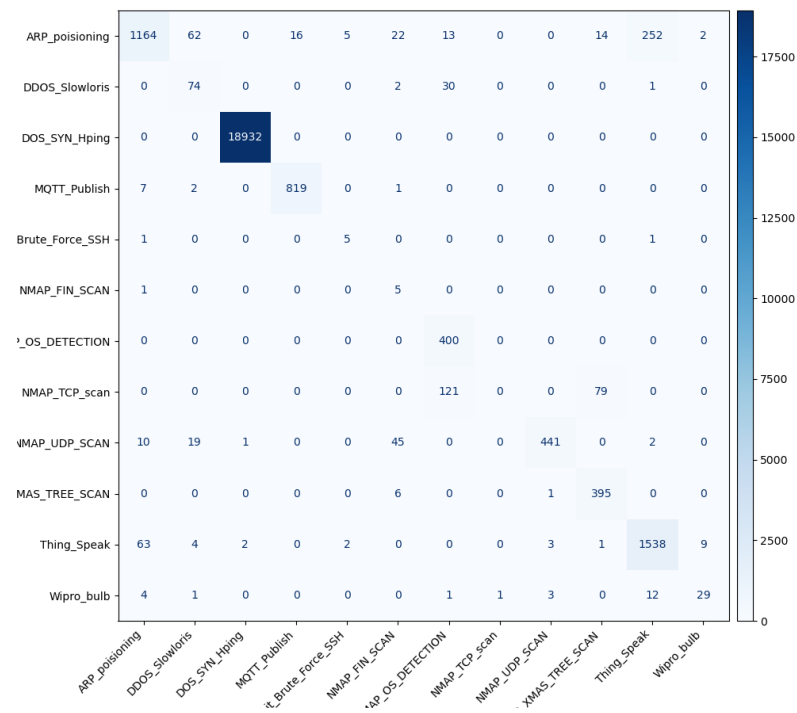


Figure 45: Confusion Matrix of the predictions for Polynomial Kernel SVC (OvR)

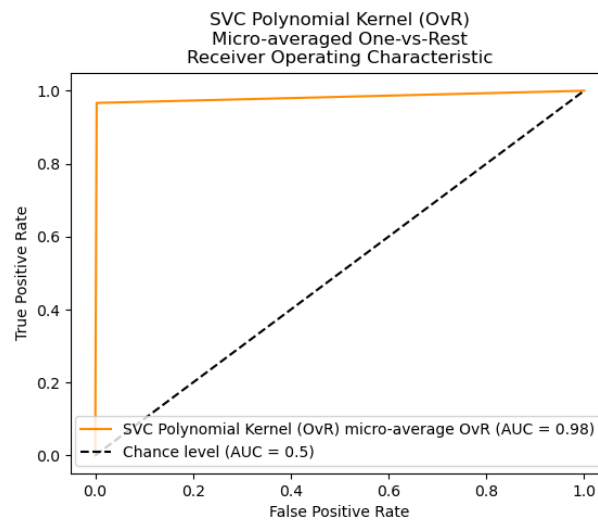


Figure 46: ROC curve for the Polynomial Kernel SVC (OvR)

5.4 Naïve Bayes

The Gaussian Naïve Bayes model was fitted for this scenario.

Table 25: Naïve Bayes accuracies for 5-fold stratified cross-validation

Fold #1 Accuracy	Fold #2 Accuracy	Fold #3 Accuracy	Fold #4 Accuracy	Fold #5 Accuracy	Average Accuracy
0.75085	0.73897	0.73850	0.73696	0.74136	0.74133

Table 26: Test metrics for the fitted Naïve Bayes

Micro-averaged One-vs-Rest ROC AUC score	Micro-averaged Specificity score	Micro-averaged Precision/Recall/F1 score
0.93968	0.98995	0.88942

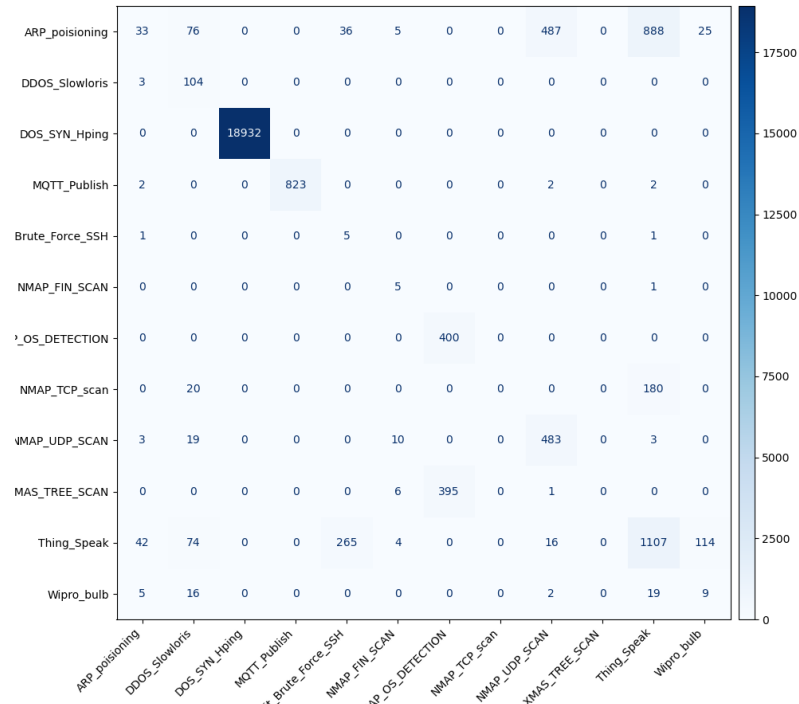


Figure 47: Confusion Matrix of the predictions for Naïve Bayes

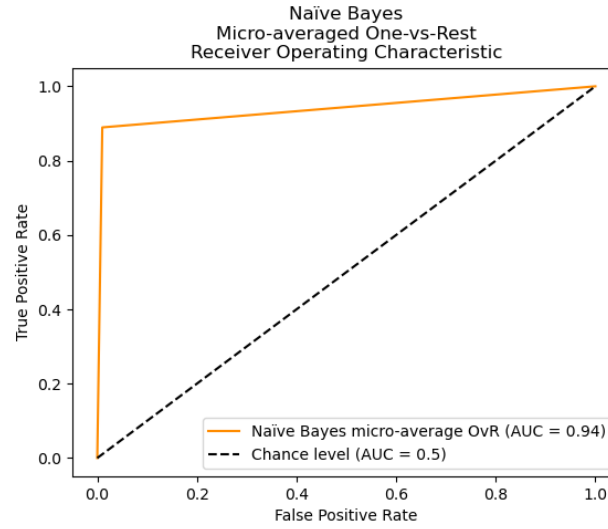


Figure 48: ROC curve for the Naïve Bayes

5.5 K-Nearest Neighbors

5.5.1 Elbow method

The Elbow method gives an optimum K of 1. This is because the initial error is already low at 2% with less room for improvement.

Table 27: *k*-Nearest Neighbors accuracies for 5-fold stratified cross-validation

Fold #1 Accuracy	Fold #2 Accuracy	Fold #3 Accuracy	Fold #4 Accuracy	Fold #5 Accuracy	Average Accuracy
0.98734	0.98988	0.99219	0.98734	0.98811	0.98897

Table 28: Test metrics for the fitted *k*-Nearest Neighbors

Micro-averaged One-vs-Rest ROC AUC score	Micro-averaged Specificity score	Micro-averaged Precision/Recall/F1 score
--	----------------------------------	--

0.98864	0.99811	0.97917
---------	---------	---------

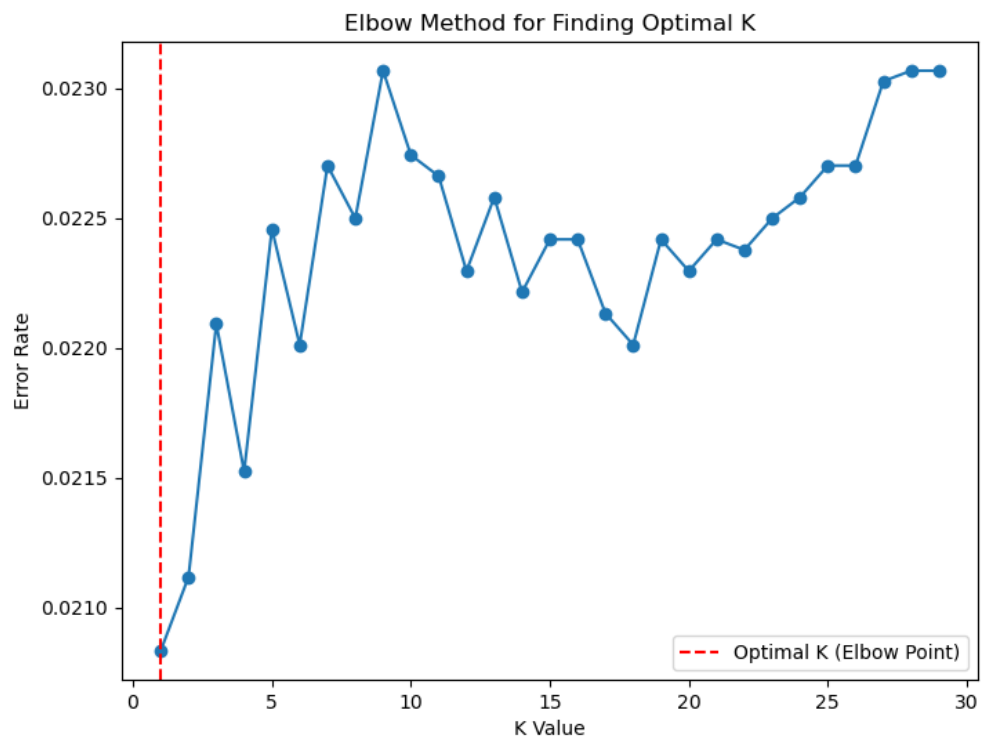


Figure 49: Line plot of error for various k -NN models with increasing k values

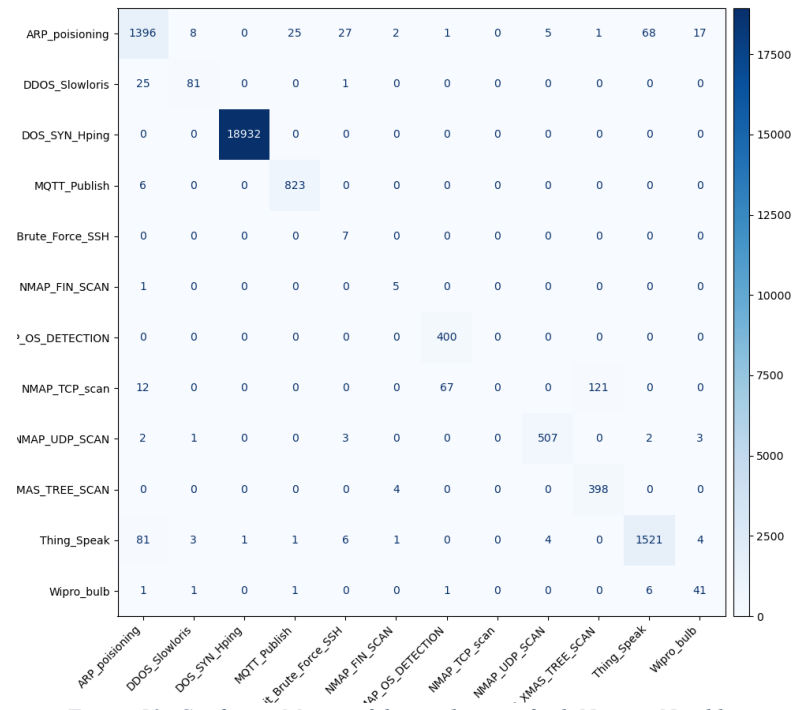


Figure 50: Confusion Matrix of the predictions for k-Nearest Neighbors

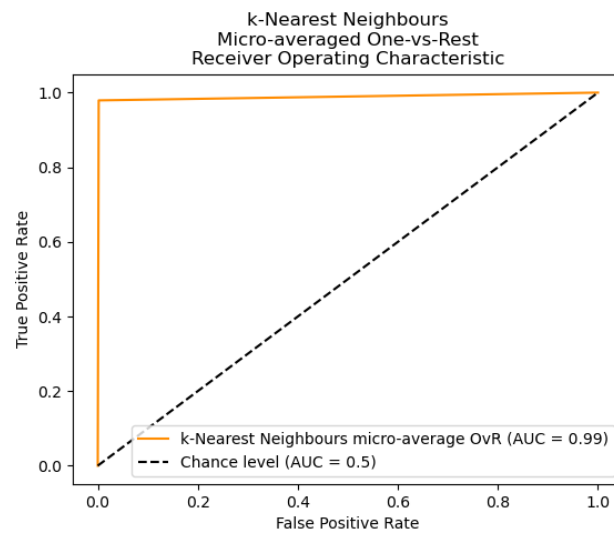


Figure 51: ROC curve for the k-Nearest Neighbors

5.6 Neural network

```
>>> ANN_model.summary()
Model: "sequential"
-----
Layer (type)                 Output Shape              Param #
-----
dense (Dense)                (None, 10)                110
dense_1 (Dense)              (None, 8)                 88
dense_2 (Dense)              (None, 7)                 63
dense_3 (Dense)              (None, 12)               96
-----
Total params: 357
Trainable params: 357
Non-trainable params: 0
-----
```

Figure 52: The schema of the 4-layer neural network

Table 29: Neural network (4-layer) accuracies for 5-fold stratified cross-validation

Fold #1 Accuracy	Fold #2 Accuracy	Fold #3 Accuracy	Fold #4 Accuracy	Fold #5 Accuracy	Average Accuracy
0.79476	0.72972	0.65970	0.54259	0.61039	0.66743

Table 30: Test metrics for the trained Neural network (4-layer)

Micro-averaged One- vs-Rest ROC AUC score	Micro-averaged Specificity score	Micro-averaged Precision/Recall/ F1 score
---	-------------------------------------	--

0.84990	0.92583	0.18413
---------	---------	---------

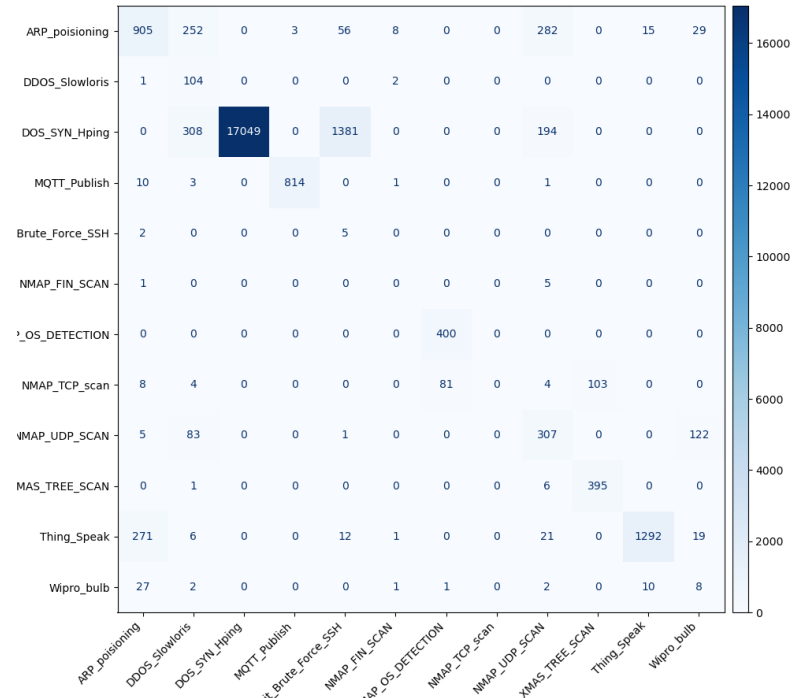


Figure 53: Confusion Matrix of the predictions for Neural net (4-layer)

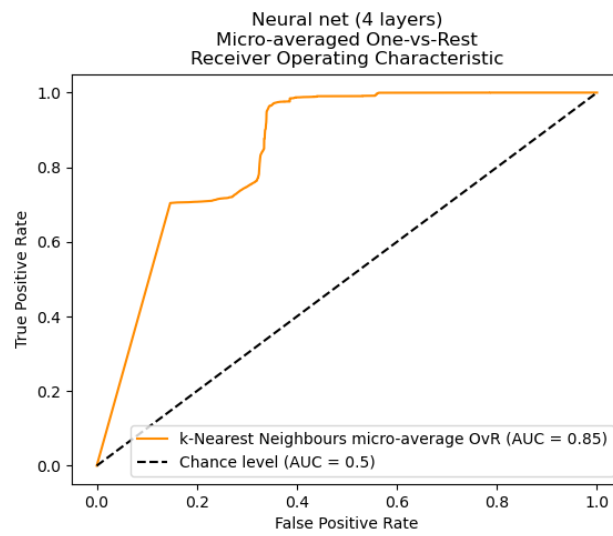


Figure 54: ROC curve for the Neural net (4-layer)

5.7 ROC-AUC Comparison of all classifiers

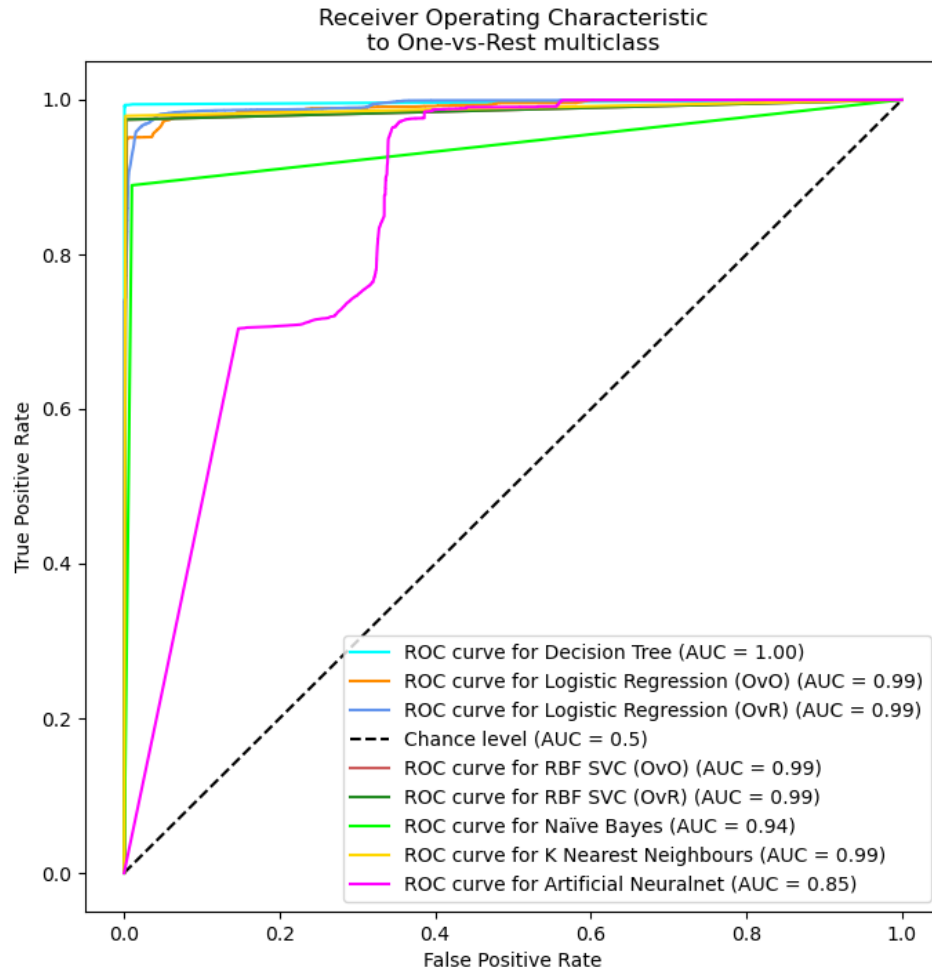


Figure 55: ROC overlay for all classifiers

The best classifier based on both the ROC-AUC analysis as well as the Micro-averaged metrics is the Decision Tree with high accuracy, specificity and F1 score.

Table 31: Comparison of test metrics for all the classifiers

Classifier	Micro-averaged One-vs-Rest ROC AUC score	Micro-averaged Specificity score	Micro-averaged Precision / Recall / F1 score
Decision Tree (Original)	0.99579	0.99927	0.99200
D-Tree (Pre-pruned)	0.99365	0.99183	0.91013
D-Tree (Post-pruned)	0.99674	0.99927	0.99192

Logistic Regression OvO	0.99035	0.99558	0.95135
Logistic Regression OvR	0.99342	0.99234	0.91573
Linear Kernel SVC OvO	0.97707	0.99618	0.95797
RBF Kernel SVC OvO	0.98602	0.99767	0.97437
Poly. Kernel SVC OvO	0.98179	0.99697	0.96662
Linear Kernel SVC OvR	0.97707	0.99618	0.95797
RBF Kernel SVC OvR	0.98602	0.99767	0.97437
Poly. Kernel SVC OvR	0.98179	0.99697	0.96662
Naïve Bayes	0.93968	0.98995	0.88942
k-Nearest Neighbors	0.98864	0.99811	0.97917
Neural net (4-layer)	0.84990	0.92583	0.18413

6. Conclusion: Recommendations

By experimenting on the RT-IoT '22 dataset, a lot regarding the data preprocessing stage was learnt as it too. For the classification analysis, the best classifier is Decision Tree. Clustering and Association Rule mining are not applied but are within scope of the project. Specialized autoencoders are utilized for detecting maladjusted traffic. These are different from the algorithms used in this course and can be explored for this dataset. Features associated with the target variable basically are the one leftover after filtering away during the feature selection. The optimum $k=1$ also suggests that encoders are beneficial for detecting the anomalies.

7. References

Adel Hamdan, M. T. (2025). Feature Reduction and Anomaly Detection in IoT Using Machine Learning Algorithms. *International Journal of Advanced Computer Science and Applications(IJACSA)*, 16(1).

- Almohaimed M, A. F. (2024). Enhancing IoT Network Security Using Feature Selection for Intrusion Detection Systems. *Applied Sciences*, 14(24).
- B S, S. &. (2024). Performance Evaluation of Parametric and Non-Parametric Machine Learning Models using Statistical Analysis for RT-IoT2022 Dataset. *Journal of Scientific & Industrial Research (Vol 83)*, 864-872.
- N. U. Sama, S. U. (2025). Cutting-Edge Intrusion Detection in IoT Networks: A Focus on Ensemble Models. *IEEE Access*, vol. 13, 8375-8392.
- Sharmila, B. N. (2023). R. Quantized autoencoder (QAE) intrusion detection system for anomaly detection in resource-constrained IoT devices using RT-IoT2022 dataset. *Cybersecurity* 6, 41.

8. Appendix

01a-data-preprocessing.py

```
# ***** Final Term Project *****

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv(r'..\data\RT-IOT22.csv')

# ===== Exploratory Data Analysis =====

plt.figure(figsize=(8, 5))
sns.countplot(data=df, x='proto', palette='pastel')
plt.title('Count Plot of variable: proto')
plt.xlabel('proto')
plt.ylabel('Count')
plt.show()

plt.figure(figsize=(8, 5))
sns.countplot(data=df, x='service', palette='pastel')
plt.title('Count Plot of variable: service')
plt.xlabel('service')
plt.ylabel('Count')
plt.show()

plt.figure(figsize=(24, 5))
sns.countplot(data=df, x='Attack_type', palette='pastel')
plt.title('Count Plot of variable: Attack_type')
plt.xlabel('Attack_type')
plt.ylabel('Count')
plt.show()

# ===== Data Cleaning =====

df.isnull().sum().sum() # No null values in the data

# ===== Data duplication & removal (1st Time) =====
```

```

# Checking for duplicate columns by comparing each column with
others
def find_duplicate_columns(df):
    duplicate_column_names = set()
    # Iterate over all the columns in the dataframe
    for i in range(len(df.columns)):
        col1 = df.columns[i]
        # Compare with all other columns
        for j in range(i + 1, len(df.columns)):
            col2 = df.columns[j]
            # Check if the columns are identical
            if df[col1].equals(df[col2]):
                duplicate_column_names.add(col2)
    return list(duplicate_column_names)
# Find duplicate columns in the dataframe
duplicate_columns = find_duplicate_columns(df)
duplicate_columns # empty

def find_constant_columns(df):
    constant_column_names = set()
    # Iterate over all the columns in the dataframe
    for i in range(len(df.columns)):
        col = df.columns[i]
        # Check if the columns are identical
        if len(df[col].unique())==1:
            constant_column_names.add(col)
    return list(constant_column_names)
constant_columns = find_constant_columns(df)
constant_columns # ['bwd_URG_flag_count']

# Dropping 'bwd_URG_flag_count'
df.drop(['bwd_URG_flag_count'], axis=1, inplace=True)

# ===== Test Train Split Stratify =====

from sklearn.model_selection import train_test_split

df_train, df_test = train_test_split(df,
                                     test_size=0.2,
                                     stratify=df['Attack_type'],
                                     random_state=5805,
                                     shuffle=True)

# ===== Class Imbalance Treatment (SMOTE) =====

class_distribution =
df_train.groupby('Attack_type')['Attack_type'].agg(['count'])
class_distribution_test =
df_test.groupby('Attack_type')['Attack_type'].agg(['count'])

plt.figure(figsize=[20,10])
sns.barplot(class_distribution, x='Attack_type', y='count',
            color='b')

```

```

plt.xlabel('Attack_type')
plt.ylabel('Count')
plt.grid(axis='both')
plt.title(label='Class Distribution')
plt.tight_layout()
plt.show()

plt.figure(figsize=[20,10])
sns.barplot(class_distribution_test, x='Attack_type', y='count',
color='b')
plt.xlabel('Attack_type')
plt.ylabel('Count')
plt.grid(axis='both')
plt.title(label='Class Distribution')
plt.tight_layout()
plt.show()

# Undersampling the 'DOS_SYN_Hping' from 94,659 to 6,500
observations

#from imblearn.under_sampling import RandomUnderSampler

#rus = RandomUnderSampler(sampling_strategy='majority',
random_state=5805)
#rus = RandomUnderSampler(sampling_strategy={'DOS_SYN_Hping':6500},
random_state=5805)

#X = df_train[[x for x in df.columns if x != 'Attack_type']]
#y = df_train[['Attack_type']].to_numpy()

#X_res, y_res = rus.fit_resample(X, y)

#df_res = pd.concat([X_res.reset_index(drop=True),
#                    pd.DataFrame(y_res.reshape(-1,1),
columns=['Attack_type'])], \
#                    axis=1)

#class_distribution_2 =
df_res.groupby('Attack_type')['Attack_type'].agg(['count'])

#plt.figure(figsize=[20,10])
#sns.barplot(class_distribution_2, x='Attack_type', y='count',
color='r')
#plt.xlabel('Attack_type')
#plt.ylabel('Count')
#plt.grid(axis='both')
#plt.title(label='Class Distribution')
#plt.tight_layout()
#plt.show()

# Oversampling all the others to majority class' observations using
SMOTE (now 94,649 previously 6500)

```



```

        axis=1)

class_distribution_2 =
df_res.groupby('Attack_type')['Attack_type'].agg(['count'])

plt.figure(figsize=[20,10])
sns.barplot(class_distribution_2, x='Attack_type', y='count',
color='r')
plt.xlabel('Attack_type')
plt.ylabel('Count')
plt.grid(axis='both')
plt.title(label='Class Distribution')
plt.tight_layout()
plt.show()

df_res2 = df_res.copy()

# ===== Data Scaling =====

from sklearn.preprocessing import StandardScaler

numerical_features = list(df_res2.select_dtypes(include=['float64',
'int64']).columns)

categorical_features = [x for x in list(df_res2.columns) if x not in
numerical_features]

scaler = StandardScaler()
scaler.fit(df_res2.loc[:, numerical_features])
array_scaled = scaler.transform(df_res2.loc[:, numerical_features])
df_train_scaled = pd.concat([pd.DataFrame(array_scaled,

columns=numerical_features,
                                index=df_res2.index),
                                df_res2.loc[:, [col for col in
df_res2.columns if col not in numerical_features]]],
                                axis=1)
array_scaled_2 = scaler.transform(df_test.loc[:, numerical_features])
df_test_scaled = pd.concat([pd.DataFrame(array_scaled_2,
                                columns=numerical_features,
                                index=df_test.index),
                                df_test.loc[:, [col for col in
df_test.columns if col not in numerical_features]]],
                                axis=1)

# ===== Data Encoding =====

from sklearn.preprocessing import OneHotEncoder

def custom_combiner(feature, category):
    return str(feature) + "_" + type(category).__name__ + "_" +
str(category)

# One-hot encoding only 'proto' and 'service' (2 out of 3) since

```

```

'Attack_type' is the target label
encoded_features = ['proto', 'service']
oneHotEnc = OneHotEncoder(drop='first',
                           feature_name_combiner=custom_combiner)
oneHotEnc.fit(df_train_scaled[encoded_features])
arr_1HE =
oneHotEnc.transform(df_train_scaled[encoded_features]).toarray()
df_train_1HE = pd.concat([df_train_scaled.drop(encoded_features,
axis=1),
                        pd.DataFrame(arr_1HE,
columns=oneHotEnc.get_feature_names_out(),
index=df_train_scaled.index)],
                        axis=1)
arr_1HE_2 =
oneHotEnc.transform(df_test_scaled[encoded_features]).toarray()
df_test_1HE = pd.concat([df_test_scaled.drop(encoded_features,
axis=1),
                        pd.DataFrame(arr_1HE_2,

columns=oneHotEnc.get_feature_names_out(),
                        index=df_test_scaled.index)],
                        axis=1)

# ===== Outlier Treatment (Inter)
=====

#df_backup = df_1HE.copy()

df_train_1HE_outlier = df_train_1HE.copy()
df_test_1HE_outlier = df_test_1HE.copy()

numerical_features =
list(df_train_1HE.select_dtypes(include=['float16', 'float32',
'float64']).columns) # Should be 91
numerical_features = [x for x in numerical_features if not
(x.startswith('proto') or x.startswith('service'))] # Now 80

for feature in numerical_features:
    df_train_1HE.loc[df_train_1HE[feature] <
df_train_1HE[feature].quantile(1/100), feature] \
        = df_train_1HE[feature].quantile(1 / 100)
    df_train_1HE.loc[df_train_1HE[feature] >
df_train_1HE[feature].quantile(1 - 1/100), feature] \
        = df_train_1HE[feature].quantile(1 - 1/100)

for feature in numerical_features:
    df_test_1HE.loc[df_test_1HE[feature] <
df_test_1HE[feature].quantile(1/100), feature] \
        = df_test_1HE[feature].quantile(1/100)
    df_test_1HE.loc[df_test_1HE[feature] >
df_test_1HE[feature].quantile(1 - 1/100), feature] \
        = df_test_1HE[feature].quantile(1 - 1/100)

# ===== Data duplication & removal (2nd Time)

```

```

=====

duplicate_columns_2a = find_duplicate_columns(df_train_1HE)
duplicate_columns_2a # empty

constant_columns_2a = find_constant_columns(df_train_1HE)
constant_columns_2a #
['fwd_bulk_packets', 'flow_CWR_flag_count', 'flow_ECE_flag_count', 'fwd_bulk_rate', 'fwd_bulk_bytes']

for constant_column in constant_columns_2a:
    # Set the style
    sns.set(style="whitegrid")
    # Create the boxplot
    plt.figure(figsize=(6, 4))
    sns.boxplot(x=df_train_1HE_outlier[constant_column])
    # Show the plot
    plt.title('Box Plot for Outliers of Train Data')
    plt.xlabel(constant_column)
    plt.show()

# Dropping the constant columns for train
df_train_1HE.drop(['fwd_bulk_packets', 'flow_CWR_flag_count', 'flow_ECE_flag_count', 'fwd_bulk_rate', 'fwd_bulk_bytes'],
                  axis=1, inplace=True)

duplicate_columns_2b = find_duplicate_columns(df_test_1HE)
duplicate_columns_2b # empty

constant_columns_2b = find_constant_columns(df_test_1HE)
constant_columns_2b #
['fwd_bulk_rate', 'flow_ECE_flag_count', 'fwd_bulk_bytes', 'flow_CWR_flag_count', 'fwd_bulk_packets',
# 'bwd_bulk_bytes', 'bwd_bulk_packets', 'idle.std', 'bwd_bulk_rate']

for constant_column in constant_columns_2b:
    # Set the style
    sns.set(style="whitegrid")
    # Create the boxplot
    plt.figure(figsize=(6, 4))
    sns.boxplot(x=df_test_1HE_outlier[constant_column])
    # Show the plot
    plt.title('Box Plot for Outliers of Train Data')
    plt.xlabel(constant_column)
    plt.show()

# Dropping the constant columns in train for test
df_test_1HE.drop(['fwd_bulk_packets', 'flow_CWR_flag_count', 'flow_ECE_flag_count', 'fwd_bulk_rate', 'fwd_bulk_bytes'],
                  axis=1, inplace=True)

# Saving this dataframe here as a checkpoint
df_train_1HE.to_csv(r'.\data\df_train_preprocessed.csv',
                    index=False)

```

```

df_test_1HE.to_csv(r'..\data\df_test_preprocessed.csv', index=False)

# ===== Sample Covariance Matrix Heatmap plot
=====

# Exclude non-numeric columns
numeric_df = df_train_1HE.select_dtypes(include=['float64',
'int64'])

# Calculate the covariance matrix
cov_matrix = numeric_df.cov()
# Set up the matplotlib figure
plt.figure(figsize=(22, 17))
# Draw the heatmap with the mask and correct aspect ratio
#sns.heatmap(cov_matrix, cmap='crest', cbar=True, annot=False,
#            fmt='.2f', square=True, linewidths=.5,
xticklabels=True, yticklabels=True)
sns.heatmap(cov_matrix, cmap='coolwarm', cbar=True, annot=False,
            fmt='.2f', square=True, linewidths=.5, xticklabels=True,
yticklabels=True)
# Set the labels
plt.title('Covariance Matrix Heatmap')
plt.xticks(rotation=90)
plt.yticks(rotation=0)
# Show the plot
plt.show()

# ===== Sample Pearson Correlation Coefficient Matrix Heatmap
Plot =====

# Exclude non-numeric columns
numeric_df = df_train_1HE.select_dtypes(include=['float64',
'int64'])

# Set up the matplotlib figure
plt.figure(figsize=(22, 17))

# Correlation Coeff Matrix
corr_numeric_df = numeric_df.corr()

# Draw the heatmap with the mask and correct aspect ratio
sns.heatmap(corr_numeric_df, cmap='coolwarm', annot=False,
            fmt='.2f',
            square=True, cbar_kws={'shrink': .5},
            xticklabels=corr_numeric_df.columns,
            yticklabels=corr_numeric_df.columns)
plt.xticks(rotation=90)
plt.yticks(rotation=0)
plt.title('Pearson Correlation Coefficient Matrix Heatmap')
plt.show()

# ===== Principal Component Analysis =====

from sklearn.decomposition import PCA

```

```

def principal_component_analysis(dataframe, components) -> dict:
    """
    Applies PCA or Principal Component Analysis to the dataframe and
    returns the dataframe after removing the features with the least
    variance.
    """
    pca = PCA(n_components=components, svd_solver='full')
    #svd_solver='covariance_eigh')
    X_columns = list(dataframe.columns)
    X_transformed = pca.fit_transform(dataframe)
    dataframe_transformed = pd.DataFrame(data=X_transformed,
                                         columns=['feature'+str(i+1)
                                         for i in range(X_transformed.shape[1])],
                                         index = dataframe.index)

    pca_expVarCumSum =
np.hstack([np.reshape(pca.get_feature_names_out(), (-1,1)),
np.reshape(pca.explained_variance_ratio_.cumsum(), (-1,1))])
    df_pca_expVarCumSum = pd.DataFrame(data=pca_expVarCumSum,
columns=['PrincipalCom', 'DepVarCumSum'])
    sns.barplot(df_pca_expVarCumSum, x="PrincipalCom",
y="DepVarCumSum", color="y")
    plt.xlabel('Principal Component')
    plt.ylabel('Dependent Variance (Cum Sum)')
    plt.grid(axis='both')
    plt.title(label='Features vs. Dependent Variance')
    plt.tight_layout()
    plt.show()
    return dataframe_transformed,
{df_pca_expVarCumSum['PrincipalCom'].iloc[i] :
int(10000*float(np.round(df_pca_expVarCumSum['DepVarCumSum'].iloc[i]
, 5)))/100
for i in
range(len(df_pca_expVarCumSum))}

plt.figure(figsize=(22, 17))
len(df_train_1HE.columns) # 87
df_pca, explainedVarianceDict =
principal_component_analysis(df_train_1HE[[x for x in
list(df_train_1HE.columns) if x != 'Attack_type']],

components=86) # Max components = Num of features

# ===== Linear Discriminant Analysis =====

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

def linear_discriminant_analysis(dataframe, y_target, components) ->
dict: # Work-In-Progress
    """
    Applies LDA or Linear Discriminant Analysis to the dataframe for
    the selected target column and returns the dataframe after reducing
    the dimensions by one and transforming them for maximum class

```

```

separation.
"""
lda = LinearDiscriminantAnalysis(n_components=components)
if y_target == None or y_target not in dataframe.columns: return
'Invalid input for y_target, cannot perform LDA'
X_columns = list(dataframe.columns)
X_columns.remove(y_target)
X, y = dataframe[X_columns], dataframe[[y_target]]
X_transformed = lda.fit_transform(X, y)
dataframe_transformed =
pd.concat([pd.DataFrame(data=X_transformed,

columns=['feature'+str(i+1) for i in range(X_transformed.shape[1])],
index =
dataframe.index),
y], axis=1)

return dataframe_transformed

max_components = len(df_train_1HE.Attack_type.unique()) - 1
df_lda = linear_discriminant_analysis(df_train_1HE, 'Attack_type',
components=max_components) # Max components is n-1 = 12-1 = 11

corr_numeric_df = df_lda.iloc[:, :-1].corr() # Checking sample
correlation coefficient matrix

# Set up the matplotlib figure
plt.figure(figsize=(8, 5))

# Draw the heatmap with the mask and correct aspect ratio
sns.heatmap(corr_numeric_df, cmap='coolwarm', annot=False,
fmt='.2f',
square=True, cbar_kws={'shrink': .5},
xticklabels=corr_numeric_df.columns,
yticklabels=corr_numeric_df.columns)
plt.xticks(rotation=90)
plt.yticks(rotation=0)
plt.title('Pearson Correlation Coefficient Matrix Heatmap')
plt.show()

# Create a color map
categories = df_lda['Attack_type'].unique()
colors = plt.cm.get_cmap('Set1', len(categories)) # You can change
'Set1' to other colormaps

# Map each category to a color
color_dict = {cat: colors(i) for i, cat in enumerate(categories)}

# Plot
plt.figure(figsize=(8, 6))
for cat in categories:
    subset = df_lda[df_lda['Attack_type'] == cat]
    plt.scatter(subset['feature1'], subset['feature3'], label=cat,
color=color_dict[cat], s=100, alpha=0.7, edgecolors='w')

```

```
plt.xlabel('principal component 1')
plt.ylabel('principal component 3')
plt.title('Scatter Plot Colored by Category')
plt.legend(title='Category')
plt.grid(True)
plt.show()
```

```
# ===== END =====
```

01b-feature-selection.py

```
# ***** Final Term Project *****
```

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
df = pd.read_csv(r'..\data\df_train_preprocessed.csv')
```

```
# = Using the target as label encoded 'Attack_type' for regression =
```

```
# Encoding the following as:-
```

```
# 0 (Normal):      'MQTT_Publish', 'Thing_Speak', 'Wipro_bulb'
# 1 (Anomalous):  'ARP_poisoning', 'DDOS_Slowloris',
                  'DOS_SYN_Hping', 'Metasploit_Brute_Force_SSH',
#                  'NMAP_FIN_SCAN', 'NMAP_OS_DETECTION',
                  'NMAP_TCP_scan', 'NMAP_UDP_SCAN', 'NMAP_XMAS_TREE_SCAN'
```

```
df.Attack_type.unique()
```

```
normal_traffic = ['MQTT_Publish', 'Thing_Speak', 'Wipro_bulb']
anomalous_traffic = ['ARP_poisoning', 'DDOS_Slowloris',
                    'DOS_SYN_Hping', 'Metasploit_Brute_Force_SSH', \
                    'NMAP_FIN_SCAN', 'NMAP_OS_DETECTION',
                    'NMAP_TCP_scan', 'NMAP_UDP_SCAN', 'NMAP_XMAS_TREE_SCAN']
```

```
df_bin = df.copy()
```

```
df_bin.loc[df_bin['Attack_type'].isin(normal_traffic),
           'Attack_type'] = 0
df_bin.loc[df_bin['Attack_type'].isin(anomalous_traffic),
           'Attack_type'] = 1
df_bin.Attack_type = df_bin.Attack_type.astype(float)
```

```
df_bin.Attack_type.unique()
```

```
# ===== Random Forest Analysis =====
```

```
from sklearn.ensemble import RandomForestRegressor
```

```
def random_forest_analysis(dataframe, y_target):
    remaining_columns = list(dataframe.columns)
```

```

    remaining_columns.remove(y_target)
    X = dataframe[remaining_columns]
    y = dataframe[y_target]
    forest = RandomForestRegressor(random_state=6604)
    forest.fit(X, y)
    importances = forest.feature_importances_
    df_featImportance = pd.DataFrame(index=X.columns,
data=np.reshape(importances, (-1,1)), columns=['FeatImport'])\
        .reset_index(drop=False)\
        .rename({'index':'FeatName'}, axis=1)\
        .sort_values(by='FeatImport', ascending=False)
    df_featImportance['FeatImportPerc'] =
(df_featImportance['FeatImport']/df_featImportance['FeatImport'].sum
()*100).apply(round) #.cumsum()
    sns.barplot(df_featImportance, x="FeatImport", y="FeatName",
color='g')
    plt.xlabel('Feature Importance')
    plt.ylabel('Feature Name')
    plt.grid(axis='both')
    plt.title(label='Feature vs. Importance')
    plt.tight_layout()
    plt.show()
    return
{df_featImportance['FeatName'].iloc[i]:float(df_featImportance['Feat
ImportPerc'].iloc[i])
    for i in range(len(df_featImportance))}

plt.figure(figsize=(22,17))
random_forest_analysis(df_bin, y_target='Attack_type') # Requires
continuous variable

# ===== Singular Value Decomposition (1st Trial)
=====

X = df[[x for x in df.columns if x !='Attack_type']]

U,S,Vt=np.linalg.svd(X, full_matrices=False)

K = np.linalg.cond(X)

K # 10008412894282.53

S < 1 # 9 out of 86 have singular value less than 1

S < 0.1 # 3 out of 86 have singular value less than 0.1

S < 0.01 # 1 out of 86 have singular value less than 0.01

# ===== Variance Inflation Factor (1st Trail)
=====

from statsmodels.stats.outliers_influence import
variance_inflation_factor
import statsmodels.api as sm

```



```

def calc_vif(X):
    # Adding constant
    #X_const = sm.add_constant(X) #, has_constant='raise')
    # VIF dataframe
    vif_data = pd.DataFrame()
    vif_data["feature"] = X.columns #X_const.columns
    # calculating VIF for each feature
    vif_data["VIF"] = [variance_inflation_factor(X.values,
i) #X_const.values, i)
                        for i in
range(len(X.columns))] #X_const.columns))]
    return vif_data

import os

directory = r"data\feat_selection_1"

# Check if the directory already exists
if not os.path.exists(directory):
    os.mkdir('.\\'+directory)
    print(f"Directory '{directory}' created.")
else:
    print(f"Directory '{directory}' already exists.")

# vif_data = calc_vif(X) # Give error as there are already columns
nearly constant (after data scaling)
vif_data = calc_vif(X)
vif_data.to_csv('.\\'+directory+r'\vif_0.csv', index=False)

def calc_vif_recursively(X, limit,
directory=r'data\feat_selection_1'):
    i=1
    while True:
        # Adding constant
        #X_const = sm.add_constant(X, has_constant='raise')
        # VIF dataframe
        vif_data = pd.DataFrame()
        vif_data["feature"] = X.columns#X_const.columns
        # calculating VIF for each feature
        vif_data["VIF"] = [variance_inflation_factor(X.values,
i) #X_const.values, i)
                        for i in
range(len(X.columns))] #X_const.columns))]
        vif_data.to_csv('.\\'+directory+r'\vif_{}.csv'.format(i),
index=False)
        max_vif = vif_data["VIF"].max()
        if max_vif < limit:
            break
        elif max_vif == np.inf:
            dropped_feature = list(vif_data.loc[vif_data['VIF'] ==
vif_data['VIF'].max(), 'feature'])[0]
            print(dropped_feature + f" (inf) dropped on iteration

```

```

{i}")
        X.drop(dropped_feature, axis=1, inplace=True)
    else:
        dropped_feature = vif_data.loc[vif_data['VIF'] ==
vif_data['VIF'].max(), 'feature'].item()
        print(dropped_feature+f" dropped on iteration {i}")
        X.drop(dropped_feature, axis=1, inplace=True)
    i+=1
    return X

#X_vif_test = calc_vif_recursively(X, 1000000)
X_vif = calc_vif_recursively(X, 10)

# ===== Singular Value Decomposition (2nd Trial)
=====

U,S,Vt=np.linalg.svd(X_vif, full_matrices=False)

K = np.linalg.cond(X_vif)

K # 363.9950917670543

S < 1 # 1 out of 33 have singular value less than 1

S < 0.1 # 0 out of 33 have singular value less than 0.1

S < 0.01 # 0 out of 33 have singular value less than 0.01

# ===== Variance Inflation Factor (2nd Trail)
=====

directory_2 = r"data\feat_selection_2"

# Check if the directory already exists
if not os.path.exists(directory_2):
    os.mkdir('.\\'+directory_2)
    print(f"Directory '{directory_2}' created.")
else:
    print(f"Directory '{directory_2}' already exists.")

#X_vif_2 = calc_vif_recursively(X=X_vif, limit=1000000,
directory=directory_2)
X_vif_2 = calc_vif_recursively(X=X_vif, limit=5,
directory=directory_2)

# ===== Singular Value Decomposition (3rd Trial)
=====

U,S,Vt=np.linalg.svd(X_vif_2, full_matrices=False)

K = np.linalg.cond(X_vif_2)

K # 341.67531791850536

```

```

S < 1 # 1 out of 30 have singular value less than 1

S < 0.1 # 0 out of 30 have singular value less than 0.1

S < 0.01 # 0 out of 30 have singular value less than 0.1

df_featsel = pd.concat([X_vif_2, df_bin[['Attack_type']], axis=1)

# ===== Random Forest Analysis (2nd Trial)
=====

plt.figure(figsize=(22,17))
gini_importances_dict = random_forest_analysis(df_featsel,
y_target='Attack_type') # Requires continuous variable
gini_importances_dict

# {'flow_iat.min': 85.0,
#  'fwd_pkts_payload.min': 3.0,
#  'active.min': 2.0,
#  'flow_RST_flag_count': 2.0,
#  'service_str_ssh': 2.0,
#  'fwd_last_window_size': 1.0,
#  'id.resp_p': 1.0,
#  'bwd_data_pkts_tot': 1.0,
#  'id.orig_p': 1.0,
#  'down_up_ratio': 1.0,
#  'bwd_subflow_bytes': 0.0,
#  'flow_pkts_payload.min': 0.0,
#  'service_str_dns': 0.0,
#  'flow_pkts_per_sec': 0.0,
#  'payload_bytes_per_second': 0.0,
#  'bwd_pkts_payload.min': 0.0,
#  'service_str_ntp': 0.0,
#  'flow_FIN_flag_count': 0.0,
#  'bwd_iat.min': 0.0,
#  'active.std': 0.0,
#  'idle.min': 0.0,
#  'service_str_http': 0.0,
#  'service_str_irc': 0.0,
#  'fwd_URG_flag_count': 0.0,
#  'bwd_iat.std': 0.0,
#  'service_str_dhcp': 0.0,
#  'idle.std': 0.0,
#  'service_str_ssl': 0.0,
#  'bwd_bulk_packets': 0.0,
#  'service_str_radius': 0.0}

final_columns =
['flow_iat.min', 'fwd_pkts_payload.min', 'active.min', 'flow_RST_flag_c
ount', 'service_str_ssh', \

'fwd_last_window_size', 'id.resp_p', 'bwd_data_pkts_tot', 'id.orig_p', '
down_up_ratio']
final_columns += ['Attack_type']

```

```

# Not making any more changes

df_train = pd.concat([X_vif_2, df[['Attack_type']]], axis=1)
df_train = df_train[final_columns].copy()

df_prefinal_train = pd.concat([X_vif_2, df[['Attack_type']]],
axis=1)

df_2 = pd.read_csv(r'..\data\df_test_preprocessed.csv')
df_2_featsel = df_2[X_vif_2.columns].copy()
df_prefinal_test = pd.concat([df_2_featsel, df_2[['Attack_type']]],
axis=1)
df_test = df_prefinal_test[final_columns].copy()

# ===== Singular Value Decomposition (3rd Trial)
=====

U,S,Vt=np.linalg.svd(df_train[[x for x in final_columns if x !=
'Attack_type']], full_matrices=False)

K = np.linalg.cond(df_train[[x for x in final_columns if x !=
'Attack_type']])

K # 12.458703470104622

S < 1 # 0 out of 10 have singular value less than 1

S < 0.1 # 0 out of 10 have singular value less than 0.1

S < 0.01 # 0 out of 10 have singular value less than 0.01

# Saving the data as the second checkpoint!

df_prefinal_train.to_csv('./data/df_prefinal_train.csv',
index=False)
df_prefinal_test.to_csv('./data/df_prefinal_test.csv', index=False)
df_train.to_csv('./data/df_train.csv', index=False)
df_test.to_csv('./data/df_test.csv', index=False)

# ===== END =====

```

02-linear-regression.py

```

# ***** Final Term Project *****

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from tensorflow.python.feature_column.feature_column import
linear_model

df_train = pd.read_csv(r'..\data\df_prefinal_train.csv')
df_test = pd.read_csv(r'..\data\df_prefinal_test.csv')

```

```

train_distribution =
df_train.groupby('Attack_type')['Attack_type'].agg(['count'])

train_distribution

test_distribution =
df_test.groupby('Attack_type')['Attack_type'].agg(['count'])

test_distribution

def custom_combiner(feature, category):
    return str(feature) + "_" + type(category).__name__ + "_" +
str(category)

from sklearn.preprocessing import OneHotEncoder

oneHotEnc = OneHotEncoder(drop='first',
                           feature_name_combiner=custom_combiner)
oneHotEnc.fit(df_train[['Attack_type']])
arr_train_1HE =
oneHotEnc.transform(df_train[['Attack_type']]).toarray()
df_train_1HE = pd.concat([df_train.drop(['Attack_type'],
axis=1).reset_index(drop=True), \
                          pd.DataFrame(arr_train_1HE,
columns=oneHotEnc.get_feature_names_out()),
                          axis=1)

arr_test_1HE =
oneHotEnc.transform(df_test[['Attack_type']]).toarray()
df_test_1HE = pd.concat([df_test.drop(['Attack_type'],
axis=1).reset_index(drop=True), \
                          pd.DataFrame(arr_test_1HE,
columns=oneHotEnc.get_feature_names_out()),
                          axis=1)

# ===== Singular Value Decomposition (Re-Trial)
=====

X = df_train_1HE[[x for x in df_train_1HE.columns if x !=
'down_up_ratio']]

U,S,Vt=np.linalg.svd(X, full_matrices=False)

K = np.linalg.cond(X)

K # 343.6331441909699

S < 1 # 1 out of 41 have singular value less than 1

S < 0.1 # 0 out of 41 have singular value less than 0.1

S < 0.01 # 0 out of 41 have singular value less than 0.01

# ===== Backward Stepwise Linear Regression =====

```

```

# Using the target variable as 'down_up_ratio'

import statsmodels.api as sm
from prettytable import PrettyTable

y_train = df_train_1HE.iloc[:, 4].copy()
y_test = df_test_1HE.iloc[:, 4].copy()
X_train = df_train_1HE.drop('down_up_ratio', axis=1).copy()
X_test = df_test_1HE.drop('down_up_ratio', axis=1).copy()
X_train = sm.add_constant(X_train)
X_test = sm.add_constant(X_test, has_constant='add') # Had to
manually add since there are zero columns
bslr_table = PrettyTable()
#table.header = False
bslr_table.field_names = ["Iteration", "AIC", "BIC", "R^2", "Adj-
R^2", "DroppedFeature", "PValue from T-test",
                          "PValue from F-test"]

droppedFeatureList = []
modelSummaryList = []
for i in range(len(df_train_1HE.columns)):
    lr_model_1 = sm.OLS(y_train, X_train).fit()
    y_pred = lr_model_1.predict(X_test)
    #print(predSales_CarSeat_scaled)
    # AIC, BIC and Adjusted R2 and t-test
    #print(f'AIC is {lr_model_1.aic}')
    #print(f'BIC is {lr_model_1.bic}')
    #print(f'Adj-R^2 is {lr_model_1.rsquared_adj}')
    #print(f'T-values are: {lr_model_1.t_test}')
    r_matrix_list = []
    for j in range(len(lr_model_1.params)):
        r_matrix = np.zeros_like(lr_model_1.params)
        r_matrix[j] = 1
        r_matrix_list.append(r_matrix)
    tTest_pValue_param_pairs =
sorted([(lr_model_1.t_test(r_matrix_list[k]).pvalue,
lr_model_1.params.index[k]) \
        for k in
range(len(lr_model_1.params))],
        reverse=True)

    pValueOfDroppedFeature, droppedFeature =
tTest_pValue_param_pairs[0]
    droppedFeatureList.append(droppedFeature)
    modelSummaryList.append(lr_model_1.summary())
    X_test.drop(droppedFeature, axis=1, inplace=True)
    X_train.drop(droppedFeature, axis=1, inplace=True)
    bslr_table.add_rows([[i+1, np.round(lr_model_1.aic, 3),
np.round(lr_model_1.bic, 3), \
        np.round(lr_model_1.rsquared, 3), \
        np.round(lr_model_1.rsquared_adj, 3),
droppedFeature, \
        np.round(pValueOfDroppedFeature, 3),
np.round(lr_model_1.f_pvalue, 7)]]
print(f'[1.] The eliminated features are

```

```

{droppedFeatureList[:4]}.)
print(f'        The remaining features we kept are:
{droppedFeatureList[4:]}.)
print('        The table below includes the stats from each
iteration of backward LR:-')
print(bslr_table)

selected_features = ['Attack_type_str_NMAP_FIN_SCAN',
'Attack_type_str_NMAP_OS_DETECTION',
'Attack_type_str_NMAP_TCP_scan',
'Attack_type_str_NMAP_UDP_SCAN', 'bwd_pkts_payload.min',
'flow_FIN_flag_count', 'flow_pkts_payload.min',
'fwd_URG_flag_count', 'bwd_iat.min',
'Attack_type_str_MQTT_Publish',
'Attack_type_str_Thing_Speak', 'bwd_data_pkts_tot',
'Attack_type_str_Metasploit_Brute_Force_SSH',
'Attack_type_str_DDOS_Slowloris', 'fwd_pkts_payload.min']

y2_train = df_train_1HE.iloc[:, 4].copy()
y2_test = df_test_1HE.iloc[:, 4].copy()
X2_train = df_train_1HE.drop('down_up_ratio', axis=1).copy()
X2_test = df_test_1HE.drop('down_up_ratio', axis=1).copy()
X2_train = X2_train[selected_features]
X2_test = X2_test[selected_features]
X2_train = sm.add_constant(X2_train)
X2_test = sm.add_constant(X2_test, has_constant='add') # Had to
manually add since there are zero columns
lr_model = sm.OLS(y2_train, X2_train).fit()
y2_pred = lr_model.predict(X2_test)

# AIC, BIC and Adjusted R2 and t-test
print(f'AIC is {lr_model.aic}')
print(f'BIC is {lr_model.bic}')
print(f'Adj-R^2 is {lr_model.rsquared_adj}')
print(f'T-test statistic and pValue : {lr_model.t_test}')
print(f'Summary of LR model is\n : {lr_model.summary()}')

# MSE:
from sklearn.metrics import mean_squared_error
MSE = mean_squared_error(y2_test, y2_pred)
MSE # 0.04387669691482695

# Index vs. Test, Predicted
plt.figure(figsize=(22, 17))
# Plot Train Data
#plt.scatter(X2_train.index, y2_train, color='red', label='Train
Data', alpha=0.6)
# Plot Test Data
plt.scatter(X2_test.index, y2_test, color='green', label='Test
Data', alpha=0.6)
# Plot Predictions
plt.scatter(X2_test.index, y2_pred, color='blue', label='Predicted
(OLS)', alpha = 0.4)#, linewidth=2)
# Customize plot

```

```

plt.xlabel('Feature (X)')
plt.ylabel('Target (y)')
plt.title('Train, Test, and Predicted Data for OLS Model')
plt.legend()
plt.grid(True)
plt.show()

# Get confidence intervals for the predictions
sample_mean = sum(y_pred)/len(y_pred)
stdev = np.sqrt(sum((y2_pred - y2_test)**2) / (len(y2_test)))
lower_bound = sample_mean - 1.96*stdev
upper_bound = sample_mean + 1.96*stdev
lower_bound # -0.41092551569492053
upper_bound # 0.4101876771549334

```

03a-classifiers-part1

```

# ***** Final Term Project *****

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df_train = pd.read_csv(r'./data/df_train.csv')
df_test = pd.read_csv(r'./data/df_test.csv')

X_train = df_train[[x for x in df_train.columns if x !=
'Attack_type']]
y_train = df_train[['Attack_type']]
X_test = df_test[[x for x in df_test.columns if x != 'Attack_type']]
y_test = df_test[['Attack_type']]

import os

directory = r"data\classification"

# Check if the directory already exists
if not os.path.exists(directory):
    os.mkdir('.\\'+directory)
    print(f"Directory '{directory}' created.")
else:
    print(f"Directory '{directory}' already exists.")

'''
from itertools import cycle

def roc_multiclass(y_test, y_score, n_classes):
    fpr = dict()
    tpr = dict()
    roc_auc = dict()
    for i in range(n_classes):
        fpr[i], tpr[i], _ = roc_curve(y_test[:, i], y_score[:, i])
        roc_auc[i] = auc(fpr[i], tpr[i])

```



```

        colors = cycle(['blue', 'red', 'green'])
        for i, color in zip(range(n_classes), colors):
            plt.plot(fpr[i], tpr[i], color=color, #lw=lw,
                    label='ROC curve of class {0} (area = {1:0.2f})'
                        ''.format(i, roc_auc[i]))
        plt.plot([0, 1], [0, 1], 'k--', )#lw=lw)
        plt.xlim([-0.05, 1.0])
        plt.ylim([0.0, 1.05])
        plt.xlabel('False Positive Rate')
        plt.ylabel('True Positive Rate')
        plt.title('Receiver operating characteristic for multi-class
data')
        plt.legend(loc="lower right")
        plt.show()
'''

'''
for class_id in range(len(d_tree.classes_)):
#for class_id in d_tree.classes_:
    class_of_interest = d_tree.classes_[class_id]
    display = RocCurveDisplay.from_predictions(
        y_test_bin[:, class_id],
        y_proba[:, class_id],
        name=f"{class_of_interest} vs the rest",
        color="darkorange",
        plot_chance_level=True,
        despine=True,
    )
    _ = display.ax_.set(
        xlabel="False Positive Rate",
        ylabel="True Positive Rate",
        title="One-vs-Rest ROC curves:\nVirginica vs (Setosa &
Versicolor)",
    )
plt.show()
'''

# ===== Decision Tree =====

from sklearn.metrics import accuracy_score, recall_score, roc_curve,
roc_auc_score, auc, RocCurveDisplay
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree

d_tree = DecisionTreeClassifier(random_state=5805) # DOUBT
d_tree.fit(X_train,y_train)
y_pred = d_tree.predict(X_test)
y_pred_train = d_tree.predict(X_train)
print(f'Accuracy on train: %.5f' % round(accuracy_score(y_train,
y_pred_train), 2))
print(f'Accuracy on test: %.5f' % round(accuracy_score(y_test,
y_pred), 2))

```

```

from sklearn.model_selection import StratifiedKFold

skf = StratifiedKFold(n_splits=5, random_state=5805, shuffle=True)

accuracies = []
for train, validation in skf.split(X_train, y_train):
    # Using the indices to split the data into train (2) and
    validation
    XV_train = X_train.iloc[train, :]
    XV_valid = X_train.iloc[validation, :]
    yv_train = y_train.iloc[train, :]
    yv_valid = y_train.iloc[validation, :]

    # Train the model
    d_tree.fit(XV_train, yv_train)

    # Predict on validation set
    yv_pred = d_tree.predict(XV_valid)

    # Evaluate the model
    accuracy = accuracy_score(yv_valid, yv_pred)
    accuracies.append(accuracy)
    print(f'Fold accuracy: {accuracy:.5f}')

# Print avg accuracy across all 5 folds:
print(f"\nAvg accuracy: {sum(accuracies)/len(accuracies):.5f}")

print(d_tree.get_params())

plt.figure(figsize=(16,8))
tree.plot_tree(d_tree,rounded=True,filled=True)
plt.show()

# RESULTS:-
from sklearn.preprocessing import label_binarize
from sklearn.metrics import RocCurveDisplay
from sklearn.metrics import precision_score, recall_score, f1_score
from imblearn.metrics import specificity_score
from sklearn.metrics import ConfusionMatrixDisplay

#plt.figure(figsize=(22, 17))
model = d_tree
cnf_mtx = ConfusionMatrixDisplay.from_estimator(model, X_test,
y_test, cmap=plt.cm.Blues)
fig, ax = plt.subplots(figsize=(10,10))
#cnf_mtx.plot(ax=ax, cmap=plt.cm.Blues)
# Deactivate default colorbar
cnf_mtx.plot(ax=ax, colorbar=False, cmap=plt.cm.Blues)
plt.xticks(rotation=45, ha='right')
# Adding custom colorbar
cax =
fig.add_axes([ax.get_position().x1+0.01,ax.get_position().y0,0.02,ax
.get_position().height])
plt.colorbar(cnf_mtx.im_, cax=cax)

```

```

plt.tight_layout()
plt.show()

y_proba = model.predict_proba(X_test)
y_test_bin = label_binarize(y_test, classes=model.classes_)

df_proba = pd.DataFrame(y_proba, columns=model.classes_)
df_testbin = pd.DataFrame(y_test_bin, columns=model.classes_)
df_proba.to_csv(r'./data/classification/y_proba_'+ 'dtree1.csv',
index=False)
df_testbin.to_csv(r'./data/classification/y_testbin_'+ 'dtree1.csv',
index=False)

display = RocCurveDisplay.from_predictions(
    y_test_bin.ravel(),
    y_proba.ravel(),
    name="Decision Tree micro-average OvR",
    color="darkorange",
    plot_chance_level=True,
    #despine=True,
)
_ = display.ax_.set(
    xlabel="False Positive Rate",
    ylabel="True Positive Rate",
    title="Decision Tree\nMicro-averaged One-vs-Rest\nReceiver
Operating Characteristic",
)
plt.show()

micro_roc_auc_ovr = roc_auc_score(
    y_test_bin,
    y_proba,
    multi_class="ovr",
    average="micro",
)
print(f"Micro-averaged One-vs-Rest ROC AUC
score:\n{micro_roc_auc_ovr:.5f}")

precision = precision_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Precision score:\n{precision:.5f}")

recall = recall_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Recall score:\n{recall:.5f}")

specificity = specificity_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Specificity score:\n{specificity:.5f}")

f1 = f1_score(y_test, y_pred, average='micro')
print(f"Micro-averaged F1 score:\n{f1:.5f}")

# ===== Decision Tree - Pre-pruning
=====

from sklearn.model_selection import GridSearchCV

```

```

#from sklearn.metrics import confusion_matrix

tuned_parameters = [{'max_depth': [1, 2, 3, 4, 5],
                        'min_samples_split': [20,30,40],
                        'min_samples_leaf': [10,20,30],
                        'criterion':['gini','entropy','log_loss'],
                        'splitter':['best','random'],
                        'max_features':['sqrt','log2']}]
d_tree2 = DecisionTreeClassifier(random_state=5805)
grid_search = GridSearchCV(estimator=d_tree2,
param_grid=tuned_parameters, cv=5, n_jobs=-1)
grid_search.fit(X_train,y_train)
print(grid_search.best_params_)

criterion, max_depth, max_features, min_samples_leaf,
min_samples_split, splitter =
tuple(grid_search.best_params_.values())
d_tree2 = DecisionTreeClassifier(criterion=criterion,
                                max_depth=max_depth,
                                max_features=max_features,
                                min_samples_leaf=min_samples_leaf,

min_samples_split=min_samples_split,
                                splitter=splitter)

accuracies = []
for train, validation in skf.split(X_train, y_train):
    # Using the indices to split the data into train (2) and
    validation
    XV_train = X_train.iloc[train, :]
    XV_valid = X_train.iloc[validation, :]
    yv_train = y_train.iloc[train, :]
    yv_valid = y_train.iloc[validation, :]

    # Train the model
    d_tree2.fit(XV_train, yv_train)

    # Predict on validation set
    yv_pred = d_tree2.predict(XV_valid)

    # Evaluate the model
    accuracy = accuracy_score(yv_valid, yv_pred)
    accuracies.append(accuracy)
    print(f'Fold accuracy: {accuracy:.5f}')

# Print avg accuracy across all 5 folds:
print(f"\nAvg accuracy: {sum(accuracies)/len(accuracies):.5f}")

d_tree2.fit(X_train,y_train)
y_pred_train = d_tree2.predict(X_train)
y_pred = d_tree2.predict(X_test)
print(f'Accuracy on train: %.5f' % round(accuracy_score(y_train,
y_pred_train), 2))
accuracy_dt_prepruned = round(accuracy_score(y_test, y_pred), 2)

```

```

print(f'Accuracy on test: %.5f' % accuracy_dt_prepruned)

#cnf_mtx_dt_prepruned = confusion_matrix(y_pred, y_test)
#df_cnf_mtx_dt_prepruned = pd.DataFrame(cnf_mtx_dt_prepruned,
index=['True-0', 'True-1'], columns=['Pred-0', 'Pred-1'])
#recall_dt_prepruned = recall_score(y_test, y_pred, average='micro')
#fpr, tpr, thresholds = roc_curve(y_test, y_pred)
#roc_auc_dt_prepruned = auc(fpr, tpr)
#display = RocCurveDisplay(fpr=fpr, tpr=tpr,
roc_auc=roc_auc_dt_prepruned,
# estimator_name=f'Decision Tree Prepruned,
AUC = {roc_auc_dt_prepruned}')
#display.plot()
#plt.show()

plt.figure(figsize=(16,8))
tree.plot_tree(d_tree2,rounded=True,filled=True)
plt.show()

# RESULTS:-
plt.figure(figsize=(22, 17))
model = d_tree2
cnf_mtx = ConfusionMatrixDisplay.from_estimator(model, X_test,
y_test, cmap=plt.cm.Blues)
fig, ax = plt.subplots(figsize=(10,10))
#cnf_mtx.plot(ax=ax, cmap=plt.cm.Blues)
# Deactivate default colorbar
cnf_mtx.plot(ax=ax, colorbar=False, cmap=plt.cm.Blues)
plt.xticks(rotation=45, ha='right')
# Adding custom colorbar
cax =
fig.add_axes([ax.get_position().x1+0.01,ax.get_position().y0,0.02,ax
.get_position().height])
plt.colorbar(cnf_mtx.im_, cax=cax)
#plt.tight_layout()
plt.show()

y_proba = model.predict_proba(X_test)
y_test_bin = label_binarize(y_test, classes=model.classes_)

df_proba = pd.DataFrame(y_proba, columns=model.classes_)
df_testbin = pd.DataFrame(y_test_bin, columns=model.classes_)
df_proba.to_csv(r'..\data\classification\y_proba_'+ 'dtree2.csv',
index=False)
df_testbin.to_csv(r'..\data\classification\y_testbin_'+ 'dtree2.csv',
index=False)

display = RocCurveDisplay.from_predictions(
    y_test_bin.ravel(),
    y_proba.ravel(),
    name="Decision Tree (Prepruned) micro-average OvR",
    color="darkorange",
    plot_chance_level=True,
    #despine=True,

```

```

)
- = display.ax_.set(
    xlabel="False Positive Rate",
    ylabel="True Positive Rate",
    title="Decision Tree (Prepruned)\nMicro-averaged One-vs-
Rest\nReceiver Operating Characteristic",
)
plt.show()

micro_roc_auc_ovr = roc_auc_score(
    y_test_bin,
    y_proba,
    multi_class="ovr",
    average="micro",
)
print(f"Micro-averaged One-vs-Rest ROC AUC
score:\n{micro_roc_auc_ovr:.5f}")

precision = precision_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Precision score:\n{precision:.5f}")

recall = recall_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Recall score:\n{recall:.5f}")

specificity = specificity_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Specificity score:\n{specificity:.5f}")

f1 = f1_score(y_test, y_pred, average='micro')
print(f"Micro-averaged F1 score:\n{f1:.5f}")

# ===== Decision Tree - Post-pruning
=====

path = d_tree.cost_complexity_pruning_path(X_train, y_train)
print(path)
ccp_alphas, impurities = path.ccp_alphas, path.impurities
ccp_alphas, impurities = path.ccp_alphas, path.impurities
print("ccp alpha will give list of values :",ccp_alphas)
print("*****")
print("Impurities in Decision Tree :",impurities)

plt.figure(figsize=(10, 6))
plt.plot(ccp_alphas, impurities)
plt.xlabel("effective alpha")
plt.ylabel("total impurity of leaves")
plt.show()

d_trees = []

for ccp_alpha in ccp_alphas[0:201:20]: # Choosing 11 across 232 ccp
values
    d_tree_alpha = DecisionTreeClassifier(random_state=5805,
ccp_alpha=ccp_alpha)
    d_tree_alpha.fit(X_train, y_train)

```

```

    d_trees.append(d_tree_alpha)

acc_scores_tr = np.array([accuracy_score(y_train,
d_tree_alpha.predict(X_train)) for d_tree_alpha in d_trees])
acc_scores = np.array([accuracy_score(y_test,
d_tree_alpha.predict(X_test)) for d_tree_alpha in d_trees])

acc_df = pd.DataFrame(np.hstack((ccp_alphas[0:201:20].reshape(-1,1),
                                acc_scores_tr.reshape(-1,1),
                                acc_scores.reshape(-1,1))),
                      columns=['CCP Alpha', 'Train Accuracy', 'Test
Accuracy']).set_index('CCP Alpha', drop=True)

plt.figure(figsize=(10, 6))
plt.grid()
plt.plot(acc_df, label=['Train Accuracy', 'Test Accuracy'])
plt.xlabel("CCP alpha")
plt.ylabel("Accuracy scores")
plt.legend()
plt.show()

optimum_alpha = 0.000039 #0.000069
print(f'The optimum ccp_alpha for highest accuracy on test is
{optimum_alpha}')
```

```

d_tree3=DecisionTreeClassifier(random_state=0, ccp_alpha=optimum_alpha)

accuracies = []
for train, validation in skf.split(X_train, y_train):
    # Using the indices to split the data into train (2) and
validation
    XV_train = X_train.iloc[train, :]
    XV_valid = X_train.iloc[validation, :]
    yv_train = y_train.iloc[train, :]
    yv_valid = y_train.iloc[validation, :]

    # Train the model
    d_tree3.fit(XV_train, yv_train)

    # Predict on validation set
    yv_pred = d_tree3.predict(XV_valid)

    # Evaluate the model
    accuracy = accuracy_score(yv_valid, yv_pred)
    accuracies.append(accuracy)
    print(f'Fold accuracy: {accuracy:.5f}')
```

```

# Print avg accuracy across all 5 folds:
print(f"\nAvg accuracy: {sum(accuracies)/len(accuracies):.5f}")

d_tree3.fit(X_train, y_train)
y_pred_train = d_tree3.predict(X_train)
y_pred = d_tree3.predict(X_test)
```

```

plt.figure(figsize=(12,8))
tree.plot_tree(d_tree3,rounded=True,filled=True)
plt.show()

accuracy_dt_postpruned = round(accuracy_score(y_test,y_pred), 5) #2)
print(f'Accuracy of pruned tree {accuracy_dt_postpruned}')
```

```

#cnf_mtx_dt_postpruned = confusion_matrix(y_pred, y_test)
#df_cnf_mtx_dt_postpruned = pd.DataFrame(cnf_mtx_dt_postpruned,
index=['True-0','True-1'], columns=['Pred-0','Pred-1'])
#recall_dt_postpruned = recall_score(y_test, y_pred,
average='micro')
#fpr, tpr, thresholds = roc_curve(y_test, y_pred)
#roc_auc_dt_postpruned = auc(fpr, tpr)
#display = RocCurveDisplay(fpr=fpr, tpr=tpr,
roc_auc=roc_auc_dt_postpruned,
# estimator_name=f'Decision Tree
Postpruned, AUC = {roc_auc_dt_postpruned}')
#display.plot()
#plt.show()
```

```

# RESULTS:-
#plt.figure(figsize=(22, 17))
model = d_tree3
cnf_mtx = ConfusionMatrixDisplay.from_estimator(model, X_test,
y_test, cmap=plt.cm.Blues)
fig, ax = plt.subplots(figsize=(10,10))
#cnf_mtx.plot(ax=ax, cmap=plt.cm.Blues)
# Deactivate default colorbar
cnf_mtx.plot(ax=ax, colorbar=False, cmap=plt.cm.Blues)
plt.xticks(rotation=45, ha='right')
# Adding custom colorbar
cax =
fig.add_axes([ax.get_position().x1+0.01,ax.get_position().y0,0.02,ax
.get_position().height])
plt.colorbar(cnf_mtx.im_, cax=cax)
#plt.tight_layout()
plt.show()
```

```

y_proba = model.predict_proba(X_test)
y_test_bin = label_binarize(y_test, classes=model.classes_)

df_proba = pd.DataFrame(y_proba, columns=model.classes_)
df_testbin = pd.DataFrame(y_test_bin, columns=model.classes_)
df_proba.to_csv(r'..\data\classification\y_proba_'+ 'dtree3.csv',
index=False)
df_testbin.to_csv(r'..\data\classification\y_testbin_'+ 'dtree3.csv',
index=False)
```

```

display = RocCurveDisplay.from_predictions(
    y_test_bin.ravel(),
    y_proba.ravel(),
    name="Decision Tree (Postpruned) micro-average OvR",
    color="darkorange",
```



```

        plot_chance_level=True,
        #despine=True,
    )
    _ = display.ax_.set(
        xlabel="False Positive Rate",
        ylabel="True Positive Rate",
        title="Decision Tree (Postpruned)\nMicro-averaged One-vs-
Rest\nReceiver Operating Characteristic",
    )
plt.show()

micro_roc_auc_ovr = roc_auc_score(
    y_test_bin,
    y_proba,
    multi_class="ovr",
    average="micro",
)
print(f"Micro-averaged One-vs-Rest ROC AUC
score:\n{micro_roc_auc_ovr:.5f}")

precision = precision_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Precision score:\n{precision:.5f}")

recall = recall_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Recall score:\n{recall:.5f}")

specificity = specificity_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Specificity score:\n{specificity:.5f}")

f1 = f1_score(y_test, y_pred, average='micro')
print(f"Micro-averaged F1 score:\n{f1:.5f}")

# ===== Logistic Regression (One vs All)
=====

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold

skf = StratifiedKFold(n_splits=5, random_state=5805, shuffle=True)

log_model = LogisticRegression(multi_class='ovr', random_state=5805)

#tuned_parameters = [{'penalty': ['l1', 'l2', 'elasticnet', None],
#                      'solver': ['saga']}]

#skfcv_model = GridSearchCV(estimator=log_model,
param_grid=tuned_parameters, cv=skf, scoring='recall')

accuracies = []

for train, validation in skf.split(X_train, y_train):
    # Using the indices to split the data into train (2) and
validation
    XV_train = X_train.iloc[train, :]

```

```

XV_valid = X_train.iloc[validation, :]
yv_train = y_train.iloc[train, :]
yv_valid = y_train.iloc[validation, :]

# Train the model
log_model.fit(XV_train, yv_train)

# Predict on validation set
yv_pred = log_model.predict(XV_valid)

# Evaluate the model
accuracy = accuracy_score(yv_valid, yv_pred)
accuracies.append(accuracy)
print(f'Fold accuracy: {accuracy:.5f}')
```

Print avg accuracy across all 5 folds:

```
print(f"\nAvg accuracy: {sum(accuracies)/len(accuracies):.5f}")
```

```
log_model.fit(X_train, y_train)
y_pred_train = log_model.predict(X_train)
y_pred = log_model.predict(X_test)
```

```
print(f'Accuracy on train: %.5f' % round(accuracy_score(y_train,
y_pred_train), 2))
accuracy_log_model = round(accuracy_score(y_test, y_pred), 2)
print(f'Accuracy on test: %.5f' % accuracy_log_model)
```

```
#cnf_mtx_log_model = confusion_matrix(y_pred, y_test)
#df_cnf_mtx_log_model = pd.DataFrame(cnf_mtx_log_model,
index=['True-0', 'True-1'], columns=['Pred-0', 'Pred-1'])
#recall_log_model = recall_score(y_test, y_pred)
#fpr, tpr, thresholds = roc_curve(y_test, y_pred)
#roc_auc_log_model = auc(fpr, tpr)
#display = RocCurveDisplay(fpr=fpr, tpr=tpr,
roc_auc=roc_auc_log_model,
# estimator_name=f'Logistic Regression, AUC
= {roc_auc_log_model}')
#display.plot()
#plt.show()
```

RESULTS:-

```
#plt.figure(figsize=(22, 17))
model = log_model
cnf_mtx = ConfusionMatrixDisplay.from_estimator(model, X_test,
y_test, cmap=plt.cm.Blues)
fig, ax = plt.subplots(figsize=(10,10))
#cnf_mtx.plot(ax=ax, cmap=plt.cm.Blues)
# Deactivate default colorbar
cnf_mtx.plot(ax=ax, colorbar=False, cmap=plt.cm.Blues)
plt.xticks(rotation=45, ha='right')
# Adding custom colorbar
cax =
fig.add_axes([ax.get_position().x1+0.01,ax.get_position().y0,0.02,ax
.get_position().height])
```

```

plt.colorbar(cnf_mtx.im_, cax=cax)
#plt.tight_layout()
plt.show()

y_proba = model.predict_proba(X_test)
y_test_bin = label_binarize(y_test, classes=model.classes_)

df_proba = pd.DataFrame(y_proba, columns=model.classes_)
df_testbin = pd.DataFrame(y_test_bin, columns=model.classes_)
df_proba.to_csv(r'./data/classification/y_proba_'+ 'logR_OvR.csv',
index=False)
df_testbin.to_csv(r'./data/classification/y_testbin_'+ 'logR_OvR.csv'
, index=False)

display = RocCurveDisplay.from_predictions(
    y_test_bin.ravel(),
    y_proba.ravel(),
    name="Logistic Regression (OvR) micro-average OvR",
    color="darkorange",
    plot_chance_level=True,
    #despine=True,
)
_ = display.ax_.set(
    xlabel="False Positive Rate",
    ylabel="True Positive Rate",
    title="Logistic Regression (OvR)\nMicro-averaged One-vs-
Rest\nReceiver Operating Characteristic",
)
plt.show()

micro_roc_auc_ovr = roc_auc_score(
    y_test_bin,
    y_proba,
    multi_class="ovr",
    average="micro",
)
print(f"Micro-averaged One-vs-Rest ROC AUC
score:\n{micro_roc_auc_ovr:.5f}")

precision = precision_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Precision score:\n{precision:.5f}")

recall = recall_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Recall score:\n{recall:.5f}")

specificity = specificity_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Specificity score:\n{specificity:.5f}")

f1 = f1_score(y_test, y_pred, average='micro')
print(f"Micro-averaged F1 score:\n{f1:.5f}")

# ===== Logistic Regression (One vs One)
=====

```

```

from sklearn.multiclass import OneVsOneClassifier

#log_model = LogisticRegression(multi_class='ovo',
random_state=5805)
log_model =
OneVsOneClassifier(LogisticRegression(random_state=5805))

accuracies = []

for train, validation in skf.split(X_train, y_train):
    # Using the indices to split the data into train (2) and
validation
    XV_train = X_train.iloc[train, :]
    XV_valid = X_train.iloc[validation, :]
    yv_train = y_train.iloc[train, :]
    yv_valid = y_train.iloc[validation, :]

    # Train the model
    log_model.fit(XV_train, yv_train)

    # Predict on validation set
    yv_pred = log_model.predict(XV_valid)

    # Evaluate the model
    accuracy = accuracy_score(yv_valid, yv_pred)
    accuracies.append(accuracy)
    print(f'Fold accuracy: {accuracy:.5f}')

# Print avg accuracy across all 5 folds:
print(f"\nAvg accuracy: {sum(accuracies)/len(accuracies):.5f}")

log_model.fit(X_train, y_train)
y_pred_train = log_model.predict(X_train)
y_pred = log_model.predict(X_test)

print(f'Accuracy on train: %.5f' % round(accuracy_score(y_train,
y_pred_train), 2))
accuracy_log_model = round(accuracy_score(y_test, y_pred), 2)
print(f'Accuracy on test: %.5f' % accuracy_log_model)

# RESULTS:-
#plt.figure(figsize=(22, 17))
model = log_model
cnf_mtx = ConfusionMatrixDisplay.from_estimator(model, X_test,
y_test, cmap=plt.cm.Blues)
fig, ax = plt.subplots(figsize=(10,10))
#cnf_mtx.plot(ax=ax, cmap=plt.cm.Blues)
# Deactivate default colorbar
cnf_mtx.plot(ax=ax, colorbar=False, cmap=plt.cm.Blues)
plt.xticks(rotation=45, ha='right')
# Adding custom colorbar
cax =
fig.add_axes([ax.get_position().x1+0.01,ax.get_position().y0,0.02,ax
.get_position().height])

```

```

plt.colorbar(cnf_mtx.im_, cax=cax)
#plt.tight_layout()
plt.show()

#y_proba = label_binarize(y_pred, classes=model.classes_)
y_proba = log_model.decision_function(X_test)
y_test_bin = label_binarize(y_test, classes=model.classes_)

df_proba = pd.DataFrame(y_proba, columns=model.classes_)
df_testbin = pd.DataFrame(y_test_bin, columns=model.classes_)
df_proba.to_csv(r'./data/classification/y_proba_'+logR_OvO.csv',
index=False)
df_testbin.to_csv(r'./data/classification/y_testbin_'+logR_OvO.csv',
, index=False)

display = RocCurveDisplay.from_predictions(
    y_test_bin.ravel(),
    y_proba.ravel(),
    name="Logistic Regression (OvO) micro-average OvR",
    color="darkorange",
    plot_chance_level=True,
    #despine=True,
)
_ = display.ax_.set(
    xlabel="False Positive Rate",
    ylabel="True Positive Rate",
    title="Logistic Regression (OvO)\nMicro-averaged One-vs-
Rest\nReceiver Operating Characteristic",
)
plt.show()

micro_roc_auc_ovr = roc_auc_score(
    y_test_bin,
    y_proba,
    multi_class="ovr",
    average="micro",
)
print(f"Micro-averaged One-vs-Rest ROC AUC
score:\n{micro_roc_auc_ovr:.5f}")

precision = precision_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Precision score:\n{precision:.5f}")

recall = recall_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Recall score:\n{recall:.5f}")

specificity = specificity_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Specificity score:\n{specificity:.5f}")

f1 = f1_score(y_test, y_pred, average='micro')
print(f"Micro-averaged F1 score:\n{f1:.5f}")

# ===== END =====

```

03b-classifiers-part2

```

# ***** Final Term Project *****

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df_train = pd.read_csv(r'./data/df_train.csv')
df_test = pd.read_csv(r'./data/df_test.csv')

X_train = df_train[[x for x in df_train.columns if x !=
'Attack_type']]
y_train = df_train[['Attack_type']]
X_test = df_test[[x for x in df_test.columns if x != 'Attack_type']]
y_test = df_test[['Attack_type']]

# ===== Support Vector Machine (Linear Kernel, One vs One)
=====

from sklearn.svm import SVC
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score, recall_score, roc_curve,
roc_auc_score, auc, RocCurveDisplay

skf = StratifiedKFold(n_splits=5, random_state=5805, shuffle=True)

linear_svc_ovo = SVC(kernel='linear', decision_function_shape='ovo',
random_state=5805)

accuracies = []

for train, validation in skf.split(X_train, y_train):
    # Using the indices to split the data into train (2) and
    validation
    XV_train = X_train.iloc[train, :]
    XV_valid = X_train.iloc[validation, :]
    yv_train = y_train.iloc[train, :]
    yv_valid = y_train.iloc[validation, :]

    # Train the model
    linear_svc_ovo.fit(XV_train, yv_train)

    # Predict on validation set
    yv_pred = linear_svc_ovo.predict(XV_valid)

    # Evaluate the model
    accuracy = accuracy_score(yv_valid, yv_pred)
    accuracies.append(accuracy)
    print(f'Fold accuracy: {accuracy:.5f}')

# Print avg accuracy across all 5 folds:

```

```

print(f"\nAvg accuracy: {sum(accuracies)/len(accuracies):.5f}")

linear_svc_ovo.fit(X_train,y_train)
y_pred_train = linear_svc_ovo.predict(X_train)
y_pred = linear_svc_ovo.predict(X_test)

print(f'Accuracy on train: %.5f' % round(accuracy_score(y_train,
y_pred_train), 2))
accuracy_log_model = round(accuracy_score(y_test, y_pred), 2)
print(f'Accuracy on test: %.5f' % accuracy_log_model)

# RESULTS:-
from sklearn.preprocessing import label_binarize
from sklearn.metrics import RocCurveDisplay
from sklearn.metrics import precision_score, recall_score, f1_score
from imblearn.metrics import specificity_score
from sklearn.metrics import ConfusionMatrixDisplay

#plt.figure(figsize=(22, 17))
model = linear_svc_ovo
cnf_mtx = ConfusionMatrixDisplay.from_estimator(model, X_test,
y_test, cmap=plt.cm.Blues)
fig, ax = plt.subplots(figsize=(10,10))
#cnf_mtx.plot(ax=ax, cmap=plt.cm.Blues)
# Deactivate default colorbar
cnf_mtx.plot(ax=ax, colorbar=False, cmap=plt.cm.Blues)
plt.xticks(rotation=45, ha='right')
# Adding custom colorbar
cax =
fig.add_axes([ax.get_position().x1+0.01,ax.get_position().y0,0.02,ax
.get_position().height])
plt.colorbar(cnf_mtx.im_, cax=cax)
#plt.tight_layout()
plt.show()

y_proba = label_binarize(y_pred, classes=model.classes_)
y_test_bin = label_binarize(y_test, classes=model.classes_)

df_proba = pd.DataFrame(y_proba, columns=model.classes_)
df_testbin = pd.DataFrame(y_test_bin, columns=model.classes_)
df_proba.to_csv(r'..\data\classification\y_proba_'+ 'linear_svc_ovo.cs
v', index=False)
df_testbin.to_csv(r'..\data\classification\y_testbin_'+ 'linear_svc_ov
o.csv', index=False)

display = RocCurveDisplay.from_predictions(
    y_test_bin.ravel(),
    y_proba.ravel(),
    name="SVC Linear Kernel (OvO) micro-average OvR",
    color="darkorange",
    plot_chance_level=True,
    #despine=True,
)
_ = display.ax_.set(

```

```

        xlabel="False Positive Rate",
        ylabel="True Positive Rate",
        title="SVC Linear Kernel (OvO)\nMicro-averaged One-vs-
Rest\nReceiver Operating Characteristic",
    )
plt.show()

micro_roc_auc_ovr = roc_auc_score(
    y_test_bin,
    y_proba,
    multi_class="ovr",
    average="micro",
)
print(f"Micro-averaged One-vs-Rest ROC AUC
score:\n{micro_roc_auc_ovr:.5f}")

precision = precision_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Precision score:\n{precision:.5f}")

recall = recall_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Recall score:\n{recall:.5f}")

specificity = specificity_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Specificity score:\n{specificity:.5f}")

f1 = f1_score(y_test, y_pred, average='micro')
print(f"Micro-averaged F1 score:\n{f1:.5f}")

# ===== Support Vector Machine (Radial Basis Func Kernel, One vs
One) =====

rbf_svc_ovo = SVC(kernel='rbf', decision_function_shape='ovo',
random_state=5805)

accuracies = []

for train, validation in skf.split(X_train, y_train):
    # Using the indices to split the data into train (2) and
validation
    XV_train = X_train.iloc[train, :]
    XV_valid = X_train.iloc[validation, :]
    yv_train = y_train.iloc[train, :]
    yv_valid = y_train.iloc[validation, :]

    # Train the model
    rbf_svc_ovo.fit(XV_train, yv_train)

    # Predict on validation set
    yv_pred = rbf_svc_ovo.predict(XV_valid)

    # Evaluate the model
    accuracy = accuracy_score(yv_valid, yv_pred)
    accuracies.append(accuracy)
    print(f'Fold accuracy: {accuracy:.5f}')

```



```

# Print avg accuracy across all 5 folds:
print(f"\nAvg accuracy: {sum(accuracies)/len(accuracies):.5f}")

rbf_svc_ovo.fit(X_train,y_train)
y_pred_train = rbf_svc_ovo.predict(X_train)
y_pred = rbf_svc_ovo.predict(X_test)

print(f'Accuracy on train: %.5f' % round(accuracy_score(y_train,
y_pred_train), 2))
accuracy_log_model = round(accuracy_score(y_test, y_pred), 2)
print(f'Accuracy on test: %.5f' % accuracy_log_model)

# RESULTS:-
#plt.figure(figsize=(22, 17))
model = rbf_svc_ovo
cnf_mtx = ConfusionMatrixDisplay.from_estimator(model, X_test,
y_test, cmap=plt.cm.Blues)
fig, ax = plt.subplots(figsize=(10,10))
#cnf_mtx.plot(ax=ax, cmap=plt.cm.Blues)
# Deactivate default colorbar
cnf_mtx.plot(ax=ax, colorbar=False, cmap=plt.cm.Blues)
plt.xticks(rotation=45, ha='right')
# Adding custom colorbar
cax =
fig.add_axes([ax.get_position().x1+0.01,ax.get_position().y0,0.02,ax
.get_position().height])
plt.colorbar(cnf_mtx.im_, cax=cax)
#plt.tight_layout()
plt.show()
y_proba = label_binarize(y_pred, classes=model.classes_)
y_test_bin = label_binarize(y_test, classes=model.classes_)

df_proba = pd.DataFrame(y_proba, columns=model.classes_)
df_testbin = pd.DataFrame(y_test_bin, columns=model.classes_)
df_proba.to_csv(r'..\data\classification\y_proba_'+rbf_svc_ovo.csv',
index=False)
df_testbin.to_csv(r'..\data\classification\y_testbin_'+rbf_svc_ovo.c
sv', index=False)

display = RocCurveDisplay.from_predictions(
    y_test_bin.ravel(),
    y_proba.ravel(),
    name="SVC RBF Kernel (OvO) micro-average OvR",
    color="darkorange",
    plot_chance_level=True,
    #despine=True,
)
_ = display.ax_.set(
    xlabel="False Positive Rate",
    ylabel="True Positive Rate",
    title="SVC RBF Kernel (OvO)\nMicro-averaged One-vs-
Rest\nReceiver Operating Characteristic",
)

```

```

plt.show()

micro_roc_auc_ovr = roc_auc_score(
    y_test_bin,
    y_proba,
    multi_class="ovr",
    average="micro",
)
print(f"Micro-averaged One-vs-Rest ROC AUC
score:\n{micro_roc_auc_ovr:.5f}")

precision = precision_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Precision score:\n{precision:.5f}")

recall = recall_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Recall score:\n{recall:.5f}")

specificity = specificity_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Specificity score:\n{specificity:.5f}")

f1 = f1_score(y_test, y_pred, average='micro')
print(f"Micro-averaged F1 score:\n{f1:.5f}")

# ===== Support Vector Machine (Polynomial Kernel, One vs One)
=====

poly_svc_ovo = SVC(kernel='poly', decision_function_shape='ovo',
random_state=5805)

accuracies = []

for train, validation in skf.split(X_train, y_train):
    # Using the indices to split the data into train (2) and
validation
    XV_train = X_train.iloc[train, :]
    XV_valid = X_train.iloc[validation, :]
    yv_train = y_train.iloc[train, :]
    yv_valid = y_train.iloc[validation, :]

    # Train the model
    poly_svc_ovo.fit(XV_train, yv_train)

    # Predict on validation set
    yv_pred = poly_svc_ovo.predict(XV_valid)

    # Evaluate the model
    accuracy = accuracy_score(yv_valid, yv_pred)
    accuracies.append(accuracy)
    print(f'Fold accuracy: {accuracy:.5f}')

# Print avg accuracy across all 5 folds:
print(f"\nAvg accuracy: {sum(accuracies)/len(accuracies):.5f}")

poly_svc_ovo.fit(X_train, y_train)

```

```

y_pred_train = poly_svc_ovo.predict(X_train)
y_pred = poly_svc_ovo.predict(X_test)

print(f'Accuracy on train: %.5f' % round(accuracy_score(y_train,
y_pred_train), 2))
accuracy_log_model = round(accuracy_score(y_test, y_pred), 2)
print(f'Accuracy on test: %.5f' % accuracy_log_model)

# RESULTS:-
#plt.figure(figsize=(22, 17))
model = poly_svc_ovo
cnf_mtx = ConfusionMatrixDisplay.from_estimator(model, X_test,
y_test, cmap=plt.cm.Blues)
fig, ax = plt.subplots(figsize=(10,10))
#cnf_mtx.plot(ax=ax, cmap=plt.cm.Blues)
# Deactivate default colorbar
cnf_mtx.plot(ax=ax, colorbar=False, cmap=plt.cm.Blues)
plt.xticks(rotation=45, ha='right')
# Adding custom colorbar
cax =
fig.add_axes([ax.get_position().x1+0.01,ax.get_position().y0,0.02,ax
.get_position().height])
plt.colorbar(cnf_mtx.im_, cax=cax)
#plt.tight_layout()
plt.show()

y_proba = label_binarize(y_pred, classes=model.classes_)
y_test_bin = label_binarize(y_test, classes=model.classes_)

df_proba = pd.DataFrame(y_proba, columns=model.classes_)
df_testbin = pd.DataFrame(y_test_bin, columns=model.classes_)
df_proba.to_csv(r'..\data\classification\y_proba_'+poly_svc_ovo.csv',
, index=False)
df_testbin.to_csv(r'..\data\classification\y_testbin_'+poly_svc_ovo.
csv', index=False)

display = RocCurveDisplay.from_predictions(
    y_test_bin.ravel(),
    y_proba.ravel(),
    name="SVC Polynomial Kernel (OvO) micro-average OvR",
    color="darkorange",
    plot_chance_level=True,
    #despine=True,
)
_ = display.ax_.set(
    xlabel="False Positive Rate",
    ylabel="True Positive Rate",
    title="SVC Polynomial Kernel (OvO) \nMicro-averaged One-vs-
Rest\nReceiver Operating Characteristic",
)
plt.show()

micro_roc_auc_ovr = roc_auc_score(
    y_test_bin,

```

```

    y_proba,
    multi_class="ovr",
    average="micro",
)
print(f"Micro-averaged One-vs-Rest ROC AUC
score:\n{micro_roc_auc_ovr:.5f}")

precision = precision_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Precision score:\n{precision:.5f}")

recall = recall_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Recall score:\n{recall:.5f}")

specificity = specificity_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Specificity score:\n{specificity:.5f}")

f1 = f1_score(y_test, y_pred, average='micro')
print(f"Micro-averaged F1 score:\n{f1:.5f}")

# ===== Support Vector Machine (Linear Kernel, One vs All)
=====

linear_svc_ova = SVC(kernel='linear', decision_function_shape='ovr',
random_state=5805)

accuracies = []

for train, validation in skf.split(X_train, y_train):
    # Using the indices to split the data into train (2) and
    validation
    XV_train = X_train.iloc[train, :]
    XV_valid = X_train.iloc[validation, :]
    yv_train = y_train.iloc[train, :]
    yv_valid = y_train.iloc[validation, :]

    # Train the model
    linear_svc_ova.fit(XV_train, yv_train)

    # Predict on validation set
    yv_pred = linear_svc_ova.predict(XV_valid)

    # Evaluate the model
    accuracy = accuracy_score(yv_valid, yv_pred)
    accuracies.append(accuracy)
    print(f'Fold accuracy: {accuracy:.5f}')

# Print avg accuracy across all 5 folds:
print(f"\nAvg accuracy: {sum(accuracies)/len(accuracies):.5f}")

linear_svc_ova.fit(X_train, y_train)
y_pred_train = linear_svc_ova.predict(X_train)
y_pred = linear_svc_ova.predict(X_test)

print(f'Accuracy on train: %.5f' % round(accuracy_score(y_train,

```

```

y_pred_train), 2))
accuracy_log_model = round(accuracy_score(y_test, y_pred), 2)
print(f'Accuracy on test: %.5f' % accuracy_log_model)

# RESULTS:-
plt.figure(figsize=(22, 17))
model = linear_svc_ova
cnf_mtx = ConfusionMatrixDisplay.from_estimator(model, X_test,
y_test, cmap=plt.cm.Blues)
fig, ax = plt.subplots(figsize=(10,10))
#cnf_mtx.plot(ax=ax, cmap=plt.cm.Blues)
# Deactivate default colorbar
cnf_mtx.plot(ax=ax, colorbar=False, cmap=plt.cm.Blues)
plt.xticks(rotation=45, ha='right')
# Adding custom colorbar
cax =
fig.add_axes([ax.get_position().x1+0.01,ax.get_position().y0,0.02,ax
.get_position().height])
plt.colorbar(cnf_mtx.im_, cax=cax)
plt.tight_layout()
plt.show()

y_proba = label_binarize(y_pred, classes=model.classes_)
y_test_bin = label_binarize(y_test, classes=model.classes_)

df_proba = pd.DataFrame(y_proba, columns=model.classes_)
df_testbin = pd.DataFrame(y_test_bin, columns=model.classes_)
df_proba.to_csv(r'.\data\classification\y_proba_'+ 'linear_svc_ova.cs
v', index=False)
df_testbin.to_csv(r'.\data\classification\y_testbin_'+ 'linear_svc_ov
a.csv', index=False)

display = RocCurveDisplay.from_predictions(
    y_test_bin.ravel(),
    y_proba.ravel(),
    name="SVC Linear Kernel (OvR) micro-average OvR",
    color="darkorange",
    plot_chance_level=True,
    #despine=True,
)
_ = display.ax_.set(
    xlabel="False Positive Rate",
    ylabel="True Positive Rate",
    title="SVC Linear Kernel (OvR)\nMicro-averaged One-vs-
Rest\nReceiver Operating Characteristic",
)
plt.show()

micro_roc_auc_ovr = roc_auc_score(
    y_test_bin,
    y_proba,
    multi_class="ovr",
    average="micro",
)

```

```

print(f"Micro-averaged One-vs-Rest ROC AUC
score:\n{micro_roc_auc_ovr:.5f}")

precision = precision_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Precision score:\n{precision:.5f}")

recall = recall_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Recall score:\n{recall:.5f}")

specificity = specificity_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Specificity score:\n{specificity:.5f}")

f1 = f1_score(y_test, y_pred, average='micro')
print(f"Micro-averaged F1 score:\n{f1:.5f}")

# ===== Support Vector Machine (Radial Basis Func Kernel, One vs
All) =====

rbf_svc_ova = SVC(kernel='rbf', decision_function_shape='ovr',
random_state=5805)

accuracies = []

for train, validation in skf.split(X_train, y_train):
    # Using the indices to split the data into train (2) and
validation
    XV_train = X_train.iloc[train, :]
    XV_valid = X_train.iloc[validation, :]
    yv_train = y_train.iloc[train, :]
    yv_valid = y_train.iloc[validation, :]

    # Train the model
    rbf_svc_ova.fit(XV_train, yv_train)

    # Predict on validation set
    yv_pred = rbf_svc_ova.predict(XV_valid)

    # Evaluate the model
    accuracy = accuracy_score(yv_valid, yv_pred)
    accuracies.append(accuracy)
    print(f'Fold accuracy: {accuracy:.5f}')

# Print avg accuracy across all 5 folds:
print(f"\nAvg accuracy: {sum(accuracies)/len(accuracies):.5f}")

rbf_svc_ova.fit(X_train, y_train)
y_pred_train = rbf_svc_ova.predict(X_train)
y_pred = rbf_svc_ova.predict(X_test)

print(f'Accuracy on train: %.5f' % round(accuracy_score(y_train,
y_pred_train), 2))
accuracy_log_model = round(accuracy_score(y_test, y_pred), 2)
print(f'Accuracy on test: %.5f' % accuracy_log_model)

```

```

# RESULTS:-
plt.figure(figsize=(22, 17))
model = rbf_svc_ova
cnf_mtx = ConfusionMatrixDisplay.from_estimator(model, X_test,
y_test, cmap=plt.cm.Blues)
fig, ax = plt.subplots(figsize=(10,10))
#cnf_mtx.plot(ax=ax, cmap=plt.cm.Blues)
# Deactivate default colorbar
cnf_mtx.plot(ax=ax, colorbar=False, cmap=plt.cm.Blues)
plt.xticks(rotation=45, ha='right')
# Adding custom colorbar
cax =
fig.add_axes([ax.get_position().x1+0.01,ax.get_position().y0,0.02,ax
.get_position().height])
plt.colorbar(cnf_mtx.im_, cax=cax)
plt.tight_layout()
plt.show()

y_proba = label_binarize(y_pred, classes=model.classes_)
y_test_bin = label_binarize(y_test, classes=model.classes_)

df_proba = pd.DataFrame(y_proba, columns=model.classes_)
df_testbin = pd.DataFrame(y_test_bin, columns=model.classes_)
df_proba.to_csv(r'..\data\classification\y_proba_+'rbf_svc_ova.csv',
index=False)
df_testbin.to_csv(r'..\data\classification\y_testbin_+'rbf_svc_ova.c
sv', index=False)

display = RocCurveDisplay.from_predictions(
    y_test_bin.ravel(),
    y_proba.ravel(),
    name="SVC RBF Kernel (OvR) micro-average OvR",
    color="darkorange",
    plot_chance_level=True,
    #despine=True,
)
_ = display.ax_.set(
    xlabel="False Positive Rate",
    ylabel="True Positive Rate",
    title="SVC RBF Kernel (OvR)\nMicro-averaged One-vs-
Rest\nReceiver Operating Characteristic",
)
plt.show()

micro_roc_auc_ovr = roc_auc_score(
    y_test_bin,
    y_proba,
    multi_class="ovr",
    average="micro",
)
print(f"Micro-averaged One-vs-Rest ROC AUC
score:\n{micro_roc_auc_ovr:.5f}")

precision = precision_score(y_test, y_pred, average='micro')

```

```

print(f"Micro-averaged Precision score:\n{precision:.5f}")

recall = recall_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Recall score:\n{recall:.5f}")

specificity = specificity_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Specificity score:\n{specificity:.5f}")

f1 = f1_score(y_test, y_pred, average='micro')
print(f"Micro-averaged F1 score:\n{f1:.5f}")

# ===== Support Vector Machine (Polynomial Kernel, One vs All)
=====

poly_svc_ova = SVC(kernel='poly', decision_function_shape='ovr',
random_state=5805)

accuracies = []

for train, validation in skf.split(X_train, y_train):
    # Using the indices to split the data into train (2) and
    validation
    XV_train = X_train.iloc[train, :]
    XV_valid = X_train.iloc[validation, :]
    yv_train = y_train.iloc[train, :]
    yv_valid = y_train.iloc[validation, :]

    # Train the model
    poly_svc_ova.fit(XV_train, yv_train)

    # Predict on validation set
    yv_pred = poly_svc_ova.predict(XV_valid)

    # Evaluate the model
    accuracy = accuracy_score(yv_valid, yv_pred)
    accuracies.append(accuracy)
    print(f'Fold accuracy: {accuracy:.5f}')

# Print avg accuracy across all 5 folds:
print(f"\nAvg accuracy: {sum(accuracies)/len(accuracies):.5f}")

poly_svc_ova.fit(X_train, y_train)
y_pred_train = poly_svc_ova.predict(X_train)
y_pred = poly_svc_ova.predict(X_test)

print(f'Accuracy on train: %.5f' % round(accuracy_score(y_train,
y_pred_train), 2))
accuracy_log_model = round(accuracy_score(y_test, y_pred), 2)
print(f'Accuracy on test: %.5f' % accuracy_log_model)

# RESULTS:-
#plt.figure(figsize=(22, 17))
model = poly_svc_ova
cnf_mtx = ConfusionMatrixDisplay.from_estimator(model, X_test,

```



```

y_test, cmap=plt.cm.Blues)
fig, ax = plt.subplots(figsize=(10,10))
#cnf_mtx.plot(ax=ax, cmap=plt.cm.Blues)
# Deactivate default colorbar
cnf_mtx.plot(ax=ax, colorbar=False, cmap=plt.cm.Blues)
plt.xticks(rotation=45, ha='right')
# Adding custom colorbar
cax =
fig.add_axes([ax.get_position().x1+0.01,ax.get_position().y0,0.02,ax
.get_position().height])
plt.colorbar(cnf_mtx.im_, cax=cax)
#plt.tight_layout()
plt.show()

y_proba = label_binarize(y_pred, classes=model.classes_)
y_test_bin = label_binarize(y_test, classes=model.classes_)

df_proba = pd.DataFrame(y_proba, columns=model.classes_)
df_testbin = pd.DataFrame(y_test_bin, columns=model.classes_)
df_proba.to_csv(r'.\data\classification\y_proba_+'poly_svc_ova.csv'
, index=False)
df_testbin.to_csv(r'.\data\classification\y_testbin_+'poly_svc_ova.
csv', index=False)

display = RocCurveDisplay.from_predictions(
    y_test_bin.ravel(),
    y_proba.ravel(),
    name="SVC Polynomial Kernel (OvR) micro-average OvR",
    color="darkorange",
    plot_chance_level=True,
    #despine=True,
)
_ = display.ax_.set(
    xlabel="False Positive Rate",
    ylabel="True Positive Rate",
    title="SVC Polynomial Kernel (OvR)\nMicro-averaged One-vs-
Rest\nReceiver Operating Characteristic",
)
plt.show()

micro_roc_auc_ovr = roc_auc_score(
    y_test_bin,
    y_proba,
    multi_class="ovr",
    average="micro",
)
print(f"Micro-averaged One-vs-Rest ROC AUC
score:\n{micro_roc_auc_ovr:.5f}")

precision = precision_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Precision score:\n{precision:.5f}")

recall = recall_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Recall score:\n{recall:.5f}")

```

```
specificity = specificity_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Specificity score:\n{specificity:.5f}")

f1 = f1_score(y_test, y_pred, average='micro')
print(f"Micro-averaged F1 score:\n{f1:.5f}")

# ===== END =====
```

03c-classifiers-part3

```
# ***** Final Term Project *****

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df_train = pd.read_csv(r'./data/df_train.csv')
df_test = pd.read_csv(r'./data/df_test.csv')

X_train = df_train[[x for x in df_train.columns if x !=
'Attack_type']]
y_train = df_train[['Attack_type']]
X_test = df_test[[x for x in df_test.columns if x != 'Attack_type']]
y_test = df_test[['Attack_type']]

# ===== Naïve Bayes =====

from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score, recall_score, roc_curve,
roc_auc_score, auc, RocCurveDisplay

skf = StratifiedKFold(n_splits=5, random_state=5805, shuffle=True)

gnb = GaussianNB()

accuracies = []

for train, validation in skf.split(X_train, y_train):
    # Using the indices to split the data into train (2) and
    validation
    XV_train = X_train.iloc[train, :]
    XV_valid = X_train.iloc[validation, :]
    yv_train = y_train.iloc[train, :]
    yv_valid = y_train.iloc[validation, :]

    # Train the model
    gnb.fit(XV_train, yv_train)

    # Predict on validation set
    yv_pred = gnb.predict(XV_valid)
```

```

    # Evaluate the model
    accuracy = accuracy_score(yv_valid, yv_pred)
    accuracies.append(accuracy)
    print(f'Fold accuracy: {accuracy:.5f}')

# Print avg accuracy across all 5 folds:
print(f"\nAvg accuracy: {sum(accuracies)/len(accuracies):.5f}")

gnb.fit(X_train, y_train)
y_pred_train = gnb.predict(X_train)
y_pred = gnb.predict(X_test)

print(f'Accuracy on train: %.5f' % round(accuracy_score(y_train,
y_pred_train), 2))
accuracy_log_model = round(accuracy_score(y_test, y_pred), 2)
print(f'Accuracy on test: %.5f' % accuracy_log_model)

# RESULTS:-
from sklearn.preprocessing import label_binarize
from sklearn.metrics import RocCurveDisplay
from sklearn.metrics import precision_score, recall_score, f1_score
from imblearn.metrics import specificity_score
from sklearn.metrics import ConfusionMatrixDisplay

#plt.figure(figsize=(22, 17))
model = gnb
cnf_mtx = ConfusionMatrixDisplay.from_estimator(model, X_test,
y_test, cmap=plt.cm.Blues)
fig, ax = plt.subplots(figsize=(10,10))
#cnf_mtx.plot(ax=ax, cmap=plt.cm.Blues)
# Deactivate default colorbar
cnf_mtx.plot(ax=ax, colorbar=False, cmap=plt.cm.Blues)
plt.xticks(rotation=45, ha='right')
# Adding custom colorbar
cax =
fig.add_axes([ax.get_position().x1+0.01,ax.get_position().y0,0.02,ax
.get_position().height])
plt.colorbar(cnf_mtx.im_, cax=cax)
#plt.tight_layout()
plt.show()

y_proba = label_binarize(y_pred, classes=model.classes_)
y_test_bin = label_binarize(y_test, classes=model.classes_)

df_proba = pd.DataFrame(y_proba, columns=model.classes_)
df_testbin = pd.DataFrame(y_test_bin, columns=model.classes_)
df_proba.to_csv(r'.\data\classification\y_proba_'+ 'gaussian_naive_ba
yes.csv', index=False)
df_testbin.to_csv(r'.\data\classification\y_testbin_'+ 'gaussian_naiv
e_bayes.csv', index=False)

display = RocCurveDisplay.from_predictions(
    y_test_bin.ravel(),
    y_proba.ravel(),

```

```

        name="Naïve Bayes micro-average OvR",
        color="darkorange",
        plot_chance_level=True,
        #despine=True,
    )
    _ = display.ax_.set(
        xlabel="False Positive Rate",
        ylabel="True Positive Rate",
        title="Naïve Bayes\nMicro-averaged One-vs-Rest\nReceiver\nOperating Characteristic",
    )
plt.show()

micro_roc_auc_ovr = roc_auc_score(
    y_test_bin,
    y_proba,
    multi_class="ovr",
    average="micro",
)
print(f"Micro-averaged One-vs-Rest ROC AUC\nscore:\n{micro_roc_auc_ovr:.5f}")

precision = precision_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Precision score:\n{precision:.5f}")

recall = recall_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Recall score:\n{recall:.5f}")

specificity = specificity_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Specificity score:\n{specificity:.5f}")

f1 = f1_score(y_test, y_pred, average='micro')
print(f"Micro-averaged F1 score:\n{f1:.5f}")

# ===== k-Nearest Neighbours =====
from sklearn.neighbors import KNeighborsClassifier

k_values = range(1, 30)
error_rates = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    error_rate = 1 - accuracy_score(y_test, y_pred)
    error_rates.append(error_rate)

plt.figure(figsize=(8, 6))
plt.plot(k_values, error_rates, marker='o')
plt.xlabel('K Value')
plt.ylabel('Error Rate')
plt.title('Elbow Method for Finding Optimal K')
plt.axvline(x=1, linestyle='--', color='red', label="Optimal K\n(Elbow Point)")

```

```

plt.legend()
plt.show()

knn = KNeighborsClassifier(n_neighbors=1)

accuracies = []

for train, validation in skf.split(X_train, y_train):
    # Using the indices to split the data into train (2) and
    validation
    XV_train = X_train.iloc[train, :]
    XV_valid = X_train.iloc[validation, :]
    yv_train = y_train.iloc[train, :]
    yv_valid = y_train.iloc[validation, :]

    # Train the model
    knn.fit(XV_train, yv_train)

    # Predict on validation set
    yv_pred = knn.predict(XV_valid)

    # Evaluate the model
    accuracy = accuracy_score(yv_valid, yv_pred)
    accuracies.append(accuracy)
    print(f'Fold accuracy: {accuracy:.5f}')

# Print avg accuracy across all 5 folds:
print(f"\nAvg accuracy: {sum(accuracies)/len(accuracies):.5f}")

knn.fit(X_train, y_train)
y_pred_train = knn.predict(X_train)
y_pred = knn.predict(X_test)

print(f'Accuracy on train: %.5f' % round(accuracy_score(y_train,
y_pred_train), 2))
accuracy_log_model = round(accuracy_score(y_test, y_pred), 2)
print(f'Accuracy on test: %.5f' % accuracy_log_model)

# RESULTS:-
plt.figure(figsize=(22, 17))
model = knn
cnf_mtx = ConfusionMatrixDisplay.from_estimator(model, X_test,
y_test, cmap=plt.cm.Blues)
fig, ax = plt.subplots(figsize=(10,10))
#cnf_mtx.plot(ax=ax, cmap=plt.cm.Blues)
# Deactivate default colorbar
cnf_mtx.plot(ax=ax, colorbar=False, cmap=plt.cm.Blues)
plt.xticks(rotation=45, ha='right')
# Adding custom colorbar
cax =
fig.add_axes([ax.get_position().x1+0.01, ax.get_position().y0, 0.02, ax
.get_position().height])
plt.colorbar(cnf_mtx.im_, cax=cax)
plt.tight_layout()

```

```

plt.show()

y_proba = label_binarize(y_pred, classes=model.classes_)
y_test_bin = label_binarize(y_test, classes=model.classes_)

df_proba = pd.DataFrame(y_proba, columns=model.classes_)
df_testbin = pd.DataFrame(y_test_bin, columns=model.classes_)
df_proba.to_csv(r'..\data\classification\y_proba_'+k_nearest_neighbo
urs.csv', index=False)
df_testbin.to_csv(r'..\data\classification\y_testbin_'+k_nearest_nei
ghbours.csv', index=False)

display = RocCurveDisplay.from_predictions(
    y_test_bin.ravel(),
    y_proba.ravel(),
    name="k-Nearest Neighbours micro-average OvR",
    color="darkorange",
    plot_chance_level=True,
    #despine=True,
)
_ = display.ax_.set(
    xlabel="False Positive Rate",
    ylabel="True Positive Rate",
    title="k-Nearest Neighbours\nMicro-averaged One-vs-
Rest\nReceiver Operating Characteristic",
)
plt.show()

micro_roc_auc_ovr = roc_auc_score(
    y_test_bin,
    y_proba,
    multi_class="ovr",
    average="micro",
)
print(f"Micro-averaged One-vs-Rest ROC AUC
score:\n{micro_roc_auc_ovr:.5f}")

precision = precision_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Precision score:\n{precision:.5f}")

recall = recall_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Recall score:\n{recall:.5f}")

specificity = specificity_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Specificity score:\n{specificity:.5f}")

f1 = f1_score(y_test, y_pred, average='micro')
print(f"Micro-averaged F1 score:\n{f1:.5f}")

# ===== Neural Network (3-layer) =====

from tensorflow.keras import Sequential
from tensorflow.keras.layers import InputLayer
from tensorflow.keras.layers import Dense

```

```

import tensorflow as tf
#from tensorflow.keras.utils import to_categorical

#y_train_enc = to_categorical(y_train, 12)
#y_test_enc = to_categorical(y_test, 12)

from sklearn.preprocessing import OneHotEncoder

oneHotEnc = OneHotEncoder()
oneHotEnc.fit(y_train)
y_train_enc = oneHotEnc.transform(y_train).toarray()
y_test_enc = oneHotEnc.transform(y_test).toarray()
df_train_enc = pd.DataFrame(y_train_enc,
                             columns=oneHotEnc.categories_)
df_test_enc = pd.DataFrame(y_test_enc,
                            columns=oneHotEnc.categories_)

# Step 1: Define the neural network architecture
# Step 2: Instantiate a model of the Keras Sequential() class
ANN_model = Sequential()
# Step 3: Input Layer + 1st Hidden Layer
ANN_model.add(Dense(units=10, activation='relu', input_shape=(10,)))
# Step 3.5: Hidden Layers
ANN_model.add(Dense(units=8, activation='relu'))
ANN_model.add(Dense(units=7, activation='relu'))
# Step 4: Output Layer
ANN_model.add(Dense(12, activation = 'sigmoid'))
#=====
# Step 5: Compile the model
#=====
optimizer = tf.keras.optimizers.Adam(learning_rate = 0.05)
ANN_model.compile(optimizer = optimizer,
                  loss = 'categorical_crossentropy',
                  # 'binary_crossentropy',
                  metrics = ['accuracy'])
ANN_model.summary()

accuracies = []

for train, validation in skf.split(X_train, y_train):
    # Using the indices to split the data into train (2) and
    validation
    XV_train = X_train.iloc[train, :]
    XV_valid = X_train.iloc[validation, :]
    yv_train = df_train_enc.iloc[train, :]
    yv_valid = df_train_enc.iloc[validation, :]

    # Train the model
    ANN_model.fit(XV_train, yv_train)

    # Predict on validation set
    yv_pred = ANN_model.predict(XV_valid)

    yv_valid_cat = oneHotEnc.inverse_transform(yv_valid)

```

```

    yv_pred_cat = oneHotEnc.inverse_transform(yv_pred)

    # Evaluate the model
    accuracy = accuracy_score(yv_valid_cat, yv_pred_cat)
    accuracies.append(accuracy)
    print(f'Fold accuracy: {accuracy:.5f}')

# Print avg accuracy across all 5 folds:
print(f"\nAvg accuracy: {sum(accuracies)/len(accuracies):.5f}")

'''
# Step 6: Fit the model
history = ANN_model.fit(X_train, y_train_enc,
                        epochs = 5,
                        batch_size = 32,
                        validation_split = 0.2,
                        shuffle = False)

# Step 7:
# Plot the performance of the model during training
# Plot training and validation accuracy scores
# against the number of epochs.
plt.plot(history.history['accuracy'], label = 'train')
plt.plot(history.history['val_accuracy'], label = 'validation')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.title('Model Accuracy')
plt.legend(loc='upper left')
plt.show()

# Plot training and validation loss scores
# against the number of epochs.
plt.plot(history.history['loss'], label = 'train')
plt.plot(history.history['val_loss'], label = 'validation')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.title('Model Accuracy')
plt.legend(loc='upper right')
plt.show()

# Step 8: Evaluate the model on the test data
test_loss, test_acc = ANN_model.evaluate(X_test, y_test_enc)
print(f"Test loss : {test_loss*100:.5f}%")
print(f"Test accuracy : {test_acc*100:.5f}%")
'''

y_pred = ANN_model.predict(X_test)
y_pred_cat = oneHotEnc.inverse_transform(y_pred)

# RESULTS:-
plt.figure(figsize=(22, 17))
model = ANN_model
#cnf_mtx = ConfusionMatrixDisplay.from_estimator(model, X_test,
y_test, cmap=plt.cm.Blues)
cnf_mtx = ConfusionMatrixDisplay.from_predictions(y_test,
y_pred_cat, cmap=plt.cm.Blues)

```



```

fig, ax = plt.subplots(figsize=(10,10))
#cnf_mtx.plot(ax=ax, cmap=plt.cm.Blues)
# Deactivate default colorbar
cnf_mtx.plot(ax=ax, colorbar=False, cmap=plt.cm.Blues)
plt.xticks(rotation=45, ha='right')
# Adding custom colorbar
cax =
fig.add_axes([ax.get_position().x1+0.01,ax.get_position().y0,0.02,ax
.get_position().height])
plt.colorbar(cnf_mtx.im_, cax=cax)
#plt.tight_layout()
plt.show()

y_proba = y_pred
y_test_bin = y_test_enc
y_pred = y_pred_cat

df_proba = pd.DataFrame(y_proba, columns=oneHotEnc.categories_)
df_testbin = pd.DataFrame(y_test_bin, columns=oneHotEnc.categories_)
df_proba.to_csv(r'.\data\classification\y_proba_'+ 'artificial_neural
_net.csv', index=False)
df_testbin.to_csv(r'.\data\classification\y_testbin_'+ 'artificial_ne
ural_net.csv', index=False)

display = RocCurveDisplay.from_predictions(
    y_test_bin.ravel(),
    y_proba.ravel(),
    name="k-Nearest Neighbours micro-average OvR",
    color="darkorange",
    plot_chance_level=True,
    #despine=True,
)
_ = display.ax_.set(
    xlabel="False Positive Rate",
    ylabel="True Positive Rate",
    title="Neural net (4 layers)\nMicro-averaged One-vs-
Rest\nReceiver Operating Characteristic",
)
plt.show()

micro_roc_auc_ovr = roc_auc_score(
    y_test_bin,
    y_proba,
    multi_class="ovr",
    average="micro",
)
print(f"Micro-averaged One-vs-Rest ROC AUC
score:\n{micro_roc_auc_ovr:.5f}")

precision = precision_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Precision score:\n{precision:.5f}")

recall = recall_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Recall score:\n{recall:.5f}")

```

```

specificity = specificity_score(y_test, y_pred, average='micro')
print(f"Micro-averaged Specificity score:\n{specificity:.5f}")

f1 = f1_score(y_test, y_pred, average='micro')
print(f"Micro-averaged F1 score:\n{f1:.5f}")

# ===== END =====

```

03d-classifiers-part4

```

# ***** Final Term Project *****

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

y_proba_dtrees =
pd.read_csv(r'./data/classification/y_proba_dtrees.csv')
y_testbin_dtrees =
pd.read_csv(r'./data/classification/y_testbin_dtrees.csv')
y_proba_logR_OvR =
pd.read_csv(r'./data/classification/y_proba_logR_OvR.csv')
y_testbin_logR_OvR =
pd.read_csv(r'./data/classification/y_testbin_logR_OvR.csv')
y_proba_logR_OvO =
pd.read_csv(r'./data/classification/y_proba_logR_OvO.csv')
y_testbin_logR_OvO =
pd.read_csv(r'./data/classification/y_testbin_logR_OvO.csv')
y_proba_rbf_svc_ovo =
pd.read_csv(r'./data/classification/y_proba_rbf_svc_ovo.csv')
y_testbin_rbf_svc_ovo =
pd.read_csv(r'./data/classification/y_testbin_rbf_svc_ovo.csv')
y_proba_rbf_svc_ova =
pd.read_csv(r'./data/classification/y_proba_rbf_svc_ova.csv')
y_testbin_rbf_svc_ova =
pd.read_csv(r'./data/classification/y_testbin_rbf_svc_ova.csv')
y_proba_gaussian_naive_bayes =
pd.read_csv(r'./data/classification/y_proba_gaussian_naive_bayes.csv')
y_testbin_gaussian_naive_bayes =
pd.read_csv(r'./data/classification/y_testbin_gaussian_naive_bayes.csv')
y_proba_k_nearest_neighbours =
pd.read_csv(r'./data/classification/y_proba_k_nearest_neighbours.csv')
y_testbin_k_nearest_neighbours =
pd.read_csv(r'./data/classification/y_testbin_k_nearest_neighbours.csv')
y_proba_artificial_neural_net =
pd.read_csv(r'./data/classification/y_proba_artificial_neural_net.csv')
y_testbin_artificial_neural_net =

```

```

pd.read_csv(r'./data/classification/y_testbin_artificial_neural_net.
csv')

clf_predictions_dict = {'Decision Tree':[y_proba_dtrees3,
y_testbin_dtrees3],
                        'Logistic Regression
(OvO)':[y_proba_logR_OvO, y_testbin_logR_OvO],
                        'Logistic Regression
(OvR)':[y_proba_logR_OvR, y_testbin_logR_OvR],
                        'RBF SVC (OvO)':[y_proba_rbf_svc_ovo,
y_testbin_rbf_svc_ovo],
                        'RBF SVC (OvR)':[y_proba_rbf_svc_ova,
y_testbin_rbf_svc_ova],
                        'Naïve Bayes':[y_proba_gaussian_naive_bayes,
y_testbin_gaussian_naive_bayes],
                        'K Nearest
Neighbours':[y_proba_k_nearest_neighbours,
y_testbin_k_nearest_neighbours],
                        'Artificial
Neuralnet':[y_proba_artificial_neural_net,
y_testbin_artificial_neural_net]}

# ===== Grand ROC Curve =====

from itertools import cycle
from sklearn.metrics import accuracy_score, recall_score, roc_curve,
roc_auc_score, auc, RocCurveDisplay

fig, ax = plt.subplots(figsize=(8, 8))

colors = ["aqua", "darkorange", "cornflowerblue", "indianred",
"forestgreen", "lime", "gold", "fuchsia"]

for class_id in range(len(clf_predictions_dict.keys())):
    RocCurveDisplay.from_predictions(

clf_predictions_dict[list(clf_predictions_dict.keys())[class_id]][1]
.to_numpy().ravel(),

clf_predictions_dict[list(clf_predictions_dict.keys())[class_id]][0]
.to_numpy().ravel(),
        name=f"ROC curve for
{list(clf_predictions_dict.keys())[class_id]}",
        color=colors[class_id],
        ax=ax,
        plot_chance_level=(class_id == 2),
        #despine=True,
    )

_ = ax.set(
    xlabel="False Positive Rate",
    ylabel="True Positive Rate",
    title="Receiver Operating Characteristic\nto One-vs-Rest
multiclass",

```

```
)  
plt.show()  
# ===== END =====
```